Bio-O-Ton

Pipeline-Handbuch und Fehlerreferenz

READMEs und Betriebsanleitungen / Deutsch und Englisch

Dokumentationsstand: 12. Oktober 2026

52 Quelldokumente. Separater Kern- und Bioakustiklauf, Master-Schema v7, Schritt-Diagnosen und Vorbereitung fuer Horeka 2.

Dieses Handbuch enthaelt den Schemaabgleich mit der ci-tec-PDF vom 09.10.2026: Zeitquellen-Flag, gepruefte Audiodauer und informative Dauer-/Wetterhinweise. Clean-Filter werden erst in der Analyse angewendet; alle Masterzeilen bleiben erhalten. Ausfuehrungs- und Veroeffentlichungsnachweise stehen im jeweiligen Laufbericht; eine Horeka-2-Migration wird hier nicht als ausgefuehrt dargestellt. Historische Audit-Inhalte bleiben als solche gekennzeichnet.

Inhaltsverzeichnis

01 Bio-O-Ton Data Pipeline 5

02 Master table 15

03 Bio-O-Ton Master Table Reference 19

04 Bio-O-Ton Documentation Index 27

05 Zwei getrennte Pipeline-Läufe 28

06 Weniger Slurm-Jobs 30

07 Gemeinsame Master und sichere Wiederaufnahme 31

08 Dokumentationspruefung 32

09 Master-Schema: Abgleich mit ci-tec-PDF vom 09.10.2026 33

10 Bio-O-Ton Pipeline: Schritte, Dateien und Abhaengigkeiten 35

11 Bio-O-Ton: lokaler Windows-Lauf mit LSDF 42

12 HoreKa Recovery-Runbook 47

13 Earth-Engine-Schlüssel für unbeaufsichtigte Sentinel-Läufe 49

14 GitHub-Workflow 50

15 Nomenclature Migration 51

16 Pipeline-Steuerung (DE) 52

17 Pipeline Control (EN) 55

18 Step 1 Metadata Extraction (DE) 58

19 Step 1 Metadata Extraction (EN) 62

20 Step 2_0 LRT Cleaning (DE) 66

21 Step 2_0 LRT Cleaning (EN) 69

22 Step 2_1 100m Formation Status (DE) 72

23 Step 2_1 100m Formation Status (EN) 75

24 Step 2_2 Point Assignment (DE) 78

25 Step 2_2 Point Assignment (EN) 81

26 Step 2_3 Grid Aggregation (DE) 84

27 Step 2_3 Grid Aggregation (EN) 87

28 Step 2_4 10m Formation Status (DE) 89

29 Step 2_4 10m Formation Status (EN) 92

30 Step 2 Varianten - LRT-Sensitivitaetsanalyse 95

31 Step 2 Variants - LRT Sensitivity Analysis 99

32 Step 3 Media Inventory And Download (DE) 102

33 Step 3 Media Inventory And Download (EN) 108

34 Step 4 Sentinel-2 Mirror And Inventory (DE) 114

35 Step 4 Sentinel-2 Mirror And Inventory (EN) 121

36 Step 5_2 HOSTRADA Weather Per Recording (DE) 128

37 Step 5_2 HOSTRADA Weather Per Recording (EN) 132

38 Step 5_3 HOSTRADA Monatsdaten (DE) 136

39 Step 5_3 HOSTRADA Monthly Data (EN) 137

40 Step 5_4 HOSTRADA 100m Raster (DE) 138

41 Step 5_4 HOSTRADA 100m Rasters (EN) 140

42 Step 5_5 HOSTRADA Raster-QC (DE) 142

43 Step 5_5 HOSTRADA Raster QC (EN) 144

44 Step 6 - Bioakustische Embeddings und Arteninferenz 146

45 Step 6 - Bioacoustic Embeddings and Species Inference 153

46 Step 7_0 - Finale Mastertabelle 158

47 Step 7_0 - Final Master Table 164

48 Step 9 Visuelle Reports (DE) 170

49 Step 9 Visual Reports (EN) 171

50 Zeitkonflikte und LRT-Zellen 172

51 Validation And Formation-Status Comparison (DE) 173

52 Validation And Formation-Status Comparison (EN) 176

01 Bio-O-Ton Data Pipeline

README.md

Bio-O-Ton is a restartable data-processing pipeline for Dawn Chorus recordings. It combines cleaned recording metadata, habitat-formation products at 100 m and 10 m, media inventories and downloads, Sentinel-2 data, HOSTRADA weather, bioacoustic model results, validation, and visual reports. Production runs use LSDF storage and Slurm on HoreKa; a Windows orchestrator is available for controlled local execution.

This document reflects the implementation checked on **2026-09-24**. Paths and operational defaults come from `config.horeka.json`, the executable step registry is `pipeline_steps.json`, and the final table is written by `scripts/Step_7_0_update_master_table.py`. The master reconciliation and restart checks explain independent spatial coverage checks, all-variant processing and protection against a truncated source. The final workflow uses `tools/finalize_master.py` to reconcile recording and variant status together.

For ci-tec, use the short README_CI_TEC.pdf (text version). The time review documents the German-time correction and the LRT check of all 47 source conflicts.

The complete README_Paket.pdf combines the maintained manuals and the per-step diagnostic reference, including the ci-tec agreement checklist.

Master table at a glance

The main result is one row per unique `dawn_chorus_id` in:

```
/lsdf/kit/ipf/projects/Bio-O-Ton/Data_automatisation_skripts/outputs/
├── Bio_O_Ton_Master.csv
├── Bio_O_Ton_Master.parquet
└── Bio_O_Ton_Master_summary.json
```

The CSV is the portable exchange format, the compressed Parquet file is intended for analysis, and the JSON file contains row counts, readiness totals, status distributions, and update metadata. The current schema version is `2026-10-09-mastertable-v7`. Detailed definitions, derivation rules, issue-code conventions, and source products are in the master-table reference.

The 9 October PDF contract is implemented by the shared Horeka/local writer: `time_has_issues` is false only for source `localtimes`; `duration_seconds` is retained from audio QC. Weather `missing_value` alone remains an informational code. Schema audit and validation.

The master keeps every recording in the source population, including fallback times, short audio and flagged products. Clean-data filters are applied later by the consumer; the suggested `duration_seconds >= 40` is not a row-selection rule in either pipeline.

The five proximity-review fields are defined in the master reference.

The compact dictionary below covers **the main column groups written by Step 7_0**. Related fields are grouped to keep the overview readable; every group links to its full definition.

Column(s)	Short description	Details
`mastertable_schema_version`	Version of the row contract.	Identity and provenance
`workflow_run_id`	Workflow that most recently updated the row.	Identity and provenance
`dawn_chorus_id`	Unique numeric recording identifier and primary key.	Identity and provenance
`source_fingerprint`	SHA-256 fingerprint of relevant source fields.	Identity and provenance
`datetime_local`, `datetime_utc`, `date_local`, `time_local`	Clean local/UTC recording time and convenient date/time components.	Time and coordinates
`timestamp_source`, `timestamp_changed`, `timestamp_change_reason`	Origin and normalization history of the timestamp.	Time and coordinates
`lat`, `lon`	Cleaned WGS84 recording coordinates.	Time and coordinates

Column(s)	Short description	Details
record_added_to_mastertable_utc, record_updated_in_mastertable_utc	First insertion and most recent row-update times.	Identity and provenance
metadata_status	Canonical validity status for time and coordinates.	Status fields
sound_exists, sound_has_issues, sound_issue_codes, sound_status	Audio presence, QC result, compact causes, and canonical status.	Audio and photos
photo_exists, photo_has_issues, photo_issue_codes, photo_status	Photo presence, QC result, compact causes, and canonical status.	Audio and photos
sentinel_exists, sentinel_has_issues, sentinel_quality_score, sentinel_issue_codes, sentinel_status	Sentinel-2 presence, technical QC, optional score, causes, and status.	Sentinel-2
weather_point_exists, weather_point_has_issues, weather_point_issue_codes, weather_point_status	Per-recording HOSTRADA file presence, QC, causes, and status.	Weather
weather_raster_hostrada_100m_exists, weather_raster_hostrada_100m_has_issues, weather_raster_hostrada_100m_issue_codes	Global availability and QC state of optional 100 m weather rasters.	Weather
grid_100m_id, grid_100m_assignment_exists, grid_100m_has_majority_formation	Assigned 100 m cell and availability of its majority formation.	100 m formation
inside_lrt_polygon, lrt_polygon_count	Whether and how often the point directly intersects cleaned LRT polygons.	Direct LRT intersections
lrt_code_count, lrt_formation_count, lrt_status_count, lrt_mapping_year_count	Counts of distinct attributes among direct polygon hits.	Direct LRT intersections
lrt_codes, lrt_formations, lrt_conservation_statuses, lrt_mapping_years	Aggregated attributes of directly intersected polygons.	Direct LRT intersections
majority_formation_100m, majority_formation_status_100m	Dominant 100 m formation and its dominant conservation status.	100 m formation
majority_value_100m, second_value_100m, majority_delta_100m, majority_disputed_100m	Dominance metrics in centi-percent and the two-percentage-point dispute flag.	100 m formation
formation_100m_status	Canonical 100 m assignment/formation status.	100 m formation
grid_10m_id, grid_10m_assignment_exists, grid_10m_has_majority_formation	Derived 10 m cell and availability of its majority formation.	10 m formation
majority_formation_10m, majority_formation_status_10m	Dominant 10 m formation and its dominant conservation status.	10 m formation
majority_value_10m, second_value_10m, majority_delta_10m, majority_disputed_10m	10 m dominance metrics in centi-percent and dispute flag.	10 m formation
formation_10m_status	Canonical 10 m assignment/formation status.	10 m formation
formation_100m_10m_agree, formation_status_100m_10m_agree	Agreement between 100 m and 10 m formation and status.	Cross-resolution checks
formation_primary_variant	LRT variant supplying detailed formation fields to the main row.	Variant coverage
formation_variant_count_expected, formation_variants_with_100m_majority, formation_variants_with_10m_majority, formation_variant_products_complete	Expected variants, per-ID coverage, and global product completeness.	Variant coverage

Column(s)	Short description	Details
`bioacoustic_status`, `bioacoustic_has_issues`, `bioacoustic_issue_codes`	Canonical Step-6 outcome and compact failure causes.	Bioacoustics
`bioacoustic_models_expected`, `bioacoustic_models_complete`, `bioacoustic_required_models_complete`	Planned/completed models and required-model completeness.	Bioacoustics
`bioacoustic_inference_version`	Version of the inference/output transformation.	Bioacoustics
`bioacoustic_species_count`, `bird_species_count`, `nonbird_species_count`	Aggregated model-predicted taxon counts; not confirmed observations.	Bioacoustics
`bioacoustic_max_confidence`, `top_species_scientific`, `top_species_model_support`	Highest normalized score, top-ranked taxon, and supporting-model count.	Bioacoustics
`ready_for_general_analysis`	Metadata, audio, point weather, and Sentinel-2 are technically usable.	Readiness
`ready_for_formation_analysis_100m`, `ready_for_formation_analysis_10m`	General readiness plus a majority formation at the named resolution.	Readiness
`ready_for_direct_lrt_analysis`	General readiness plus a direct cleaned-LRT polygon hit.	Readiness
`ready_for_formation_weather_raster_analysis_100m`	100 m formation readiness plus clean optional HOSTRADA rasters.	Readiness
`ready_for_multimodal_analysis`	General readiness plus a technically valid photo.	Readiness
`ready_for_bioacoustic_analysis`	Valid audio plus successful required bioacoustic models; independent of weather/formation.	Readiness
`record_blocking_issue_codes`	Pipe-separated reasons that block one or more readiness paths.	Issues and statuses
`record_status`, `release_status`	Overall technical row status and separate manual release state.	Release model
`manual_review_comment`, `manual_reviewed_by`, `manual_reviewed_utc`	Preserved manual review metadata.	Release model

Quick start on HoreKa

Run all commands from the repository root. The recommended entry point is the persistent hybrid controller:

```
bash run_horeka.sh add_new_ids
```

It starts a detached `tmux` session on the HoreKa login node, creates the run plan and pipeline lock, submits only planned compute work to Slurm, waits for the submitted jobs, and then runs final validation and HTML report generation locally with one CPU before releasing the lock.

Bioacoustics is now a separate run: after the core run finishes, start `bash run_horeka.sh bioacoustics` (or `bash slurm_bioacoustics.sh` for direct Slurm submission). Details: two phases and Horeka-2 preparation.

Monitor it with:

```
tmux ls
tmux attach -t <session-name>
squeue -u "$USER"
```

Before the first data run, create the core environment. Create the Bacpipe environment when you first run the separate bioacoustics phase:

```
bash bootstrap_env.sh
bash bootstrap_bacpipe_env.sh
```

The core bootstrap prefers `micromamba`, `mamba`, or `conda` and falls back to `python3 -m venv`. Its default target is `.venv`. Bioacoustics uses the separate `.venv_bacpipe` environment. A core interpreter may be supplied through `PYTHON`; a Bacpipe interpreter may be supplied through `BIOOTON_BACPIPE_PYTHON`.

Run modes

Mode	Command	Meaning
Incremental production	`bash run_horeka.sh add_new_ids`	Core only: metadata, spatial products, downloads, master and validation. No Step 6 jobs.
Separate bioacoustics	`bash run_horeka.sh bioacoustics`	Start after the core run: all Step 6 stages, checkpoint reuse and bioacoustic master fields.
Logical full rebuild	`bash run_horeka.sh from_scratch`	Recomputes current IDs and core global steps with `--force`; original source files and downloads are not deleted. Incomplete full-run generations resume from successful step markers.
Functionality test	`bash run_horeka.sh functionality_test`	Runs fast import, config, schema, syntax, and regression checks locally through the controller.
Formation comparison	`bash run_horeka.sh formation_compare`	Submits the formation-product comparison as a Slurm job.
Safe submission preview	`bash run_horeka.sh add_new_ids --local-test`	Runs controller-side planning/preflights and prints the Slurm jobs that would be submitted; no Slurm work, final validation, or reports are executed.

Useful controller options:

```
bash run_horeka.sh add_new_ids --session bioton_update
bash run_horeka.sh add_new_ids --foreground
bash run_horeka.sh add_new_ids --update --branch main
```

`--update` explicitly runs `git pull --ff-only` through `update_horeka_from_git.sh`; without it, the checked-out commit is used unchanged.

End-to-end workflow

The production path is:

```
run_horeka.sh
  -> tools/horeka_controller.py
     -> acquire pipeline lock
     -> tools/plan_pipeline_run.py
     -> submit_bio_o_ton_horeka.sh
        -> planned Slurm DAG (core: Steps 1-5; bioacoustics: Step 6)
        -> final Step 7_0 master-table snapshot
     -> wait for all submitted Slurm jobs
     -> tools/final_validation_report.py
     -> tools/generate_pipeline_visual_reports.py
     -> release pipeline lock
```

Most Slurm edges use `afterany`. Downstream jobs therefore start after their inputs stop, even if an upstream job failed, and can record missing/partial products instead of leaving `DependencyNeverSatisfied` jobs behind. Bioacoustic stages that cannot produce a meaningful result after failure use `afterok` where required. The master table, final validation, and visual report are designed to expose the resulting state.

Pipeline step catalog

Step	Purpose	Main implementation	Detailed English documentation
Control	Run planning, locking, manifests, and full-rebuild generation state.	`tools/plan_pipeline_run.py`, `tools/pipeline_lock.py`, `tools/run_with_manifest.py`	Pipeline control
1	Normalize timestamps/coordinates and maintain per-domain fingerprints.	`scripts/Step_1_metadata_extraction.py`	Step 1
2.0	Clean the configured primary LRT source.	`scripts/Step_2_0_clean_lrts.py`	Step 2.0
2.1	Overlay LRT polygons with the 100 m grid and compute formation/status shares.	`scripts/Step_2_1_merge_lrts_and_grid.py`	Step 2.1
2.2	Assign recording points to 100 m cells and direct LRT polygons.	`scripts/Step_2_2_assign_points_to_lrt_grid.py`	Step 2.2
2.3	Aggregate 100 m products to 1 km, 5 km, and 10 km.	`scripts/Step_2_3_generate_remaining_grid_products.py`	Step 2.3
2.4	Derive Susi-compatible 10 m formation/status products.	`scripts/Step_2_4_generate_10m_formation_status_products.py`	Step 2.4
2 variants	Run Steps 2.0-2.4 independently for every `All_Bundeslander_*.gpkg`.	`tools/step2_variants.py`	Step-2 variants
3.0/3.1	Inventory, QC, download, and re-inventory audio and photos.	`scripts/Step_3_0_*`, `scripts/Step_3_1_*`	Step 3
4.1/4.0	Mirror Sentinel-2 GeoTIFFs from Google Drive/GEE outputs, then inventory/QC LSDF files.	`scripts/Step_4_1_Sentinel2_download.py`, `scripts/Step_4_0_Sentinel2_inventory.py`	Step 4
5.1/5.2	Inventory and generate per-recording HOSTRADA weather time series.	`scripts/Step_5_1_Weather_inventory.py`, `scripts/Step_5_2_download_weather_data.py`	Step 5.2
5.3	Mirror HOSTRADA monthly NetCDF source files.	`scripts/Step_5_3_download_hostrada_monthly.py`	Step 5.3
5.4	Convert variable/year combinations to tiled 100 m rasters.	`tools/run_hostrada_raster_all.py`, `scripts/Step_5_4_prepare_hostrada_rasters.py`	Step 5.4
5.5	Recursively inventory and QC HOSTRADA raster products.	`scripts/Step_5_5_check_hostrada_raster_products.py`	Step 5.5
6.0-6.6	Preflight models, prepare work, infer, normalize, filter, aggregate, and QC bioacoustics.	`scripts/Step_6_*.py`	Step 6
7.0	Atomically update the main ID-level master table and append status events.	`scripts/Step_7_0_update_master_table.py`	Step 7.0, column reference
7.1	Build normalized ID-by-LRT-variant tables and summaries.	`scripts/Step_7_1_update_formation_variant_table.py`	Step-2 variants
Validation	Validate current-run manifests, required artifacts, and configured readiness rules.	`tools/final_validation_report.py`	Validation and comparison
9	Generate static per-step HTML dashboards.	`tools/generate_pipeline_visual_reports.py`	Step 9

The English documentation index provides the same step links in a shorter list.

Incremental planning and invalidation

Every phase receives its own `workflow_run_id`. The core planner compares the Dawn Chorus source, per-domain fingerprints, current step state, inventory baselines, configured global inputs, and the previous master table. It writes:

```
outputs/step_0_control/run_plans/<workflow_run_id>/run_plan.json
outputs/step_0_control/run_plans/<workflow_run_id>/metadata_ids.csv
outputs/step_0_control/run_plans/<workflow_run_id>/point_assignment_ids.csv
outputs/step_0_control/run_plans/<workflow_run_id>/audio_ids.csv
outputs/step_0_control/run_plans/<workflow_run_id>/photo_ids.csv
outputs/step_0_control/run_plans/<workflow_run_id>/sentinel_ids.csv
outputs/step_0_control/run_plans/<workflow_run_id>/weather_ids.csv
outputs/step_0_control/run_plans/<workflow_run_id>/bioacoustic_ids.csv
```

Key invalidation behavior:

Change or condition	Work scheduled
New ID or changed time/GPS	Metadata, point assignment, point weather, and coordinate-dependent products for that ID.
Changed audio URL/fingerprint	Audio retry state is reset; download and post-inventory are reconsidered. The separately initiated bioacoustics phase reconciles current audio work keys.
Changed photo URL/fingerprint	Photo retry state is reset and the photo path is reconsidered.
Changed LRT source	Cleaning and all dependent 100 m, point-assignment, aggregation, and 10 m formation products.
Changed grid source	Grid-dependent formation products and point assignment.
Changed Sentinel source metadata	The corresponding Drive file is mirrored again based on file ID, MD5, modified time, and size.
Previous missing/issue state	The affected ID/domain is retried by the next incremental plan.
Healthy, unchanged inventory baseline	Expensive recursive inventory/download work may be skipped.
`from_scratch`	Logical recomputation of current data without deleting sources or original downloads.

An interrupted full rebuild is tracked as a generation:

```
outputs/step_0_control/full_rebuild/current.json
outputs/step_0_control/full_rebuild/<generation_id>/completed_steps/*.json
```

A later `from_scratch` submission of the same incomplete generation queues only steps without successful completion markers.

Checkpoints, batching, and master-table updates

Long-running steps persist restart state rather than relying on a single all-or-nothing job:

Area	Resume mechanism
Step 2.1	Overlay chunk checkpoints.
Step 2.4	10 m Parquet chunks and batch state.
Step 3.1	Persistent retry CSVs, atomic downloads, and `progress.json`.
Step 4.1	Drive/GEE metadata logs and batch state.
Step 5.1	Reuse of unchanged, error-free weather QC rows.
Step 5.2	Per-ID files/state, Slurm shards, verification job, and `progress.json`.
Step 5.3	Existing monthly NetCDF files.
Step 5.4	Per-tile status files and array verification.
Step 5.5	Reuse of unchanged, error-free raster QC rows.
Step 6.2	Per-model/per-shard inference state and batch checkpoints.

Step 7.0 is serialized and uses an atomic file replacement plus a write lock. ID-scoped upstream stages pass `--ids-file`, replacing only affected master rows; global formation or raster changes trigger a full refresh. In direct, non-hybrid Slurm

operation, audio, photo, and point-weather scripts may also request an intermediate master update every **5,000 completed IDs** according to `config.horeka.json`. The hybrid controller disables these in-step updates and submits one dependency-aware final master job.

Formation products and variants

The primary LRT variant is currently `no_K_post2017`, sourced from:

```
Biodiversity_data/Bundeslander/All_Bundeslander/
  All_Bundeslander_no_K_post2017.gpkg
```

Formation and LRT shares in Susi-compatible 100 m/10 m products are integer centi-percent values: `10000` means 100.00%. Formation totals include A/B/C/K. The majority formation status is selected within the majority formation from A/B/C only. A result is marked disputed when `majority_delta <= 200`, i.e. the leading two formations differ by at most two percentage points.

Run every available LRT variant as an isolated sensitivity branch with:

```
bash submit_step2_variants_horeka.sh add_new_ids
```

Use `from_scratch` as the argument to force all variant stages. Outputs are isolated under `outputs/step_2_variants/<suffix>/`; normalized results are written to:

```
outputs/Bio_O_Ton_Formation_Variants.csv
outputs/Bio_O_Ton_Formation_Variants.parquet
outputs/Bio_O_Ton_Formation_Variants_summary.json
outputs/Bio_O_Ton_Variant_Summary.csv
outputs/Bio_O_Ton_Variant_Temporal_Summary.csv
```

The optional public-LRT workflow is separate from the main DAG:

```
bash run_public_lrt_products.sh
bash run_public_lrt_products.sh --force
```

It runs Steps 2.5 and 2.6 using `public_lrt_cleaning` and `public_lrt_grid_merge` from the configuration.

Media, Sentinel-2, and weather downloads

Only domain downloads are written outside `outputs`:

Product	Source	LSDF target
Audio	`audio` URL in the Dawn Chorus source table	`PointData/SoundRecordings/<id>_audio.<ext>`
Photo	`photo` URL in the source table	`PointData/Images_SoundRecordings/<id>_photo.<ext>`
Sentinel-2	Configured Google Drive folder populated by the GEE export path	`PointData/S2/<Drive-filename>.tif`
Point weather	DWD Open Data HOSTRADA	`PointData/Weather/Hostrada/weather_<id>.csv`

Sentinel credentials are checked before a planned Step 4.1 run. Unattended HoreKa execution requires prepared non-interactive credentials/token files; setup is described in `HOREKA_GEE_SECRET_SETUP_DE.md` (German operational note). Manual mirroring and score validation are available through:

```
bash run_sentinel2_mirror.sh
bash run_sentinel_score_validation.sh
```

Step 5.2 currently requests 10 preceding days plus the recording day, yielding an expected 264 hourly rows with temperature, cloud cover, relative humidity, downwelling radiation, wind direction, and wind speed. It uses up to eight Slurm shards with at most four concurrent tasks by default.

Optional HOSTRADA raster workflow

The area-wide Steps 5.3-5.5 are disabled by default with `hostrada_raster_products.enabled: false`. They are intended for deliberate, usually annual updates and do not block general or normal 100 m formation readiness. When present and clean, they enable `ready_for_formation_weather_raster_analysis_100m`.

Set `hostrada_raster_products.enabled` to `true` for a planned pipeline run, or run targeted maintenance commands:

```
bash run_hostrada_raster_year.sh Ta 2017
bash run_hostrada_raster_quality_check.sh
```

Supported variables are `Ta`, `Rh`, `Radiation`, `CloudCover`, `Winddirection`, and `Windspeed`. The Step-5.4 Slurm array defaults to two concurrent tasks, each with 8 CPUs and 32 GB:

```
BIOOTON_STEP54_MAX_CONCURRENT_TASKS=1 bash run_horeka.sh add_new_ids
```

Bioacoustics

Bioacoustics is enabled by default and uses Bacpipe `1.3.1`. CPU inference is the default; GPU resources are optional. The configured required classifier models are BirdNET, Perch v2, AudioProtoPNet, and ConvNeXt BirdSet. Insect66 and NatureBeats are staged as optional embedding models. Predictions are model outputs, not confirmed species observations.

Step 6 performs:

```
6.0 model/checkpoint preflight
 -> 6.1 valid-audio worklist
 -> 6.2 model x shard embeddings/inference array and verification
 -> 6.3 normalized species predictions
 -> 6.4 Germany taxonomy/season filtering
 -> 6.5 per-recording and recording-species aggregation
 -> 6.6 completeness and quality control
```

The default is 64 logical shards per model, distributed over at most four Slurm workers, with 4 CPUs and 24 GB per worker. Each worker runs several logical tasks sequentially. Override resources only after checking model memory and LSDF I/O, for example:

```
BIOOTON_BIOACOUSTICS_CPUS=4 \
BIOOTON_BIOACOUSTICS_MEMORY=24G \
bash run_horeka.sh bioacoustics
```

See the Step-6 reference for models, schemas, checkpoints, and interpretation limits.

Status, validation, release, and reports

Validation reports identify the checked `phase`: core excludes bioacoustic requirements; bioacoustics checks Step 6 and its prepared prerequisites. A direct validation command without `BIOOTON_RUN_PLAN` checks the legacy full scope.

Every wrapped step writes a manifest to:

```
outputs/step_0_manifests/<step_name>/<step_run_id>.json
```

Manifests include workflow/step run IDs, state, timing, Slurm context, Python/package versions, input fingerprints, parameters, log paths, and results. Slurm logs use one UTC submission stamp:

```
outputs/step_0_slurm_logs/<YYYYMMDDTHHMMSSZ>_<job_name>_<job_id>.out
outputs/step_0_slurm_logs/<YYYYMMDDTHHMMSSZ>_<job_name>_<job_id>.err
```

Array logs additionally contain array job/task IDs. Controller state is stored under `outputs/step_0_control/controllers/`.

Step 7 appends ID lifecycle and tracked status/readiness changes to:

```
outputs/step_0_control/status_events.csv
```

Final validation examines only manifests from the current workflow and checks required artifacts plus the configured readiness policy. Defaults are report-only, require general/100 m/10 m readiness checks, do not require weather rasters or bioacoustic readiness, and never approve records automatically (`automatic_release: false`). Reports are written to:

```
outputs/step_9_validation/final_validation_<timestamp>.json
outputs/step_9_validation/final_validation_<timestamp>.md
```

```
outputs/step_9_visual_reports/index.html
```

Run the two post-processing stages manually when needed:

```
bash run_final_validation_report.sh
.venv/bin/python tools/generate_pipeline_visual_reports.py --config config.horeka.json
```

Locking, recovery, and safe reruns

The atomic directory lock prevents two writing workflows from running concurrently. Check it with:

```
PYTHON="$(bash bootstrap_env.sh | tail -n 1)"
"$PYTHON" tools/pipeline_lock.py --config config.horeka.json status
```

If the controller is interrupted, it intentionally leaves Slurm jobs and the lock untouched. Inspect `tmux`, `squeue`, `sacct`, controller state, and logs first. Force-release only after confirming that no writer remains:

```
"$PYTHON" tools/pipeline_lock.py --config config.horeka.json release --force
```

The detailed recovery checklist is in `HOREKA_RECOVERY_RUNBOOK_DE.md` (German operational runbook). Direct Slurm entry points remain available for recovery/backward compatibility:

```
bash slurm_functionality_test.sh
bash slurm_add_new_ids.sh
bash slurm_from_scratch.sh
bash slurm_compare_formation_status.sh
```

These bypass the persistent hybrid controller and call `submit_bio_o_ton_horeka.sh` directly. In this mode, final validation, visual reports, and lock release are themselves Slurm jobs.

For a temporary checkpoint test, cap all requested pipeline walltimes:

```
BIOOTON_PIPELINE_TIME_OVERRIDE=00:30:00 bash slurm_add_new_ids.sh
```

Local Windows workflow

The local orchestrator can run `add_new_ids`, `from_scratch`, `bioacoustics`, or `functionality_test` on Windows while mounting LSDF, generating a path-adjusted configuration, optionally bootstrapping existing HoreKa outputs, and publishing successful outputs back to LSDF. Bioacoustics must now be started separately after the core run; see two phases and cluster migration.

1. Copy/edit `scripts_local_run/local.settings.example.json` as `scripts_local_run/local.settings.json`.

2. Verify the LSDF host/user, mount drive, local workspace/environment directories, CPU limits, and publication settings.

3. Start from PowerShell:

```
powershell -ExecutionPolicy Bypass -File scripts_local_run\run_pipeline_local.ps1 -Mode
functionality_test
powershell -ExecutionPolicy Bypass -File scripts_local_run\run_pipeline_local.ps1 -Mode
add_new_ids
```

Use `-CpuOnly` to disable CUDA preference, `-RefreshHorekaOutputs` to refresh the local baseline, and `-SkipLsdfPublish` to keep successful outputs local. The current local design keeps HOSTRADA raster work on HoreKa (`hostrada_execution: "horeka_only"`). Full local setup, storage/offloading, resume, and publishing behavior is documented in `scripts_local_run/README.md` (currently German).

Configuration and resource overrides

`config.horeka.json` is the production source of paths, thresholds, worker limits, model definitions, and output locations. Do not embed secrets in it; `.secrets/` is ignored. Common runtime overrides include:

Variable	Default	Purpose
`BIOOTON_PARTITION`	`cpuonly`	Core Slurm partition.
`BIOOTON_ACCOUNT`	empty	Optional Slurm account.
`BIOOTON_STEP2_CPUS`	16	Step 2.0/2.1 CPUs.

Variable	Default	Purpose
BIOOTON_STEP3_DOWNLOAD_CPUS	16	Media download CPUs.
BIOOTON_STEP41_CPUS	1	Sentinel mirror CPUs.
BIOOTON_WEATHER_SHARD_COUNT	config: 8	Maximum weather shard count.
BIOOTON_WEATHER_MAX_CONCURRENT_TASKS	config: 4	Concurrent weather array tasks.
BIOOTON_STEP54_CPUS	8	CPUs per HOSTRADA raster task.
BIOOTON_STEP54_MEMORY	32G	Memory per HOSTRADA raster task.
BIOOTON_STEP54_MAX_CONCURRENT_TASKS	2	Concurrent HOSTRADA raster tasks.
BIOOTON_BIOACOUSTICS_CPUS	16	CPUs per bioacoustic task.
BIOOTON_BIOACOUSTICS_MEMORY	48G	Memory per bioacoustic task.
BIOOTON_BIOACOUSTICS_PARTITION	core partition	Bioacoustic partition.
BIOOTON_BIOACOUSTICS_GRES	empty	Optional GPU GRES such as `gpu:1`.
BIOOTON_MASTER_CPUS	2	Master-table job CPUs.

Tests and machine-readable contracts

Run the maintained regression suite with:

```
bash run_tests.sh
```

The suite covers run planning, locks, manifests, changed-ID handling, incremental upserts, checkpoints, media/weather selection, formation scaling and variants, Sentinel logic, bioacoustics, master-table history, reports, and source integrity. Formal JSON Schema Draft 2020-12 files live in `schemas/`; the master row schema is `schemas/master_table.schema.json`.

Before uploading or deploying a changed pipeline package, at minimum run:

```
bash run_tests.sh
bash run_horeka.sh functionality_test
bash run_horeka.sh add_new_ids --local-test
```

The last command validates planning and submission construction without consuming Slurm resources.

Repository map

```
config.horeka.json              production paths and defaults
pipeline_steps.json             machine-readable step/dependency registry
schemas/                        JSON contracts for rows, manifests, plans, and inventories
scripts/                        data-processing steps
tools/                          planning, orchestration, validation, reporting, and maintenance
scripts_local_run/              Windows/LSDF local runner
Readmes/                        per-step English and German documentation
reference_data/                 maintained taxonomy/reference inputs
tests/                          regression and integrity tests
submit_bio_o_ton_horeka.sh      Slurm DAG constructor
run_horeka.sh                   recommended persistent controller entry point
```

For Git deployment conventions, see `GITHUB_WORKFLOW.md` (currently German). The repository layout is intentionally stable because production configuration and HoreKa deployment paths depend on it.

Sentinel quality score exactly `0` sets `sentinel_has_issues=True` and the issue code `too_cloudy_sentinel_not_available` (too cloudy; Sentinel data unavailable for use). The numeric score and physical file-presence flag are retained. Positive low scores alone do not trigger this code.

02 Master table

README_CI_TEC.md

ci-tec | README | 12.10.2026 | Master schema v7

Use **Bio_O_Ton_Master.csv**, one row per recording. No separate ci-tec table is required. This is the latest recorded inventory, not a guarantee that processing is finished. Original metadata source: dawn-chorus-soundscape.csv.

Column	Meaning
`dawn_chorus_id`	Recording ID.
`lon`, `lat`	WGS84 coordinates in decimal degrees.
`grid_100m_id`, `grid_10m_id`	Assigned grid cells. A grid ID does not imply that LRT formation data are available.
`datetime_local`	German local date and clock time in Europe/Berlin, stored without a timezone suffix. Its origin is recorded separately in timestamp_source.
`datetime_utc`	UTC instant of the selected recording time. When localtimes is absent, this is the datetime instant from the original Dawn Chorus CSV.
`timestamp_source`	Exact values: localtimes for an explicitly supplied local clock; datetime for the UTC fallback. A populated datetime_local does not imply source localtimes.
`time_has_issues`	False only for source localtimes; True for datetime, absent or unknown provenance. Invalid local times are separately rejected by metadata checks.
`duration_seconds`	Checked audio duration in seconds; null if unavailable.
`inside_lrt_polygon`	Recording point lies within an included LRT polygon.
`grid_100m_has_majority_formation`, `grid_10m_has_majority_formation`	Majority formation is available for this cell; different from a direct point/LRT intersection.

Grid embeddings, the LRT GeoPackage and grid reference data are separate products. Recording/bioacoustic flags do not certify their completeness.

Recording time: source and German clock

The original Dawn Chorus column is named **localtimes** (the supplied local time). **Only when this field is absent, empty or whitespace-only** is the original **datetime** value used as a UTC instant and converted to **Europe/Berlin**. German winter time is UTC+1 and summer time is UTC+2; the conversion uses the timezone rules for the recording date, including date changes at midnight. Do not add a fixed offset or convert the resulting German clock a second time.

Examples without localtimes: `2025-01-15T06:00:00Z` becomes `2025-01-15 07:00:00`; `2025-07-15T06:00:00Z` becomes `2025-07-15 08:00:00`. A UTC evening can become the following German calendar day.

When localtimes is supplied, its local clock retains priority and is interpreted as German time. These recordings are unchanged by the fallback rule. A nonempty but invalid or unresolved localtimes value is flagged; it is not silently replaced with datetime. Source conflicts remain visible in the Step-1 time log.

Audit on 28.09.2026: all 3,288 recordings without localtimes already match this UTC-to-Germany conversion in both published master copies, including UTC, German date and clock, and the source label. No timestamp correction was required. Of these, 3,264 use UTC+2 and 24 use UTC+1. The published master has 108,578 recordings with source localtimes and 3,288 with source datetime.

Clean-data filters from the 9 October PDF

Apply the filters needed by your analysis:

```
time_has_issues=False
sound_exists=True
duration_seconds >= 40
sound_has_issues=False
weather_point_exists=True
weather_point_has_issues=False
```

```
sentinel_exists=True
sentinel_has_issues=False
```

The 40-second threshold is an analysis choice and can be replaced. Domain flags do not automatically invalidate the entire recording. For training in cells with a majority formation add `grid_100m_has_majority_formation=True`; for direct point-in-LRT training add `inside_lrt_polygon=True`. Grid IDs alone certify neither.

Recording-time confidence and weather use

Use weather data only for recordings with `time_has_issues=False`, equivalently `timestamp_source == "localtimes"`. UTC fallback timestamps are converted correctly but can differ from the actual supplied recording clock by up to two hours according to the PDF; conversion alone does not establish confidence in the source.

This source restriction supplements weather availability and quality checks. In schema v7, combined readiness flags include it. It does not automatically exclude the recording from audio-only work, and the audio-only bioacoustic readiness flag remains independent of time confidence. Recordings and stored files remain in the shared master.

Availability and issues

_exists: `True` means the corresponding files were recorded as present on LSDF. These flags describe the recorded inventory, not a live file check.

_has_issues: `True` means a problem or missing required data/check results. It can be `True` whether `_exists` is `True` or `False`.

Empty means no check result is available.

_issue_codes gives the reason. Multiple codes are separated by a pipe; details may follow a colon. **_has_issues is not a severity level or a universal exclusion rule.** Rules below concern the affected file/domain, not automatically the entire recording. Unknown codes or a True issue flag without codes require review.

Columns	Data
`sound_exists`, `sound_has_issues`	Audio file and audio checks.
`sentinel_exists`, `sentinel_has_issues`	Sentinel-2 files and technical checks.
`weather_point_exists`, `weather_point_has_issues`	Weather file for the recording and its checks.

Sound examples:

- `True / False`: file found, no issues reported.

- `True / True`: file found, but a technical property or decoding check failed.

- `False / True`: file missing.

Sound codes and suggested use

sound_issue_codes	Meaning / consequence
`duration_not_60s`	Outside 59-61 seconds. **Information**; decide in application-specific preprocessing. Not a general exclusion.
`missing_file`	File absent: unavailable.
`empty_file`, `no_audio_stream_found`, `no_audio_frames_decoded`	No usable audio in the checked file: exclude that file pending repair.
`pyav_decode_failed`, `invalid_or_missing_sample_rate`, `invalid_or_missing_channel_count`, `invalid_duration`, `duration_unavailable`	Technical check failed or a property could not be established: hold for review. Does not prove the original source is permanently corrupt.
`multiple_audio_streams`	Select the intended stream before use.
`filename_does_not_match_<id>_audio_pattern`	File-to-ID association needs clarification.
`source_stat_failed`, `copy_failed`, `WinError`	Access/copy problem; investigate connection and download before concluding source corruption.

PDF audio filter: sound_exists=True, duration_seconds >= 40, sound_has_issues=False. A pure duration_not_60s note does not set the issue flag. Shorter or longer files can be usable; change the duration threshold for your analysis. Technical codes still require review.

Point weather codes and suggested use

Expected window: ten preceding complete German calendar days plus the recording day, hourly. Normally 264 rows; across spring DST 263, across autumn DST 265. The repeated local autumn hour is valid; a timezone-free timestamp alone cannot distinguish its two UTC instants.

weather_point_issue_codes	Meaning / consequence
`missing_file`	No weather file. Cause is not established by this code.
`missing_value`	At least one required value is missing. **Information only:** this code alone does not set weather_point_has_issues. Check variables/hours needed by the application; do not fill values blindly.
`recording_date_changed_recheck_required`	Recording date changed while previous results were retained. Temporal association is unconfirmed: hold until rechecked.
`unexpected_time_interval`	Full timestamp sequence and multiplicities differ from the expected window. May mean missing, extra or shifted hours. Correct DST transitions are accepted by the current inventory. **Check/correct before use.**
`unexpected_time_window`, `unexpected_row_count`	Wrong boundaries or row count: check/correct before use.
`duplicate_datetime`	More repeats than allowed by the expected window, not the legitimate repeated autumn hour. Check/correct before use.
`missing_required_column`, `missing_datetime_column`, `unparseable_datetime`, `invalid_metadata_datetime`	Structure or time basis is unreliable: hold until clarified.
`empty_file`, `read_error`	Empty or unreadable; an exception suffix may follow read_error. File currently unusable.
`implausible_value`	Temperature/humidity range check failed. Investigate values and source before deciding usability.

PDF weather filter: time_has_issues=False, weather_point_exists=True and weather_point_has_issues=False. This permits missing_value as an informational code; verify the variables/hours your analysis needs. A stricter application can additionally require no codes. Additional time/structure errors remain blocking.

Sentinel

Code / field	Meaning / consequence
Sentinel: `missing_file`, `score_without_tif`	Image missing; a score does not replace it.
Sentinel: `too_cloudy_sentinel_not_available`	Quality score is exactly 0: too cloudy, Sentinel data unavailable for use. The numeric score stays 0; sentinel_has_issues=True. sentinel_exists still reports physical file presence.
Sentinel: `quality_score_missing`	Image may exist, but its quality assessment is missing.
Sentinel: `empty_file`, `raster_read_failed`	Image empty or unreadable: exclude pending repair.
`inventory_not_run`, `qc_not_run`	Check results unavailable; not equivalent to passing QC.

Weather and Sentinel do not automatically exclude a recording from audio/LRT-only work. Photos are not a filter criterion for the stated ci-tec priorities.

Processing completion is separate from quality

is_final does not currently exist. ready_for_general_analysis is not a ci-tec release flag: it requires trusted time provenance plus unflagged sound, weather and Sentinel together. It does not enforce an analysis-specific audio duration threshold; apply duration_seconds >= 40 separately for the PDF clean filter.

Completion requires finished processing and checks for the required products, a specified input/code version and no outstanding repairs. Missing data and suitability remain separate. A permanent release applies only to a frozen version; later versions can reveal new errors. The shared master retains all rows.

03 Bio-O-Ton Master Table Reference

MASTER_TABLE_README.md

This reference defines the final ID-level table written by `scripts/Step_7_0_update_master_table.py`. The implementation's `MASTER_COLUMNS` list is the authoritative output order; the current version is `2026-10-09-mastertable-v7` and contains 106 columns.

Products and row model

```
Bio_O_Ton_Master.csv
Bio_O_Ton_Master.parquet
Bio_O_Ton_Master_summary.json
outputs/step_0_control/status_events.csv
```

The master table has exactly one row per unique `dawn_chorus_id`. The CSV and Parquet contain the same ordered columns. The summary JSON records creation/update metadata and aggregate readiness/status counts. The append-only event CSV records additions, deletions, and changes to tracked status/readiness fields.

`schemas/master_table.schema.json` is the machine-readable validation schema for required/core fields and permits the writer's additional derived fields. For the complete output surface and column order, use the writer's `MASTER_COLUMNS` list and this reference.

The LRT sensitivity analysis additionally writes:

```
Bio_O_Ton_Formation_Variants.csv
Bio_O_Ton_Formation_Variants.parquet
Bio_O_Ton_Formation_Variants_summary.json
Bio_O_Ton_Variant_Summary.csv
Bio_O_Ton_Variant_Temporal_Summary.csv
```

That normalized table contains one row per `dawn_chorus_id` and `lrt_variant`. The main master table remains one row per ID and uses the configured primary variant, currently `no_K_post2017` from `All_Bundeslander_no_K_post2017.gpkg`, for its detailed 100 m/10 m fields.

Source products

Domain	Main source consumed by Step 7.0
Metadata/time/GPS	`outputs/step_1_metadata/dawnchorus_metadata_clean.csv`, `dawnchorus_metadata_log.csv`, and `metadata_source_fingerprints.csv`
Primary 100 m formation and direct LRT hits	`outputs/step_2_variants/<primary_suffix>/step_2_2/DawnChorus_LRT_Grid_Assignment_<primary_suffix>.csv`
Primary 10 m formation	`outputs/step_2_variants/<primary_suffix>/step_2_4_susi_10m/Formation_Status_10m_Grid_withLRTCode_<primary_suffix>.parquet`
Formation-variant coverage	`outputs/Bio_O_Ton_Formation_Variants.parquet` and `variant_index.json`
Audio	Step-3 audio compact/detailed inventories and audio retry log
Photos	Step-3 photo compact/detailed inventories and photo retry log
Sentinel-2	Step-4 compact/detailed inventory plus configured score CSV
HOSTRADA point weather	`outputs/step_5_1_weather_inventory/weather_inventory_compact.csv`
HOSTRADA 100 m rasters	Step-5.4 raster tree and Step-5.5 quality outputs
Bioacoustics	Step-6.5 recording summary and Step-6.6 compact QC

The master table deliberately condenses these sources. File-level, segment-level, model-level, raster-level, and detailed failure information remains in the corresponding step outputs.

Column definitions

Identity and provenance

Column	Definition
`mastertable_schema_version`	Master-table row version. Current constant: `2026-10-09-mastertable-v7`.
`workflow_run_id`	Shared workflow ID that most recently updated the row. Outside an orchestrated run, the writer's fallback run ID is used.
`dawn_chorus_id`	Unique numeric Dawn Chorus recording ID. It is normalized to a digit string and is the table's primary key.
`source_fingerprint`	SHA-256 fingerprint from Step 1 over the source fields relevant to the ID. It supports changed-record detection.
`record_added_to_mastertable_utc`	UTC time when the ID was first inserted. Preserved during later automatic updates.
`record_updated_in_mastertable_utc`	UTC time of the latest automatic update of this row.

Time and coordinates

Column	Definition
`datetime_local`	German local clock from Step 1, stored without a timezone suffix as `YYYY-MM-DD HH:MM:SS`.
`datetime_utc`	UTC instant from the offset-aware Step-1 product, formatted as `YYYY-MM-DDTHH:MM:SSZ`; calculated before removing the offset for `datetime_local`.
`date_local`	Local calendar date derived from `datetime_local`.
`time_local`	Local wall-clock time derived from `datetime_local`; the local offset is not applied twice.
`timestamp_source`	Source selected by Step 1, such as `localtimes` or `datetime`.
`time_has_issues`	`False` exactly when `timestamp_source = localtimes`; `True` for UTC fallback, missing or unknown provenance. This is a source-confidence flag; invalid local timestamps are separately rejected by `metadata_status` and readiness.
`timestamp_changed`	`True` when Step 1 normalized, converted, or reinterpreted the source timestamp.
`timestamp_change_reason`	Step-1 explanation of the timestamp conversion/normalization.
`lat`	Cleaned WGS84 latitude; valid range is -90 to 90.
`lon`	Cleaned WGS84 longitude; valid range is -180 to 180.

Step 1 always preserves the clock in a nonempty source `localtimes` field and assigns `Europe/Berlin`. Only an absent local value enables conversion of the source `datetime` instant to Germany (including historical DST rules). Invalid, nonexistent or unresolved ambiguous local clocks do not silently fall back. The Step-1 log records `timestamp_status`, `timestamp_issue_codes`, the UTC reference and the difference in seconds. These source-consistency warnings are separate from the master's syntactic `metadata_status` check. Source offsets inconsistent with Germany never override an explicitly supplied local clock. ci-tec uses `datetime_local` in the shared full master as its recording-time column. A separate compact export is no longer generated automatically.

The 9 October 2026 PDF clean-data rule requires `time_has_issues=False`. Combined readiness now enforces that source rule. UTC fallback conversion remains unchanged. The source flag does not invalidate audio-only analyses or remove rows. See `README_CI_TEC.md` for the consumer filters.

Status fields

Canonical domain and row statuses use values from the project status model, including `not_started`, `validated`, `missing`, `has_issues`, `partial`, `failed`, `not_applicable`, `manual_review_required`, and `approved` where applicable. The master writer derives the following status columns rather than copying arbitrary free text:

Column	Definition
metadata_status	validated when both local/UTC timestamps and coordinates are valid; otherwise has_issues.
sound_status	validated when audio exists without issues, missing when absent, otherwise has_issues.
photo_status	validated when a photo exists without issues, missing when absent, otherwise has_issues.
sentinel_status	validated when Sentinel-2 data exists without technical issues, missing when absent, otherwise has_issues.
weather_point_status	validated when point weather exists without issues, missing when absent, otherwise has_issues.
formation_100m_status	missing without a grid assignment, has_issues when assigned but lacking a majority formation, otherwise validated.
formation_10m_status	missing without a derived 10 m assignment, has_issues when assigned but lacking a majority formation, otherwise validated.
bioacoustic_status	Step-6 compact QC result, normally validated, partial, failed, or missing.

Audio and photos

Column	Definition
sound_exists	At least one audio file for the ID was found by the audio inventory.
duration_seconds	Checked audio duration from the detailed inventory; null when unavailable. With multiple files, the minimum known duration is retained and individual QC issues remain visible. Focused metadata refreshes preserve this value.
sound_has_issues	The audio inventory or download retry log reports a technical problem for the ID. A pure duration_not_60s note does not set this flag; apply duration_seconds >= 40 separately for the PDF's suggested clean filter.
sound_issue_codes	Pipe-separated compact audio causes such as missing_file, duration_unavailable, or sound_missing_audio_url.
sound_status	Canonical audio status; derivation is listed under Status fields.
photo_exists	At least one photo file for the ID was found by the photo inventory.
photo_has_issues	The photo inventory or download retry log reports a problem for the ID.
photo_issue_codes	Pipe-separated compact photo causes such as missing_file, image_verify_failed, or photo_missing_photo_url.
photo_status	Canonical photo status; derivation is listed under Status fields.

Sentinel-2

Column	Definition
sentinel_exists	At least one Sentinel-2 GeoTIFF for the ID was inventoried.
sentinel_has_issues	The inventory or score join reports a technical problem. A score of exactly 0 sets too_cloudy_sentinel_not_available; positive low scores alone are not technical issues.
sentinel_quality_score	Optional quality score joined from the configured S2_Scores.csv. The value is retained, including 0; exactly 0 marks Sentinel data as unavailable for use due to cloudiness.
sentinel_issue_codes	Pipe-separated causes such as missing_file, quality_score_missing, too_cloudy_sentinel_not_available, or all_pixels_nodata.
sentinel_status	Canonical Sentinel-2 status; derivation is listed under Status fields.

Weather

Column	Definition
`weather_point_exists`	The Step-5.1 compact inventory reports a present `weather_<id>.csv`; an empty file can exist and have issues.
`weather_point_has_issues`	Point weather has missing required columns, implausible values, wrong row count/interval, a read error or missing QC. `missing_value` alone is informational and does not set this flag.
`weather_point_issue_codes`	Pipe-separated HOSTRADA point-weather causes such as `missing_file`, `missing_value`, or `unexpected_row_count`.
`weather_point_status`	Canonical point-weather status; the master writer reuses Step-5.1 QC rather than rereading all weather files.
`weather_raster_hostrada_100m_exists`	Required products are present in the configured 100 m raster output tree. This is a global state copied to each row.
`weather_raster_hostrada_100m_has_issues`	Global raster state reports missing products, NoData problems, QC gaps, or structural warnings.
`weather_raster_hostrada_100m_issue_codes`	Pipe-separated global causes such as `missing_raster`, `qc_not_run`, or `all_nodata`.

There is intentionally no 10 m weather-raster column. Optional 100 m weather rasters do not block `ready_for_general_analysis` or ordinary formation readiness.

100 m formation

Column	Definition
`grid_100m_id`	Authoritative INSPIRE 100 m grid cell assigned to the recording point by Step 2.2, whether or not the cell has a majority formation.
`grid_100m_assignment_exists`	`True` when a 100 m grid ID is available.
`grid_100m_has_majority_formation`	`True` when the assigned cell has a majority formation.
`majority_formation_100m`	Formation with the largest area share in the assigned 100 m cell.
`majority_formation_status_100m`	Most frequent A/B/C conservation status inside the majority formation. K contributes to formation totals but is not selected as the majority formation status.
`majority_value_100m`	Majority-formation share in integer centi-percent; `10000` means 100.00%.
`second_value_100m`	Second-largest formation share in integer centi-percent.
`majority_delta_100m`	Difference between majority and second place in integer centi-percent.
`majority_disputed_100m`	`True` when `majority_delta_100m <= 200`, i.e. at most two percentage points. Null when a comparison cannot be computed.
`formation_100m_status`	Canonical 100 m status; derivation is listed under Status fields.

Direct LRT intersections

These fields describe direct point-in-polygon hits against the cleaned primary LRT source. They are stricter than assignment to a grid cell whose area overlaps an LRT.

Column	Definition
`inside_lrt_polygon`	`True` when the recording point lies in at least one cleaned LRT polygon.
`lrt_polygon_count`	Number of directly intersected LRT polygons.
`lrt_code_count`	Number of distinct LRT codes among direct hits.
`lrt_formation_count`	Number of distinct formation values among direct hits.
`lrt_status_count`	Number of distinct conservation-status values among direct hits.
`lrt_mapping_year_count`	Number of distinct mapping years among direct hits.
`lrt_codes`	Pipe-separated distinct LRT codes from direct hits.

Column	Definition
`lrt_formations`	Pipe-separated distinct formations from direct hits.
`lrt_conservation_statuses`	Pipe-separated distinct conservation statuses from direct hits.
`lrt_mapping_years`	Pipe-separated distinct mapping years from direct hits.

10 m formation

Column	Definition
`grid_10m_id`	Susi-compatible 10 m identifier derived from the assigned INSPIRE 100 m ID and the point's EPSG:3035 position, whether or not the cell has a majority formation.
`grid_10m_assignment_exists`	`True` when a 10 m grid ID could be derived.
`grid_10m_has_majority_formation`	`True` when the 10 m product contains a majority formation for the derived cell.
`majority_formation_10m`	Formation with the largest share in the 10 m cell.
`majority_formation_status_10m`	Most frequent A/B/C conservation status inside the 10 m majority formation.
`majority_value_10m`	Majority share in integer centi-percent.
`second_value_10m`	Second-largest share in integer centi-percent.
`majority_delta_10m`	Difference between first and second place in integer centi-percent.
`majority_disputed_10m`	`True` when `majority_delta_10m <= 200`; null when unavailable.
`formation_10m_status`	Canonical 10 m status; derivation is listed under Status fields.

Cross-resolution and variant coverage

Column	Definition
`formation_100m_10m_agree`	`True` only when both majority formations are present and equal.
`formation_status_100m_10m_agree`	`True` only when both majority-formation statuses are present and equal.
`formation_primary_variant`	Configured LRT suffix whose detailed values populate the main master row.
`formation_variant_count_expected`	Variant count from `variant_index.json`; zero when the index is absent/unreadable.
`formation_variants_with_100m_majority`	Number of variant rows for the ID with a 100 m majority formation.
`formation_variants_with_10m_majority`	Number of variant rows for the ID with a 10 m majority formation.
`formation_variant_products_complete`	Global flag copied to every row: all expected variants have both 100 m and 10 m products, and the actual variant count equals the expected non-zero count.

Bioacoustics

Column	Definition
`bioacoustic_status`	Canonical outcome from Step-6 QC.
`bioacoustic_has_issues`	`True` when required model coverage is incomplete or inference/QC reports failure.
`bioacoustic_issue_codes`	Pipe-separated causes such as `required_models_incomplete` or `model_inference_failed`.
`bioacoustic_models_expected`	Pipe-separated models planned for the ID.
`bioacoustic_models_complete`	Pipe-separated models successfully completed for the ID.
`bioacoustic_required_models_complete`	`True` when every model configured with `required: true` completed.
`bioacoustic_inference_version`	Version of the Step-6 output/inference transformation.
`bioacoustic_species_count`	Number of distinct, non-implausible aggregated taxa across models. This is not a confirmed species count.
`bird_species_count`	Number of aggregated taxa grouped as birds.
`nonbird_species_count`	Number of aggregated taxa not grouped as birds.
`bioacoustic_max_confidence`	Highest normalized score in the recording aggregation. Scores from different models are not necessarily calibrated against one another.
`top_species_scientific`	Scientific name of the highest-ranked aggregated taxon.
`top_species_model_support`	Number of models that reported the top taxon above the configured threshold.

Readiness flags

Column	Exact intent
`ready_for_general_analysis`	Valid local/UTC time and coordinates, `time_has_issues=False`; audio exists without issues; point weather exists without issues; Sentinel-2 exists with a quality score and without technical issues. Photos, formation, raster weather, and bioacoustics do not block it.
`ready_for_formation_analysis_100m`	General readiness plus a 100 m assignment and majority formation.
`ready_for_direct_lrt_analysis`	General readiness plus `inside_lrt_polygon = True`.
`ready_for_formation_weather_raster_analysis_100m`	100 m formation readiness plus present, issue-free HOSTRADA 100 m rasters.
`ready_for_formation_analysis_10m`	General readiness plus a 10 m assignment and majority formation.
`ready_for_multimodal_analysis`	General readiness plus a present, issue-free photo.
`ready_for_bioacoustic_analysis`	Audio exists without issues, `bioacoustic_status = validated`, and all required bioacoustic models are complete. This flag is intentionally independent of weather, Sentinel-2, photos, and formation.

Issue-code and status conventions

Column	Definition
`record_blocking_issue_codes`	Pipe-separated summary of missing/invalid time, coordinates, sound, point weather, Sentinel-2, photo, and missing 100 m/10 m majority formations. The field is a navigation aid; domain logs contain the full evidence.

Issue-code fields use stable, lowercase, pipe-separated tokens:

```
missing_file|missing_value|unexpected_row_count
```

When errors originate in free-text detail/retry logs, Step 7.0 normalizes them to compact tokens. File names, HTTP codes, raster statistics, decode traces, and other evidence remain in the detailed domain products.

Record and release status

Column	Definition
`record_status`	Starts as `partial`; becomes `validated` when general, 100 m formation, and 10 m formation readiness are all true; becomes `has_issues` when metadata is problematic and sound, point weather, and Sentinel-2 are all missing/problematic.
`release_status`	Automatically `not_started`, or `manual_review_required` for technically validated rows. A previously manual `approved` value is preserved. Automatic processing never creates a new approval.
`manual_review_comment`	Free-text manual review note preserved across automatic updates.
`manual_reviewed_by`	Reviewer name/identifier preserved across automatic updates.
`manual_reviewed_utc`	Manual review time preserved across automatic updates.

Incremental update behavior

Both Horeka and local runs call the same Step 7.0 writer. Core full, incremental and focused metadata updates derive the time flag from provenance. Incremental updates also migrate retained rows, so the new flag is never left empty. Changes are recorded in status events. A missing source is conservatively flagged.

The bioacoustics phase refreshes Step-6 fields and also applies the v7 source flag and combined-readiness migration to the existing master. It updates the schema version and adds a nullable duration column if an older master lacks it; a core run fills it from audio QC. Metadata timestamps, grid/LRT assignments, other domains and manual fields are retained. Existing manual approvals remain historical approvals and do not override the current clean-data filters.

Proximity review fields

These five v6 fields are advisory. They never delete recordings or change readiness.

Column	Meaning
`proximity_check_status`	`checked` when coordinates and an aware UTC timestamp are usable; otherwise `missing_or_invalid_space_time`.
`duplicate_candidate`	At least one other recording is within 10 m and 300 seconds by default. Unknown if space/time is invalid.
`duplicate_neighbor_count`	Number of such direct neighbours, excluding the recording itself.
`spatiotemporal_cluster_id`	Connected-component label for links within 50 m and 1800 seconds by default; blank for an isolated recording. Labels can change when members change.
`spatiotemporal_cluster_size`	Number of members including the recording itself; one for an isolated valid recording, unknown for invalid space/time.

Distances are spherical and time differences use UTC, including across DST changes. Thresholds come from `proximity_review`. A connected component does not imply that every pair of members satisfies the distance/time limits.

Core update details

The orchestrator serializes master updates. With `--ids-file`, Step 7.0 rebuilds only listed IDs, replaces those rows, and preserves every unaffected row. A run without `--ids-file` rebuilds the complete current ID set. Global grid, variant, or raster changes intentionally use a full update.

Writes are protected by a file lock on Linux and use temporary-file replacement for the CSV and JSON. The Parquet file is compressed with Zstandard and also replaced atomically when the writer succeeds. If optional Parquet writing fails, the CSV remains the canonical successful table output and the summary records no Parquet path.

The first-insertion time and manual review fields are carried forward from the previous CSV. `release_status = approved` is preserved; other technical/release fields are recalculated.

Status event history

`outputs/step_0_control/status_events.csv` contains:

Field	Meaning
`event_utc`	Event creation time.
`workflow_run_id`	Workflow that observed the change.
`dawn_chorus_id`	Affected ID.
`field`	`record_lifecycle` or a tracked status/readiness field.
`previous_value`	Value before the update.
`current_value`	Value after the update.

Full updates reconcile the master with the complete clean-metadata ID set and record additions and deletions. Incremental updates replace or delete only explicitly selected IDs and record those lifecycle events; rows outside their selected ID set are retained. A valid header-only clean input permits an empty master, while missing input aborts the update. Tracked changes include canonical domain statuses, `record_status`, `release_status`, the main readiness flags, and `record_blocking_issue_codes`.

Execution

The normal pipeline schedules Step 7.0 automatically after relevant domains and always writes a final snapshot. To rebuild it manually on HoreKa:

```
bash run_master_table_update.sh
```

To update only selected IDs directly:

```
.venv/bin/python scripts/Step_7_0_update_master_table.py \
  --config config.horeka.json \
  --ids-file /path/to/ids.csv
```

The ID file is read through the common ID-list helper and must exist. For end-to-end execution, locking, manifests, validation, and release behavior, return to the root README.

04 Bio-O-Ton Documentation Index

Readmes/README_INDEX.md

Start with the root README for the master-table overview, recommended commands, complete workflow, operating model, and recovery guidance. The master-table reference defines all 104 output columns and readiness rules.

English step documentation

Area	Documentation
Pipeline control	Run planning, locking, manifests, and rebuild state
Step 1	Metadata extraction
Step 2.0	LRT cleaning
Step 2.1	100 m formation status
Step 2.2	Point assignment and direct LRT intersections
Step 2.3	Grid aggregation
Step 2.4	10 m formation status
Step-2 variants	LRT sensitivity analysis
Step 3	Media inventory and download
Step 4	Sentinel-2 mirror and inventory
Step 5.2	HOSTRADA weather per recording
Step 5.3	HOSTRADA monthly source data
Step 5.4	HOSTRADA 100 m rasters
Step 5.5	HOSTRADA raster quality control
Step 6	Bioacoustic embeddings and species inference
Step 7.0	Final master table
Validation	Final validation and formation-status comparison
Step 9	Visual pipeline reports

Matching `README_DE.md` files are retained next to the English documents for German-language operational use.

Current operation and diagnostics

- Two separately initiated phases and Horeka-2 preparation

- Bounded Slurm workers

- Every step README now contains source-linked diagnostic codes, meanings,

explicit exceptions and process-return conditions. Data issues, process exits and scheduler states are separate categories.

- Refresh references with `python tools/generate_step_readmes.py`; validate them

with `python tools/documentation_reference.py --check` from the repository root.

- Documentation review and scope

- The ci-tec quick reference remains deliberately short: README.

05 Zwei getrennte Pipeline-Läufe

Readmes/pipeline_phases.md

1. Daten aktualisieren

```
bash slurm_add_new_ids.sh
```

Metadaten, LRT-/Grid-Produkte, Audio, Fotos, Sentinel und Wetter laufen wie bisher. Unabhängige Zweige bleiben parallel. Danach folgen Master, Abschlussprüfung und Freigabe des Pipeline-Locks. Es werden **keine Schritte 6.x** eingeplant und keine Bacpipe-Umgebung installiert. `from_scratch` baut ebenfalls nur diesen Kern neu auf.

2. Bioakustik bei Bedarf starten

Nach Abschluss des ersten Laufs separat ausführen:

```
bash slurm_bioacoustics.sh
```

Dieser Lauf umfasst Modellprüfung, Worklist, Inferenz, Vorhersagen, Deutschland-Taxonomie, Aggregation, QC, Bioakustik-Master-Update und Abschlussprüfung. Er verwendet die vorbereiteten Metadaten und das Audioinventar. Abweichende IDs zwischen Metadaten und Master verhindern den Start; dann zuerst den Kernlauf abschließen.

Die Worklist wird für alle aktuellen Metadaten-IDs neu abgeglichen. Dadurch gehen neue oder geänderte Audiodateien nicht verloren, wenn Step 1 seine Änderungsmarker bereits aktualisiert hat. Gelöschte IDs werden aus der Worklist entfernt. Unveränderte Audio-/Modell-Work-Keys verwenden weiterhin ihre Inferenz-Checkpoints. Modelle, Parameter, logische Shards und Dateipfade bleiben unverändert. Standardmäßig verteilen vier Slurm-Worker die bisherigen logischen Inferenzaufgaben. Ein erneuter Start setzt die Arbeit anhand der Checkpoints fort.

Das Bioakustik-Master-Update ersetzt Bioakustikfelder und `ready_for_bioacoustic_analysis`. Seit Schema v7 migriert es zusaetzlich `time_has_issues`, zeitabhaengige kombinierte Readiness und die Schemaversion; bei alten Tabellen kommt eine nullable `duration_seconds`-Spalte hinzu. Zeiten, Koordinaten, LRT-/Grid-Zuordnungen, Medien-/Wetterstatus und zusätzliche vorhandene Spalten werden übernommen. Die Kern-Abschlussprüfung fordert keine Bioakustikprodukte; der Bioakustiklauf prüft seine eigenen Produkte und Voraussetzungen. Die gemeinsame Sperre verhindert gleichzeitige Schreibzugriffe beider Phasen. Nicht parallel auf demselben Output starten.

Wer den bisherigen tmux-Controller verwendet, nutzt entsprechend zuerst:

```
bash run_horeka.sh add_new_ids
```

Und später:

```
bash run_horeka.sh bioacoustics
```

Beide Startwege verwenden denselben Planer. Unter Windows steht zusätzlich `scripts_local_run/run_pipeline_local.ps1 -Mode bioacoustics` zur Verfügung. Die Rechenarbeit der Modelle wird nicht schneller; der normale Datenlauf wartet jetzt nicht mehr darauf.

Vorbereitung für Horeka 2

Ohne Profil gelten die bisherigen Horeka-Einstellungen. Ein vertrauenswürdiges lokales Shell-Profil kann über `BIOOTON_CLUSTER_PROFILE=/absoluter/pfad/profil.sh` gewählt werden. Launcher und Environment-Bootstrap lesen es vor ihren Standardwerten; der tmux-Start übernimmt die exportierten Einstellungen ebenfalls.

Das Profil bündelt `CONFIG`, Core-/Bacpipe-Python bzw. Environment-Präfixe, Partitionen, Account und Ressourcen. `BIOOTON_CONSTRAINT` ersetzt das bisher fest eingebaute `LSDF`; ein ausdrücklich leerer Wert lässt `--constraint` weg. Zeit-, CPU-, Speicher-, GPU- und Worker-Overrides bleiben verfügbar. Die Vorlage `cluster_profiles/horeka2.example.sh` bricht absichtlich ab, bis sie kopiert und mit den auf Horeka 2 bestätigten Angaben konfiguriert wurde. Partitionsnamen, Hardware und Mountpoints werden nicht vorweggenommen.

Bei der Migration die Konfiguration mit den dort gültigen absoluten Datenpfaden bereitstellen, beide Python-Umgebungen neu aufbauen und Modell-/Runtime-Versionen beibehalten. Keine bestehende virtuelle Umgebung zwischen Clustern kopieren. Erst Funktionstest und Submission-Vorschau, danach ein kleiner Bioakustik-Testlauf gegen bekannte Ergebnisse. LSDF-Produkte und Checkpoints nur übernehmen, wenn Pfade und Zugriffsrechte bestätigt sind. Die Migration selbst ist

noch nicht erfolgt.

06 Weniger Slurm-Jobs

Readmes/slurm_worker_limits.md

Die Startskripte fuer `add_new_ids` und `from_scratch` buendeln grosse Arrays:

Bereich	Slurm-Teiljobs	Arbeit pro Job
Bioakustik	maximal 4	Mehrere Modell-/Shard-Aufgaben nacheinander
HOSTRADA-Raster	maximal 2	Mehrere Variablen-/Jahres-Aufgaben nacheinander
Wetter-Download	unveraendert maximal 8 laut aktueller Konfiguration	Aufgeteilte Aufnahme-IDs

Die Bioakustik behaelt ihre 64 logischen Shards je Modell. Bei sechs Modellen bearbeiten vier Worker weiterhin alle 384 Aufgaben. Die Workerzahl aendert weder Shard-IDs noch vorhandene Checkpoints. Die anschliessenden Verifier pruefen weiterhin alle erwarteten Produkte. Fehler einzelner Aufgaben werden gesammelt; der Worker bearbeitet die uebrigen Aufgaben und endet mit Fehlerstatus.

Optional vor dem Start auf Horeka:

```
export BIOOTON_BIOACOUSTICS_WORKERS=4
export BIOOTON_STEP54_WORKERS=2
bash slurm_add_new_ids.sh
```

Die Einstellungen gelten auch fuer `slurm_from_scratch.sh`. Workerzahl und Parallelitaet sind getrennt: `max_concurrent_tasks` begrenzt nur gleichzeitig laufende Jobs, nicht die Zahl wartender Jobs.

Das Slurm-Zeitlimit gilt jetzt fuer einen ganzen Worker, nicht fuer jede seiner Teilaufgaben. Bei langen Laeufen kann deshalb ein weiterer inkrementeller Start noetig sein. Die Bioakustik erhaelt das Vorwarnsignal weiterhin, damit sie ihren Fortschritt sichern kann. Nach einem Signal startet der Worker keine weitere Aufgabe. Es gibt keine automatische Neueinreichung nach einem Zeitlimit.

Die Pipeline meldet weiterhin ihre einzelnen Vorbereitungs-, Pruef- und Master-Schritte an. Auch mit kleineren Arrays kann ein gemeinsam ausgelastetes Account-Limit die Einreichung verhindern.

Bereits eingereihte Jobs bleiben unveraendert. Nach einer zuvor abgebrochenen Einreichung zuerst deren Jobs und Pipeline-Lock pruefen; nicht parallel einen zweiten Lauf starten oder den Lock waehrend aktiver Jobs entfernen.

Lokale Pruefung: `python tests/test_array_worker.py` prueft Aufgabenabdeckung, Fehlerweitergabe, Signalweitergabe sowie echte und simulierte sbatch-Argumente.

07 Gemeinsame Master und sichere Wiederaufnahme

Readmes/master_reconciliation.md

Die gemeinsame Datei heißt **Bio_O_Ton_Master.csv**. CSV, Parquet und Zusammenfassung werden zusammen aktualisiert. Lokale und LSDF-Kopien eines veröffentlichten Stands müssen dieselbe SHA-256 besitzen; Uploadkopien sind keine eigene fachliche Variante.

Auswahl der Datenstände

Die Zusammenführung vom 24.09.2026 verwendet die vollständigen, aus Step-2-Produkten geprüften räumlichen Zuordnungen und die zuletzt geprüften LSDF-Inventare für Wetter, Audio, Fotos und Sentinel. IDs, Koordinaten und Aufnahmezeiten müssen zuvor exakt übereinstimmen. Größere Trefferzahlen allein sind kein Auswahlkriterium. Die zusätzlichen LRT-Varianten werden anhand ihrer Punktprodukte vervollständigt. Die konkreten Quellen und Prüfsummen stehen im jeweiligen Veröffentlichungsbericht.

`add_new_ids`

- Der Planer gleicht IDs **und Koordinaten** jeder konfigurierten LRT-Punktzuordnung

unabhängig von den Step-1-Fingerprints ab. Unterbrochene Verarbeitung wird erneut eingeplant. Die Koordinaten älterer Produkte werden aus dem zugehörigen `point_processing_log` gelesen.

- Step 2.2 prüft dieselbe Abdeckung auch bei direktem Aufruf und entfernt nicht mehr

im Metadatenbestand enthaltene IDs. Ein ausdrücklich berechnetes negatives räumliches Ergebnis bleibt gültig.

- Räumliche Stufen bearbeiten die konfigurierten Varianten innerhalb des jeweiligen

bestehenden Jobs nacheinander; es entsteht kein Slurm-Array pro Variante.

- Zwischenstände dürfen Master-Updates bis zum Abschluss von Step 2.2 aufschieben.

Der abschließende Master-Schritt verlangt vollständige Punktzuordnungen, aktualisiert die Variantentabelle und berechnet danach die Master-Zusammenfassung neu.

- `formation_variant_products_complete` wird pro Aufnahme geprüft. Die Existenz

der globalen Variantendateien allein reicht nicht aus.

- Die separate Bioakustikphase bleibt separat und verändert keine räumlichen Felder.

Neue Laufdiagnosen

Meldung	Bedeutung / Handlung
`POINT_ASSIGNMENT_INCOMPLETE`	IDs fehlen, sind doppelt oder haben andere Koordinaten im Punktprodukt. Step 2.2 nachholen; Master nicht mit scheinbar negativen Treffern ersetzen.
`MASTER_UPDATE_DEFERRED`	Ein Zwischenupdate wartet auf vollständige räumliche Verarbeitung. Bestehende Master bleibt erhalten; der finale Schritt prüft streng.
`FORMATION_10M_MISSING`	Konfiguriertes 10-m-Produkt fehlt. Step 2.4 vervollständigen.
`FORMATION_10M_UNREADABLE`	Vorhandenes 10-m-Parquet konnte nicht gelesen werden. Datei prüfen/wiederherstellen; nicht als fehlende Formation interpretieren.
`SOURCE_POPULATION_REDUCED`	Mehr als 20 % und mehr als zehn bisherige IDs würden durch die Quelle verschwinden. Vollständige Originalquelle prüfen. Nur bei beabsichtigter großer Reduktion `metadata_extraction.allow_large_source_reduction=true` setzen.

Diese Diagnosen sind Laufmeldungen, keine neu eingeführten Aufnahme-Issue-Codes. Normale einzelne Löschungen bleiben möglich. Der Reduktionsschutz gilt auch beim direkten Aufruf von Step 1 bzw. Step 7. Eine bereinigte Metadatendatei ersetzt niemals die ursprüngliche Dawn-Chorus-Quelldatei.

08 Dokumentationspruefung

Readmes/DOCUMENTATION_REVIEW.md

Stand: 2026-09-24, Arbeitsstand einschliesslich Master-Zusammenführung, unabhängiger Punktabdeckung und der Trennung von Kern und Bioakustik. Diese Beschreibung bestaetigt keinen bereits erfolgten Git-Push oder produktiven Horeka-/Horeka-2-Lauf.

Abgeglichen

- Haupt-README, Master-Referenz, lokale Anleitung, Deployment/Recovery und

deutsche/englische Schritt-READMEs gegen den lokalen Code und `config.horeka.json`.

- Separater Bioakustikstart, phasenspezifische Validierung und begrenztes

Bioakustik-Master-Update; unveraenderte deutsche Aufnahmezeitregel.

- 104 Master-Spalten in Schema v6, einschliesslich der fuenf unverbindlichen

Raum-/Zeit-Naeheflags. Eine reine Bioakustik-Aktualisierung migriert ein altes Master-Schema nicht automatisch.

- 64 logische Bioakustik-Shards pro Modell, vier Slurm-Worker; HOSTRADA-Raster

verteilen ihre logischen Variable/Jahr-Aufgaben auf zwei Worker.

- Relative Secrets-/Checkpoint-Pfade, bestehender Checkout und Clusterprofile.

- Fehler-/QC-Codes mit Bedeutung und Quellstelle in allen 18 Schrittbereichen,

jeweils auf Deutsch und Englisch. Explizite Exceptions und Rueckgabecodes stehen getrennt daneben. Dynamische Fremdbibliotheksmeldungen werden nicht als erfundene statische Fehlercodes ausgegeben.

Die Referenz erfasst ausdrueckliche Diagnosewerte aus dem zugeordneten Quellcode. Sie ist kein Versprechen, jede denkbare Bibliotheks-/Netzwerkexception aufzuzaehlen. Statuswerte ohne Fehlerbedeutung sind als solche getrennt dargestellt.

Wartung

```
python tools/generate_step_readmes.py
```

Erhaelt redaktionelle Texte und aktualisiert ausschliesslich die markierten Diagnoseabschnitte. Neue Fehlercodes ohne gepflegte Bedeutung fuehren zu einem Fehler, statt unerklaert in der Dokumentation zu landen.

```
python tools/documentation_reference.py --check
```

Prueft, ob diese Abschnitte noch dem Code entsprechen. Die maschinenlesbare Referenz steht in `diagnostic_catalog.json`. Redaktionelle Aussagen muessen weiterhin bei Verhaltensaenderungen geprueft werden.

Historische Berichte

Datierte Audits und Zeitpruefungen sind als historisch gekennzeichnet. Ihre damaligen Befunde und Zahlen wurden nicht nachtraeglich als aktuelle Messungen umgeschrieben. Aktuelle Betriebsschritte stehen in den operativen READMEs.

PDF

Das gemeinsame PDF-Handbuch enthaelt die READMEs und zentralen Betriebsreferenzen mit Inhaltsverzeichnis und Lesezeichen. Die kurze ci-tec-README ist darin ein eigenes Kapitel. Die Markdown-Dateien bleiben die wartbare Quelle.

09 Master-Schema: Abgleich mit ci-tec-PDF vom 09.10.2026

Readmes/master_schema_pdf_audit_20261009.md

Gepruefte Referenz: `C:/Users/Frede/Downloads/Bio_O_Ton_Mastertable_read_me_ci_tec (1).pdf`, beide Seiten ausgelesen und visuell geprueft. Das Dokument beschreibt den Datenvertrag; die Anweisungen zur spaeteren Analyse sind keine Aufforderung, Masterzeilen herauszufiltern. Massgeblich sind ausserdem die beiden ausdruecklichen Nutzerentscheidungen: reine Wetter-NaNs und reine Audio-Dauerabweichungen sind Hinweise; die Mastertabelle bleibt ungefiltert.

Ergebnis und Datenvertrag

Schema `2026-10-09-mastertable-v7`, 106 geordnete Spalten in CSV und Parquet. Horeka und lokale Pipeline rufen denselben `scripts/Step_7_0_update_master_table.py` auf, einschliesslich Zwischenupdates, finaler Aktualisierung, fokussiertem Metadaten-Refresh und separater Bioakustikphase.

PDF-Vorgabe	Umsetzung / Pruefergebnis
Eine Zeile pro Aufnahme-ID	Kein neuer Qualitaetsfilter und keine Dauergrenze in der Zeilenauswahl. Problematische Aufnahmen bleiben enthalten. Bestehende Regeln zur Metadatenpopulation, eindeutigen IDs und geloeschten Quell-IDs bleiben bestehen.
Deutsche Ortszeit ohne Suffix	`datetime_local` bleibt `%Y-%m-%d %H:%M:%S`; UTC wird vorher aus dem offsetbehafteten Step-1-Wert bestimmt. Winter, Sommer, Tageswechsel und beide DST-Wechsel getestet.
`time_has_issues`	False genau fuer `timestamp_source=localtimes`. Fallback `datetime`, unbekannte und fehlende Herkunft ergeben True. Eine ungueltige lokale Uhrzeit wird zusaetzlich durch Metadaten-/Readinesschecks erkannt.
Rohdaten statt Clean-Auswahl	Alle IDs bleiben in der Tabelle, auch mit Fallbackzeit, 39 s / 0 s Audio, fehlender Dauer oder problematischen Sentinel-Bildern. Die vorgeschlagenen Analysefilter werden erst vom Verbraucher angewendet.
`duration_seconds`	Gepruefte Dauer aus dem detaillierten Audioinventar. Null wenn unbekannt; bei mehreren Dateien Minimum der bekannten Dauern. Technische Dateiprobleme bleiben separat geflaggt. Metadaten-Refresh kann die bisherige Dauer erhalten.
Audio ausserhalb 59-61 s	`duration_not_60s` bleibt als Hinweis; allein setzt der Code weder im Inventar noch Download-QC noch Master eine Issue-Flag. Dekodierungsfehler und fehlende/ungueltige Eigenschaften bleiben problematisch. Die 40-s-Grenze wird nicht automatisch angewendet.
Wetter `missing_value`	Code bleibt sichtbar, setzt allein keine Issue-Flag. Zeit-, Struktur-, Plausibilitaets- und Lesefehler bleiben problematisch. Auch alte kompakte Inventare werden hinsichtlich dieses Hinweises korrekt interpretiert.
`_exists`	Inventarstatus der physischen Datei; keine Live-Dateipruefung und keine Qualitaetsfreigabe.
Fehlendes QC	Fehlende Issue-Checks gelten nicht als bestanden. Vorhandene Audiodatei mit unbekannter Dauer erhaelt `duration_unavailable`.
Sentinel	Score 0 bleibt 0 und setzt `too_cloudy_sentinel_not_available`. Physisch vorhandene Datei kann weiter exists=True sein. Fehlender Score setzt `quality_score_missing`.
Grid und direkte LRT-Lage	Grid-IDs implizieren keine Mehrheitsformation. `grid_*_has_majority_formation` und `inside_lrt_polygon` bleiben getrennte Kriterien.
Inkrementelle Laeufe	Zeitflag und abgeleitete Statuswerte werden auch auf behaltene Zeilen angewendet; keine leeren neuen Zeitflags. Statusereignisse erfassen auch diese Migration.
Bioakustik-Updates	Neue Zeitflag, nullable Dauer-Spalte und v7-Version werden ergaenzt. Auf vollstaendigen Masters werden die kombinierten Flags konsistent neu abgeleitet. Audio-only-Bioakustik bleibt unabhaengig von der Zeitquelle.
Optionaler kompakter Export	Enthaelt auch Zeitflag, Zeitquelle, Audiodauer und die drei LRT-/Mehrheitskriterien. Der vollstaendige Master bleibt das Standardprodukt.

`ready_for_*` bleibt ausschliesslich eine abgeleitete Spalte, keine Zeilenauswahl. Kombinierte Flags beruecksichtigen nun die Zeitquelle; `ready_for_bioacoustic_analysis` bleibt davon unabhaengig. Bestehende manuelle Freigaben bleiben

erhalten und ersetzen keinen aktuellen Analysefilter.

Dokumentation und weitere Korrektur

Masterreferenz, ci-tec-README, Haupt-README, lokales Handbuch, Phasenbeschreibung, deutsche/englische Step-7-Texte und generierte Diagnoseverweise wurden aktualisiert. Historische Auditberichte bleiben historische Nachweise. Die beiden gepflegten PDF-Handbuecher werden aus ihren Markdownquellen neu erstellt.

Die vorhandene editierbare LaTeX-Fassung wurde ebenfalls mit der Markdownquelle synchronisiert. Der eingebaute LaTeX-Compiler konnte wegen `windows sandbox: helper_unknown_error: setup refresh had errors` nicht starten; eine erfolgreiche LaTeX-Kompilation wird daher nicht behauptet. Die ausgelieferten PDFs werden unabhaengig davon mit ReportLab erzeugt und geprueft.

Bei der umfassenden Pruefung wurde ein separater Windows-Fehler gefunden: `os.kill(pid, 0)` erzeugt dort ein Konsolensignal statt einer POSIX-Prozessabfrage. Der lokale Orchestrator fragt jetzt ueber Windows-Prozesshandles ab, ob eine Sperre einem lebenden Prozess gehoert. Bei fehlenden Zugriffsrechten bleibt die Sperre vorsichtshalber erhalten. Ein echter gestarteter und anschliessend beendeter Testprozess prueft diese Unterscheidung.

Validierung

Gezielte Regressionen pruefen den PDF-Datenvertrag, die unveraenderte Zeilenpopulation, identische CSV-/Parquet-Spalten, alte Inventare, fehlende Zeitquellen, inkrementelle Migration, Statusereignisse und technische Fehler. Der vollstaendige Lauf von `tools/functionality_test.py --config config.horeka.json` hat am 12.10.2026 bestanden: alle 25 Testmodule sowie Imports, Konfiguration, Schemas, Step-Registry, Dokumentationsreferenzen und Syntax. Darunter sind zwoelf neue PDF-Vertragsfaelle und sieben Phasentests mit direkten/hybriden Horeka-Startwegen und lokalem Bioakustiklauf. Das Protokoll liegt unter `output/schema_pdf_audit_20261009/functionality_test.log`.

Der Quelltexttest prueft gepflegte Projektdateien und ueberspringt virtuelle Python-Umgebungen, Daten-Snapshots, Logs und temporaere Exporte. Der zuvor gemeldete Latin-1-Fall war eine absichtlich anders kodierte Joblib-Testdatei im lokalen Bacpipe-Environment, kein Fehler im Pipelinecode. Gemeinsame historische `README.md`-Dateien werden vom Dokumentationscheck akzeptiert. Die beiden PDF-Handbuecher werden neu erstellt, gerendert und visuell kontrolliert.

Anwendung auf vorhandene Daten

Diese Aenderung aktualisiert den lokalen Code und die Dokumentation. Ein Produktivlauf mit allen LSDF-Daten und eine Veroeffentlichung auf Horeka/LSDF sind damit nicht behauptet. Ein regulaerer Kernlauf erstellt v7 aus den Inventaren. Ein inkrementeller Lauf migriert die behaltenen Zeitflags ebenfalls. Ein reiner Bioakustiklauf kann fehlende historische Audiodauern nur als null kennzeichnen; zum Befuellen ist ein Kernlauf mit dem Audioinventar erforderlich. Es werden keine Audiodateien, Wetterbeobachtungen oder Sentinel-Bilder durch diese Anpassung veraendert.

10 Bio-O-Ton Pipeline: Schritte, Dateien und Abhaengigkeiten

PIPELINE_SCHRITTE_DE.md

Diese Datei beschreibt den durch `submit_bio_o_ton_horeka.sh` orchestrierten HoreKa-Lauf. Alle Pfade stammen aus `config.horeka.json`.

Speicherregeln

Generierte Analyse-, Inventar-, Status- und QC-Dateien:

```
/lsdf/kit/ipf/projects/Bio-O-Ton/Data_automatisation_skripts/outputs/<step>
```

Die finalen Mastertable-Dateien liegen direkt unter `Data_automatisation_skripts/outputs/`. Fachliche Downloads liegen unter:

```
Audio      -> PointData/SoundRecordings
Fotos      -> PointData/Images_SoundRecordings
Sentinel-2 -> PointData/S2
Punktwetter-> PointData/Weather/Hostrada
```

Zwei separat gestartete Phasen

`bash slurm_add_new_ids.sh` verarbeitet den Kern (Steps 1-5, Master, Validierung). Erst nach dessen Abschluss startet `bash slurm_bioacoustics.sh` die Steps 6.0-6.6 und aktualisiert nur Bioakustik-Masterfelder. `from_scratch` baut den Kern neu auf. Siehe Betrieb und Horeka 2.

Pipeline-Steuerung

Run-Plan

Tool: `tools/plan_pipeline_run.py`

Inputs:

```
PointData/dawn-chorus-soundscape.csv
outputs/step_1_metadata/metadata_source_fingerprints.csv
Bio_O_Ton_Master.csv
State-Dateien der globalen LRT/Grid-Schritte
```

Outputs:

```
outputs/step_0_control/run_plans/<run_id>/run_plan.json
outputs/step_0_control/run_plans/<run_id>/*_ids.csv
outputs/step_0_control/full_rebuild/current.json
outputs/step_0_control/full_rebuild/<generation_id>/completed_steps/*.json
```

Der Plan unterscheidet neue, geaenderte, geloeschte und bisher problematische IDs. Globale LRT/Grid-Schritte laufen nur bei geaenderten Inputs, fehlenden Outputs oder im Modus `from_scratch`. Ein unterbrochener Vollauf submitiert nur Steps ohne Erfolgsmarker erneut.

Pipeline-Lock

Tool: `tools/pipeline_lock.py`

Output:

```
outputs/step_0_control/pipeline.lock/owner.json
```

Der Lock verhindert parallele Gesamtlaeufe. Der letzte Slurm-Job entfernt ihn nach dem Validierungsjob.

Step 1: Metadaten und Quellfingerprints

Skript: `scripts/Step_1_metadata_extraction.py`

Input:

```
PointData/dawn-chorus-soundscape.csv
```

Output:

```
outputs/step_1_metadata/dawnchorus_metadata_clean.csv
outputs/step_1_metadata/dawnchorus_metadata_log.csv
outputs/step_1_metadata/metadata_source_fingerprints.csv
```

Der Step bereinigt ID, Zeitstempel und GPS. Er fuehrt atomare, ID-basierte Upserts aus und speichert getrennte Fingerprints fuer Metadaten, Audio, Foto, Wetter und Sentinel. Aenderungen bestehender IDs werden damit erkannt.

Abhaengig: Step 2_2, Step 3_1, Step 4, Step 5.1/5.2 und Step 6.

Step 2_0: LRT-Bereinigung und Formation

Step 2 wird variantenweise ausgefuehrt. `<suffix>` bezeichnet den Dateinamen nach `All_Bundeslander_`; Details stehen in `Readmes/step_2_variants/README_DE.md`. Die folgenden Pfade sind daher Pfadvorlagen.

Skript: `scripts/Step_2_0_clean_lrts.py`

Inputs:

```
Biodiversity_data/Bundeslander/All_Bundeslander/All_Bundeslander_<suffix>.gpkg
```

Outputs:

```
outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg
outputs/step_2_variants/<suffix>/step_2_0/state.json
```

Der Step normalisiert LRT-Polygone, Status und Formation. Das finale GPKG wird zuerst als Part-Datei geschrieben, gelesen/gezaehlt und danach atomar ersetzt.

Abhaengig: Step 2_1, Step 2_2 und Step 2_4.

Step 2_1: 100m Formation-Status

Skript: `scripts/Step_2_1_merge_lrts_and_grid.py`

Inputs:

```
InspireGrid/Vector_Data/grid.gpkg
outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg
```

Kernoutputs:

```
outputs/step_2_variants/<suffix>/step_2_1/LRT_Grid_Majority_<suffix>.csv
outputs/step_2_variants/<suffix>/step_2_1/majority_formation_grid_<suffix>.gpkg
outputs/step_2_variants/<suffix>/step_2_1/majority_formation_grid_<suffix>.parquet
outputs/step_2_variants/<suffix>/step_2_1/state.json
```

Vergleichbare Formation-Status-Matrix:

```
outputs/step_2_variants/<suffix>/step_2_1_susi_compatible/Formation_Status_Grid_withLRTCode.parqu
et
```

Der Overlay laeuft in Chunks. Flaechenanteile werden als Centi-Prozent gespeichert (`10000 = 100.00%`). `majority_disputed` gilt bei `majority_delta <= 200`.

Abhaengig: Step 2_2, Step 2_3 und Step 2_4.

Step 2_2: Recording-Punkte zu 100m Grid/LRT

Skript: `scripts/Step_2_2_assign_points_to_lrt_grid.py`

Inputs:

```
outputs/step_1_metadata/dawnchorus_metadata_clean.csv
outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg
outputs/step_2_variants/<suffix>/step_2_1/LRT_Grid_Majority_<suffix>.csv
InspireGrid/Vector_Data/grid.gpkg
```

Outputs:

```
outputs/step_2_variants/<suffix>/step_2_2/DawnChorus_LRT_Grid_Assignment_<suffix>.csv
outputs/step_2_variants/<suffix>/step_2_2/DawnChorus_LRT_Polygon_Matches_<suffix>.csv
outputs/step_2_variants/<suffix>/step_2_2/point_processing_log_<suffix>.csv
outputs/step_2_variants/<suffix>/step_2_2/state.json
```

Nur IDs mit geaenderten Koordinaten beziehungsweise durch Step 2_1 invalidierte IDs werden ersetzt. Die drei CSVs werden atomar per ID aktualisiert.

Abhaengig: Step 6.

Step 2_3: Groebere Grid-Aggregationen

Skript: `scripts/Step_2_3_generate_remaining_grid_products.py`

Input:

```
outputs/step_2_variants/<suffix>/step_2_1/majority_formation_grid_<suffix>.parquet
```

Outputs:

```
outputs/step_2_variants/<suffix>/step_2_3/*1km*
outputs/step_2_variants/<suffix>/step_2_3/*5km*
outputs/step_2_variants/<suffix>/step_2_3/*10km*
outputs/step_2_variants/<suffix>/step_2_3/state.json
```

Abhaengig: Abschlussvalidierung.

Step 2_4: 10m Formation-Status

Skript: `scripts/Step_2_4_generate_10m_formation_status_products.py`

Inputs:

```
outputs/step_2_variants/<suffix>/step_2_1_susi_compatible/Formation_Status_Grid_withLRTCode.parqu
et
outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg
```

Outputs:

```
outputs/step_2_variants/<suffix>/step_2_4_susi_10m/grid10m_chunks/*.gpkg
outputs/step_2_variants/<suffix>/step_2_4_susi_10m/ix_chunks/*.csv
outputs/step_2_variants/<suffix>/step_2_4_susi_10m/parquet_10/*.parquet
outputs/step_2_variants/<suffix>/step_2_4_susi_10m/Formation_Status_10m_Grid_withLRTCode_<suffix>
.parquet
outputs/step_2_variants/<suffix>/step_2_4_susi_10m/state.json
```

Die 100m- und 10m-Matrizen verwenden dieselbe Spalten-, Skalierungs- und Majority-Logik. Chunks werden nur bei geaenderten LRT/100m-Inputs invalidiert.

Abhaengig: Step 6.

Step 2_5/2_6: Oeffentliche Referenzprodukte

Skripte:

```
scripts/Step_2_5_clean_public_lrts.py
scripts/Step_2_6_merge_public_lrts_and_grid.py
```

Diese optionale Vergleichsroute wird mit `run_public_lrt_products.sh` gestartet. Sie ist nicht Teil des regulaeren ID-Laufs.

Step 3_0_a: Audio-Inventar

Skript: `scripts/Step_3_0_a_audio_inventory.py`

Input: `PointData/SoundRecordings/*`

Outputs:

```
outputs/step_3_0_a_audio_inventory/audio_inventory_detailed.csv
outputs/step_3_0_a_audio_inventory/audio_inventory_compact.csv
outputs/step_3_0_a_audio_inventory/state.json
```

Prueft Dateiname, Container, Codec, Decodebarkeit, Sample-Rate, Kanaele und Aufnahmedauer. Fehlerfreie unveraenderte Dateien koennen wiederverwendet werden; Fehler werden erneut geprueft.

Step 3_0_b: Foto-Inventar

Skript: `scripts/Step_3_0_b_photo_inventory.py`

Input: `PointData/Images_SoundRecordings/*`

Outputs:

```
outputs/step_3_0_b_photo_inventory/photo_inventory_detailed.csv
outputs/step_3_0_b_photo_inventory/photo_inventory_compact.csv
outputs/step_3_0_b_photo_inventory/state.json
```

Prueft Namensschema, Lesbarkeit, Pixeldekodierung, Format und Dimensionen.

Step 3_1_a/3_1_b: Medien-Downloads

Skripte:

```
scripts/Step_3_1_a_audio_download.py
scripts/Step_3_1_b_photo_download.py
```

Downloadquellen:

```
Spalten audio und photo aus dawn-chorus-soundscape.csv
```

Downloadziele:

```
PointData/SoundRecordings/<id>_audio.<ext>
PointData/Images_SoundRecordings/<id>_photo.<ext>
```

Status:

```
outputs/step_3_1_a_audio_download/audio_download_retry_log.csv
outputs/step_3_1_b_photo_download/photo_download_retry_log.csv
```

Downloads sind parallel, atomar und wiederaufnehmbar. Eine geaenderte URL setzt den Retry-Zaehler der betroffenen ID zurueck.

Step 4_1: Sentinel-2 Drive-Mirror

Skript: `scripts/Step_4_1_Sentinel2_download.py`

Quelle: konfigurierte Google-Drive-Folder-ID.

Ziel und Status:

```
PointData/S2/<Drive-Dateiname>.tif
outputs/step_4_1_sentinel2_download/download_log.csv
outputs/step_4_1_sentinel2_download/_file_status/*.json
```

Remote-Dateien werden anhand Drive-ID, MD5, Aenderungszeit und Groesse abgeglichen. Ohne verwendbare nichtinteraktive Authentifizierung wird der Step als `skipped` dokumentiert.

Step 4_0: Sentinel-2 Inventar

Skript: `scripts/Step_4_0_Sentinel2_inventory.py`

Inputs:

```
PointData/S2/*
PointData/S2_Scores.csv
```

Outputs:

```
outputs/step_4_0_Sentinel2_inventory/sentinel2_inventory_detailed.csv
outputs/step_4_0_Sentinel2_inventory/sentinel2_inventory_compact.csv
outputs/step_4_0_Sentinel2_inventory/state.json
```

Abhaengig: Step 6.

Step 5_1: Punktwetter-Inventar

Skript: `scripts/Step_5_1_Weather_inventory.py`

Inputs:

```
PointData/Weather/Hostrada/weather_<id>.csv
outputs/step_1_metadata/dawnchorus_metadata_clean.csv
```

Outputs:

```
outputs/step_5_1_weather_inventory/weather_inventory_detailed.csv
outputs/step_5_1_weather_inventory/weather_inventory_compact.csv
outputs/step_5_1_weather_inventory/state.json
```

Prueft pro CSV Zeilenzahl, Pflichtspalten, Zeitfenster, Stundenintervall, fehlende Werte und Wertebereiche. Unveraenderte fehlerfreie Dateien werden anhand Groesse, mtime und erwartetem Zeitfenster wiederverwendet. Step 5.1 laeuft vor und nach Step 5.2.

Step 5_2: Punktwetter erzeugen

Skript: `scripts/Step_5_2_download_weather_data.py`

Inputs:

```
outputs/step_1_metadata/dawnchorus_metadata_clean.csv
outputs/step_5_1_weather_inventory/weather_inventory_compact.csv
DWD Open Data HOSTRADA
```

Downloads und Status:

```
PointData/Weather/Hostrada/weather_<id>.csv
outputs/step_5_2_weather_download/hostrada_cache/*
outputs/step_5_2_weather_download/run_log.txt
PointData/Weather/Hostrada/_recording_status/<id>.json
```

Neue, geaenderte und problematische IDs werden verarbeitet. Zeit- oder GPS-Aenderungen erzwingen die Neuberechnung auch bei zuvor fehlerfreier CSV. Grosse ID-Mengen werden auf bis zu acht deterministische Slurm-Array-Shards verteilt, von denen standardmaessig vier parallel laufen. Gemeinsame Monatsdownloads sind per Lock geschuetzt. Ein abschliessender Verify-Job prueft die Vollstaendigkeit aller angeforderten Wetter-CSVs.

Step 5_3: HOSTRADA Monats-NetCDF

Skript: `scripts/Step_5_3_download_hostrada_monthly.py`

Quelle: DWD Open Data HOSTRADA fuer sechs Variablen.

Outputs:

```
outputs/step_5_3_hostrada_monthly_download/netcdf/<Variable>/*.nc
outputs/step_5_3_hostrada_monthly_download/download_log.csv
```

Vorhandene nichtleere Monatsdateien werden uebersprungen.

Step 5_4: HOSTRADA 100m Raster

Orchestrator: `tools/run_hostrada_raster_all.py`

Worker: `scripts/Step_5_4_prepare_hostrada_rasters.py`

Input: vollstaendige Variable/Jahr-Gruppen aus Step 5.3.

Outputs:

```
outputs/step_5_4_hostrada_raster_products/Hostrada_<Variable>/*.tif
outputs/step_5_4_hostrada_raster_products/Hostrada_<Variable>/_tile_status/<year>/*.json
outputs/step_5_4_hostrada_raster_products/Hostrada_<Variable>/_force_state/<Variable>_<year>.json
```

Bei `from_scratch` werden die Statusdateien pro Generation einmal invalidiert. Ein persistenter `in_progress`-Status setzt auch nach einem neuen Slurm-Submit an den neu fertiggestellten Tiles fort. Erst ein vollstaendig beendeter Vollauf setzt die Generation auf `complete`.

Step 5_5: HOSTRADA Raster-QC

Skript: `scripts/Step_5_5_check_hostrada_raster_products.py`

Input: alle Rastervariablen unter Step 5.4, rekursiv.

Outputs:

```
outputs/step_5_5_hostrada_raster_quality_check/hostrada_raster_quality.csv
outputs/step_5_5_hostrada_raster_quality_check/hostrada_raster_quality.json
outputs/step_5_5_hostrada_raster_quality_check/hostrada_raster_quality.md
outputs/step_5_5_hostrada_raster_quality_check/state.json
```

Unveraenderte fehlerfreie Raster werden wiederverwendet; alte Fehler oder geaenderte Dateien werden erneut gelesen.

Step 6: Bioakustische Embeddings und Arteninferenz

Step 6 verarbeitet nur Audiodateien, die im Post-Download-Inventar von Step 3_0_a technisch validiert wurden. Die Inferenz laeuft als Modell-mal-Shard-Slurm-Array auf GPU-Knoten und setzt nach einem Timeout am letzten Batch-Checkpoint fort.

```
Step 6_0  Bacpipe-/CUDA-/Modell-Preflight und Modellregistry
Step 6_1  ID- und modellbezogene Worklist aus validen Audios
Step 6_2  segmentweise Embeddings und native Top-k-Klassenscores
Step 6_3  einheitliches Vorhersageschema und Scorefilter
Step 6_4  Taxonomieharmonisierung, Deutschland- und Saisonplausibilitaet
Step 6_5  Aggregation pro ID und Taxon
Step 6_6  Vollstaendigkeits- und Fehler-QC pro ID und Modell
```

Wichtigste Outputs:

```
outputs/step_6_0_bioacoustic_model_preflight/model_registry.json
outputs/step_6_1_bioacoustic_worklist/worklist.parquet
outputs/step_6_2_bioacoustic_embeddings/model=*/part-*.parquet
outputs/step_6_3_species_predictions_raw/model=*/predictions.parquet
outputs/step_6_4_species_predictions_germany/model=*/predictions.parquet
outputs/step_6_5_bioacoustic_recording_summary/recording_summary.csv
outputs/step_6_5_bioacoustic_recording_summary/recording_species.parquet
outputs/step_6_6_bioacoustic_quality_control/bioacoustic_qc_compact.csv
outputs/step_6_6_bioacoustic_quality_control/bioacoustic_qc_detailed.csv
```

Die Deutschland- und Saisonpruefung ist eine Plausibilitaetskontrolle, keine Beobachtungsbestaetigung. Modell-Scores verschiedener Architekturen sind nicht automatisch kalibriert oder direkt vergleichbar. Details: `Readmes/step_6_bioacoustics/README_DE.md`.

Step 7_0: Finale Mastertabelle

Skript: `scripts/Step_7_0_update_master_table.py`

Inputs: ID-Level-Outputs aus Step 1, 2, 3, 4, 5 und 6.

Outputs:

```
Bio_O_Ton_Master.csv
Bio_O_Ton_Master.parquet
Bio_O_Ton_Master_summary.json
outputs/step_0_control/status_events.csv
```

Die Mastertabelle verwendet Step 5.1 als einzige Wetter-QC-Quelle und startet keinen zweiten Vollscan. Sie speichert kanonische Statuswerte pro Domaene, Readiness-Flags, Issue-Codes, Quellfingerprint, Workflow-Run-ID und manuelle Freigabefelder. Der Orchestrator aktualisiert sie fortlaufend nach jedem relevanten Datenbereich: ID-spezifische Updates ersetzen nur die betroffenen Zeilen, globale Formation- oder Rasteraenderungen aktualisieren den gesamten Bestand. Die Masterupdate-Jobs werden serialisiert, damit keine Zwischenupdates einander ueberschreiben.

Abschlussvalidierung

Tool: `tools/final_validation_report.py`

Outputs:

```
outputs/step_9_validation/final_validation_<timestamp>.json
outputs/step_9_validation/final_validation_<timestamp>.md
```

Geprueft werden Pflichtartefakte, nur die Manifeste des aktuellen Runs, fehlende geplante Steps und die in `final_validation` konfigurierten Mastertable-Readiness-Regeln. Technischer Status und manuelle Freigabe sind getrennt.

Abhaengigkeitsgraph

```
Dawn Chorus CSV -> Step 1
Step 1 -> Step 2_2
Step 1 -> Step 3 Downloads
Step 1 -> Step 4 Sentinel
Step 1 -> Step 5.1/5.2

LRT sources -> Step 2_0 -> Step 2_1
Step 2_1 -> Step 2_2
Step 2_1 -> Step 2_3
Step 2_1 -> Step 2_4

Medienordner -> Step 3.0 -> Step 3.1
Drive -> Step 4.1 -> Step 4.0
Punktwetter -> Step 5.1 -> Step 5.2 -> Step 5.1 post
DWD Monatsdaten -> Step 5.3 -> Step 5.4 -> Step 5.5

Kern: Step 1/2/3/4/5 -> Master -> Validierung -> Lock-Freigabe
Separater Start: valides Audio -> Step 6_0/6_1 -> Step 6_2 -> Step 6_3/6_4/6_5/6_6
Bioakustik -> Bioakustik-Masterfelder -> Validierung -> Lock-Freigabe
Direkter Slurm-Start: serielle Teilupdates; Hybrid-Start: finales Master-Update
```

11 Bio-O-Ton: lokaler Windows-Lauf mit LSDF

scripts_local_run/README.md

Getrennte Phasen (Stand 2026-09-23)

`-Mode add_new_ids` und `-Mode from_scratch` fuehren keine Bioakustik aus und benoetigen dafuer kein Bacpipe-Environment. Nach Abschluss separat starten:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline_local.ps1 -Mode bioacoustics
```

Dieser Lauf nutzt die vorbereiteten Metadaten und das Audioinventar, gleicht alle aktuellen IDs ab und verwendet vorhandene Inferenz-Checkpoints. Das Master-Update aendert Bioakustikfelder und migriert die Master-Zeitflag sowie kombinierte Readiness auf Schema v7. Details und Horeka 2.

Master-Schema gemaess PDF vom 09.10.2026

Lokale und Horeka-Laeufe verwenden denselben Step-7.0-Code. `time_has_issues` ist nur bei `timestamp_source=localtimes` False; Fallback, unbekannte oder fehlende Zeitquelle ergeben True. Inkrementelle Laeufe migrieren auch unveraenderte Zeilen. `duration_seconds` stammt aus dem Audio-QC; reine Wetter-`missing_value`-Codes bleiben als Hinweis erhalten und setzen allein keine Issue-Flag. Ein regulaerer Kernlauf oder der fokussierte Metadaten-Refresh erstellt die neue Mastertabelle. Bestehende LSDF-Produkte werden durch eine Codeaenderung allein nicht aktualisiert. Pruefbericht.

Dieser Ordner startet dieselben fachlichen Python-Skripte wie die Cluster- Pipeline. Slurm wird lokal nicht emuliert. `local_orchestrator.py` bildet den Abhaengigkeitsgraphen, parallele unabhaengige Schritte, Arrays, Manifeste, Checkpoints und Mastertable-Updates mit lokalen Prozessen nach.

Speicherorte

Inhalt	Speicherort
Pipeline-Code	uebergeordneter Git-Ordner
Generierte Outputs	`<workspace_dir>/outputs/step_*`
Finale Mastertabelle	`<workspace_dir>/outputs/Bio_O_Ton_Master.*`
Lokale Logs	`<workspace_dir>/outputs/step_0_local_logs`
Grosse/statische LSDF-Eingaben	`<workspace_dir>/lsdf_cache`
Python-Environments	`<environment_dir>/core` und `<environment_dir>/bacpipe`
Audio-Originale	LSDF `PointData/SoundRecordings`
Foto-Originale	LSDF `PointData/Images_SoundRecordings`
Sentinel-2-TIFs	LSDF `PointData/S2`
Punkt-Wetter-CSV	LSDF `PointData/Weather/Hostrada`

Die vier Originalverzeichnisse werden ueber ein eingebundenes LSDF-Laufwerk direkt gelesen und beschrieben. Vor dem ersten lokalen `add_new_ids` werden die bereits auf Horeka erzeugten Produkte einmalig nach `<workspace_dir>/outputs` uebernommen. Dadurch verwendet der lokale Planner die vorhandenen Fingerprints, Inventare, Checkpoints und die Mastertabelle. Remote-Locks, Run-Plaene und alte Slurm-Logs werden dabei ausgeschlossen.

Voraussetzungen

1. Windows 10/11, Python 3.11 und PowerShell.
2. WinFsp und SSHFS-Win muessen installiert sein.
3. Das LSDF-Passwort liegt im Windows Credential Manager.

4. Fuer den Sentinel-Drive-Mirror liegt `credentials.json` im Git-Hauptordner.

Passwort einmalig in der `BioTon`-Python-Umgebung speichern:

```
import keyring
keyring.set_password("lsdf_kit", "jk3038", "DEIN_PASSWORT")
```

Das Passwort wird von `mount_lsdf.py` nur an SSHFS uebergeben und weder in Konfigurationen noch in Logs geschrieben.

Der Mount verwendet den offiziellen SSHFS-Win-Netzwerkprovider mit der Root-UNC `\\sshfs.r\...`. Das Keyring-Passwort wird direkt ueber die Windows-Netzwerk-API uebergeben und erscheint nicht in der Prozess-Kommandozeile.

Ist ein altes Laufwerks-Mapping vorhanden, aber nicht mehr lesbar, wird es vor dem Neuaufbau entfernt. Akzeptiert der Windows-Netzwerkprovider die Verbindung nicht, versucht der Helper einmalig `sshfs.exe` direkt; auch dabei wird das Passwort nur ueber die Standardeingabe uebergeben.

Lokale Einstellungen

Beim ersten Start wird `local.settings.json` aus `local.settings.example.json` erzeugt. Die Datei ist nicht fuer Git bestimmt. Standardprofil fuer den aktuellen Rechner:

```
20 nutzbare logische CPUs von 24
maximal 2 unabhaengige Pipeline-Schritte gleichzeitig
maximal 2 lokale Array-Tasks gleichzeitig
1 paralleler Bioakustik-Task bei CUDA, sonst 2
128 GB RAM werden nicht fest reserviert; die Step-Checkpoints begrenzen Spitzen
```

`workspace_dir` steht standardmaessig auf `D:/BioOTon_local_workspace` und kann in `local.settings.json` geaendert werden.

Environment erzeugen

```
cd "C:\Users\Frede\OneDrive\Projects and Disschaptors\Bioton_automatisation\scripts_local_run"
powershell -ExecutionPolicy Bypass -File .\setup_local_env.ps1
```

Es werden zwei kompatible Environments ausserhalb des tiefen OneDrive-Pfads erzeugt. Standard ist `D:/BioOTon_envs`:

```
core     Geodaten, Medien, Wetter, Reports und Orchestrierung
bacpipe  Bacpipe, TensorFlow und PyTorch fuer Step 6
```

Der kurze Pfad verhindert Windows-Installationsfehler durch sehr lange TensorFlow-Dateinamen. Ein unvollstaendiges Environment kann explizit neu erzeugt werden:

```
powershell -ExecutionPolicy Bypass -File .\setup_local_env.ps1 -Recreate
```

Start

Schlanker Neuaufbau der Mastertabelle

Wenn nur der Deutschland-Filter, die primaeren 100-m/10-m-Formationen und die Mastertabelle aktualisiert werden sollen, verwende den gezielten Starter:

```
powershell -ExecutionPolicy Bypass -File .\run_master_refresh_local.ps1
```

ci-tec verwendet die gemeinsame `outputs/Bio_O_Ton_Master.csv`. Der Starter erzeugt keine zweite Tabelle mehr. Das folgende Werkzeug bleibt nur fuer ausdruecklich benoetigte manuelle Altexporte verfuegbar; solche Kopien werden nicht automatisch aktuell gehalten:

```
python tools/export_ci_tec_master.py --input
D:/BioOTon_local_workspace/outputs/Bio_O_Ton_Master.csv
```

Neue IDs haben noch keine uebernommenen Domaenenpruefergebnisse. Deren Sound-, Sentinel- und Wetterflags koennen daher leer sein; leer bedeutet unbekannt und ist nicht mit `False` gleichzusetzen. Ein nachfolgender regulaerer Horeka-Lauf ergaenzt diese Ergebnisse aus Inventaren und Verarbeitungsschritten.

Er verarbeitet ausschliesslich Step 1, 2.0, 2.1, 2.2, 2.4 und 7.0. Es werden keine Audio-, Foto-, Sentinel-, Wetter- oder Bioakustik-Jobs gestartet; Step 2.3 wird ebenfalls ausgelassen. Die 100-m- und 10-m-Produkte werden aus der konfigurierten primaeren GPKG erzeugt. Der Starter bricht ab, solange ein Horeka-Pipeline-Lock vorhanden ist, damit niemals parallel in denselben Bestand gearbeitet wird. Er kopiert von Horeka ausschliesslich die bestehende Master-CSV als Basis; keine gesamte Output-Struktur und keine Medien-, Wetter- oder Rasterdaten werden lokal gespiegelt. Die unveraenderte Audio-, Foto-, Sentinel-, Wetter- und Bioakustik-Information wird aus dieser Master-Basis uebernommen, waehrend Metadaten sowie die primaeren 100-m/10-m-Felder neu berechnet werden.

Step 2.4 erzeugt dabei nur die 10-m-Zellen der in Step 2.2 tatsaechlich zugeordneten Aufnahmen, nicht das vollstaendige bundesweite 10-m-Produkt.

Die Mastertabelle bleibt standardmaessig lokal unter `<workspace_dir>/outputs/Bio_O_Ton_Master.csv`. Erst mit `-PublishToLsdf` werden die erfolgreichen lokalen Produkte nach LSDF veroeffentlicht:

```
powershell -ExecutionPolicy Bypass -File .\run_master_refresh_local.ps1 `
  -SkipEnvironmentSetup -PublishToLsdf
```

Inkrementell fuer neue, geaenderte oder problematische IDs:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline_local.ps1 -Mode add_new_ids
```

Die normale lokale Pipeline verarbeitet die konfigurierte Primaervariante. Alle GeoPackages im neuen Bundeslaender-Ordner werden separat mit folgendem Einstieg verarbeitet:

```
powershell -ExecutionPolicy Bypass -File .\run_step2_variants_local.ps1 `
  -Mode add_new_ids -SkipEnvironmentSetup
```

Die Eingangs-GPKGs werden vor der Verarbeitung nach `D:\BioOTon_local_workspace\lsdf_cache\Biodiversity_data\Bundeslander\All_Bundeslander` gespiegelt und danach lokal gelesen. Unveraenderte Dateien werden anhand von Groesse und Aenderungszeit wiederverwendet. Die Ergebnisse liegen lokal unter `outputs\step_2_variants\<suffix>` und werden nach einem erfolgreichen Lauf wie andere kompatible Outputs nach LSDF publiziert. Der erste Vollauf aller 12 Varianten ist rechen- und speicherintensiv; lokale Parallelitaet bleibt deshalb standardmaessig auf zwei Varianten begrenzt.

`add_new_ids` verwendet lokal bewusst einen schnellen LSDF-Modus:

1. Audio, Fotos und Wetter werden zunaechst nur als Dateiliste mit Groesse und

Aenderungszeit erfasst. Legacy-Dateien werden dabei nicht dekodiert oder inhaltlich gelesen.

2. Aus Metadaten und Dateiliste entsteht pro Datentyp eine atomare

`*_missing_ids.csv`.

3. Nur diese fehlenden IDs werden heruntergeladen. Medien werden direkt im

Download-Step dekodiert bzw. mit Pillow geprueft; neue Wetter-CSVs werden unmittelbar danach nur fuer die angeforderten IDs validiert.

4. Alte QC-Fehler werden in diesem schnellen Modus nicht erneut bearbeitet.

Der vollstaendige Legacy-Sanity-Check bleibt `from_scratch` und den regulaeren Horeka-Inventursteps vorbehalten.

Die schnellen Listen liegen unter `outputs/step_3_0_a_audio_inventory`, `outputs/step_3_0_b_photo_inventory` und `outputs/step_5_1_weather_inventory`. Sie werden bei jedem lokalen `add_new_ids`-Lauf neu erzeugt, auch wenn der aeltere Run-Plan keinen Downloadbedarf erkannt hatte.

Neuaufbau des Kerns (ohne Step 6) mit den gleichen `--force`-Regeln wie auf dem Cluster:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline_local.ps1 -Mode from_scratch
```

Schneller Funktionstest:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline_local.ps1 -Mode functionality_test
```

Dieser Modus mountet LSDF nicht und kopiert keine grossen Eingabedateien. Er installiert und benoetigt auch das Bacpipe-Environment nicht.

Nach dem ersten erfolgreichen Setup kann die Dependency-Pruefung uebersprungen werden:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline_local.ps1 -Mode add_new_ids
-SkipEnvironmentSetup
```

CPU fuer Step 6 erzwingen:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline_local.ps1 -Mode bioacoustics
-SkipEnvironmentSetup -CpuOnly
```

Vorhandene Horeka-Outputs

Der erste inkrementelle Lauf kopiert finale Horeka-Produkte, Statusdateien und Manifeste fortsetzbar in den lokalen Workspace. Reine Zwischenchunk-Ordner wie `grid10m_chunks`, `ix_chunks`, `parquet_10` und `_chunk_checkpoints` werden nicht ueber SSHFS uebertragen. Dasselbe gilt fuer die grossen, unveraenderlichen HOSTRADA-Downloadcaches aus Step 5.2 und 5.3. Diese werden von lokalen Steps direkt unter `L:\Data_automatisation_skripts\outputs` wiederverwendet; neu heruntergeladene Quelldateien ergaenzen dort den gemeinsamen Cache. Erzeugte Raster, Tabellen, States und Reports bleiben dagegen bis zur erfolgreichen Veroeffentlichung lokal. Bereits lokal neuere Dateien werden nicht ueberschrieben. Einzelne temporaere Netzwerkfehler werden wiederholt und als `partial` protokolliert; sie blockieren den lokalen Lauf nicht und werden beim naechsten Bootstrap erneut versucht. Spaetere Horeka-Ergebnisse werden nur auf ausdruecklichen Wunsch erneut abgeglichen:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline_local.ps1 `
  -Mode add_new_ids -SkipEnvironmentSetup -RefreshHorekaOutputs
```

`from_scratch` importiert keine Horeka-Produkte. Der Bootstrap kann bei einem inkrementellen Lauf mit `-SkipHorekaBootstrap` deaktiviert werden. Cluster und lokaler Lauf duerfen waehrend des Abgleichs nicht gleichzeitig schreiben.

Die nur fuer den separat gestarteten Public-LRT-Zweig vorgesehenen Dateien `InspireGrid/Vector_Data/grid_public.gpkg` und `Biodiversity_data/Bundeslander/LRT_Germany_Clean.gpkg` sind lokal optional. Fehlen sie, wird dies protokolliert, ohne die regulaere Pipeline oder den Step-2-Variantenlauf zu blockieren. Beim expliziten Start von Step 2.5/2.6 bleiben sie Pflichtinputs.

Lokalen Speicher fuer Step 2 begrenzen

Lokale Runs erzeugen standardmaessig keine `grid10m_chunks` und `ix_chunks`. Diese beiden QA-Zwischenprodukte werden fuer Resume und Endprodukte nicht benoetigt und koennen bei grossen Varianten jeweils viele GiB belegen. Die notwendigen `parquet_10`-Checkpoints bleiben bis zum erfolgreichen Merge lokal. Danach werden auch sie auf LSDF ausgelagert; das finale 10m-Parquet bleibt lokal und wird regulaer veroeffentlicht.

Bereits vorhandene Zwischenprodukte werden einmalig so verschoben:

```
powershell -ExecutionPolicy Bypass -File .\offload_step2_local_storage.ps1
```

Das Skript kopiert Datei fuer Datei nach `Data_automatisation_skripts/outputs/step_2_variants`, prueft standardmaessig die Dateigroesse und loescht die lokale Datei erst nach erfolgreicher Verifikation. Fuer eine vollstaendige Hash-Pruefung kann `-Verification sha256` verwendet werden; dies ist ueber SSHFS deutlich langsamer. Ein aktiver Pipeline-Lock verhindert die manuelle Auslagerung.

Erfolgreiche lokale Ergebnisse auf LSDF veroeffentlichen

Nach einem vollstaendig erfolgreichen `add_new_ids`- oder `from_scratch`-Lauf werden kompatible lokale Outputs automatisch in den kanonischen LSDF-Ordner `Data_automatisation_skripts/outputs` hochgeladen. Pfade in CSV-, JSON- und pfadhaltigen Parquet-Dateien werden dabei von Windows auf LSDF uebersetzt. Dadurch koennen die Horeka-Planung und ihre Checkpoints lokal abgeschlossene Arbeit erkennen und ueberspringen.

Nicht veroeffentlicht werden lokale Logs, Slurm-Logs, Manifeste, Run-Plaene, Locks, lokale Reports und die umgebungsspezifische Bacpipe-Model-Registry. Der Upload loescht keine Remote-Dateien, ueberschreibt keine neueren Remote-Dateien und bricht ab, wenn ein Remote-Pipeline-Lock existiert.

Fuer einen einzelnen Lauf kann die Veroeffentlichung abgeschaltet werden:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline_local.ps1 `
  -Mode add_new_ids -SkipEnvironmentSetup -SkipLsdfPublish
```

Dauerhaft steuerbar ist das Verhalten in `local.settings.json` mit `publish_successful_outputs_to_lsdf`.

HOSTRADA nur auf Horeka ausfuehren

Lokale Einstellungen verwenden standardmaessig `"hostrada_execution": "horeka_only"`. Dadurch markieren lokale Run-Plaene die Schritte 5.1 bis 5.5 mit `execution_policy:horeka_only`, und der lokale Orchestrator startet sie auch im schnellen `add_new_ids`-Modus nicht. Die Horeka-Pipeline bleibt unveraendert fuer Inventur, Download, Monats-NetCDF, Rastererzeugung und QC verantwortlich. Alle anderen geplanten Schritte laufen weiter lokal.

Der Horeka-Bootstrap uebernimmt weiterhin die kleinen States, Inventare und QC-Berichte, aber nicht die grossen HOSTRADA-Caches. So koennen Mastertable und Diagnose den Horeka-Stand verwenden, ohne die lokalen NetCDF- und Rasterdaten zu duplizieren. Im `horeka_only`-Modus wird dieser kleine Statusabgleich bei jedem inkrementellen lokalen Lauf aktualisiert; die ausgeschlossenen HOSTRADA-Payloads werden dabei nicht uebertragen.

Vorhandene lokale HOSTRADA-Payloads werden zuerst nur geprueft:

```
powershell -ExecutionPolicy Bypass -File .\cleanup_local_hostrada.ps1
```

Das Skript verlangt fuer jede lokale Datei ein gleich grosses Gegenstueck im kanonischen Horeka/LSDF-Outputordner. Fehlt eine Datei oder weicht ihre Groesse ab, wird ohne Loeschung abgebrochen. Nach erfolgreicher Pruefung entfernt folgender Aufruf die lokalen Payload-Ordner:

```
powershell -ExecutionPolicy Bypass -File .\cleanup_local_hostrada.ps1 -Apply
```

Ein aktiver lokaler Pipeline-Lock blockiert das Cleanup. Kleine Status- und Logdateien bleiben absichtlich lokal; entfernt werden nur `hostrada_cache`, die Monats-`netcdf`-Daten und die erzeugten HOSTRADA-Rasterprodukte.

Resume und Fehler

Ein Abbruch mit `Ctrl+C` behaelt alle fachlichen Checkpoints. Beim naechsten Aufruf erkennt `plan_pipeline_run.py` neue und problematische IDs erneut und die Steps verwenden vorhandene Chunk-, Datei- und Batch-States.

Lokale Logs tragen einen UTC-Zeitstempel. Ein Step-Fehler laesst nur wirklich unabhaengige Zweige weiterlaufen; alle deklarierten Abhaengigkeiten verlangen einen erfolgreichen Vorgaenger. Vor jedem LSDF-abhaengigen Step wird der Mount mit einer kleinen Sentinel-Datei geprueft. Bei einem erkannten Transportfehler wird einmal neu verbunden und der Step einmal wiederholt. Mastertable-Updates und die finale Validierung laufen nicht auf fehlgeschlagenen Teilprodukten.

Das lokale Lock liegt ausschliesslich im lokalen Workspace. Cluster- und lokaler Lauf duerfen trotzdem nicht gleichzeitig Originaldateien herunterladen oder auf dem LSDF pruefen, da beide dieselben Medienverzeichnisse verwenden.

LSDF trennen

Nach Ende aller lokalen Pipeline-Prozesse:

```
net use L: /delete /y
```

Audited recording-time correction

`scripts_local_run/refresh_recording_times.py` applies an already audited time correction to the local workspace. It verifies source/master hashes against the supplied audit JSON, acquires the local lock, backs up all replaced metadata and master products, runs Steps 1 and 7, and independently checks all resulting times. It does not generate a second ci-tec CSV, publish to LSDF or run Horeka jobs. Use `--help` for required paths. The full CSV retains technical UTC, date and time fields; ci-tec uses `datetime_local`, without timezone, as the recording clock.

12 HoreKa Recovery-Runbook

HOREKA_RECOVERY_RUNBOOK_DE.md

Dieses Runbook beschreibt die sichere Reihenfolge nach Codeaenderungen. Es veraendert oder loescht keine LSDF-Quelldaten.

Vor dem Submit

1. Sicherstellen, dass keine alte Pipeline mehr laeuft.

2. Eine verwaiste Pipeline-Sperre erst nach der Jobkontrolle freigeben.

3. Den Git-Stand aktualisieren und den Commit-Hash notieren.

4. Hauptumgebung pruefen; Bacpipe nur fuer die separate Bioakustikphase benoetigt.

5. Zuerst den zehnminuetigen Funktionstest starten.

Technische Recovery-Regeln

- Der Wetterplan nimmt neue/geaenderte IDs, Mastertable-Probleme und Problem-IDs

aus dem letzten kompakten Wetterinventar auf.

- Step 5_2 ist geshardet und setzt vorhandene gueltige CSVs fort. Noch nicht

veroeffentlichte DWD-Monate werden als `upstream_unavailable` protokolliert, nicht als interner Python-Fehler.

- HOSTRADA-Jahresdateien werden anhand der zehnstelligen Zeitstempel erkannt.

Der Step-5_4-Verifier darf bei null erkannten vollstaendigen Jahren nicht erfolgreich enden.

- Mastertable-Readiness wird standardmaessig im Modus `report_only` berichtet.

Fehlende technische Pflichtartefakte und fehlgeschlagene Manifeste bleiben harte Fehler. Nicht freigabebereite Einzel-IDs bleiben sichtbar, blockieren aber nicht die Aussage, dass die Pipeline technisch durchgelaufen ist.

- Medien- und Wetter-Batchfortschritt wird laufend geschrieben. Die

Mastertable wird bei Medien erst nach 5.000 abgeschlossenen IDs aktualisiert, damit Volltabellen-Updates nicht zum Download-Bottleneck werden.

Empfohlene Ausfuehrungsreihenfolge

Im bestehenden Checkout arbeiten; Datenordner und Secrets nicht umbenennen. Code nur aktualisieren, wenn keine alte Pipeline mehr auf diesen Checkout zugreift. Die Bacpipe-Umgebung wird erst fuer die separate Bioakustikphase benoetigt.

```
bash update_horeka_from_git.sh main
```

```
bash bootstrap_env.sh
```

```
bash slurm_functionality_test.sh
```

Den Funktionstest mit `squeue`, `sacct` und dem neuen datierten Log pruefen. Nur bei `COMPLETED 0:0` folgt der Recovery-Lauf:

```
bash slurm_add_new_ids.sh
```

Fuer diesen Lauf keinen globalen 10- oder 30-Minuten-Override setzen. Die einzelnen Steps besitzen aufgabenspezifische Laufzeiten und Checkpoints.

Nach Abschluss des Kernlaufs bei Bedarf separat:

```
bash slurm_bioacoustics.sh
```

Abnahmekriterien

- Funktionstest: `COMPLETED`, ExitCode `0:0`.

- Run-Plan enthaelt die erwarteten Wetter-Problem-IDs.

- Step 2_4 setzt vorhandene Chunk-Checkpoints fort und erzeugt das finale

10-m-Parquet.

- Step 5_4 erkennt mindestens ein vollstaendiges Variable/Jahr-Paar; der

Verify-Job meldet keine fehlenden erwarteten Tiles.

- Nur Bioakustikphase: Step 6_0 initialisiert alle durch die Konfiguration verlangten Modelle. Mit `require_all_models_complete=true` sind derzeit alle sechs Modelle auch fuer den Verify-Gate erforderlich.

- Die finale Validierung trennt `technical_status` und `release_status`.

Eine fachliche Datenfreigabe bleibt ein manueller Teamentscheid.

13 Earth-Engine-Schlüssel für unbeaufsichtigte Sentinel-Läufe

HOREKA_GEE_SECRET_SETUP_DE.md

Der Service-Account `bio-o-ton@bio-o-ton-gee.iam.gserviceaccount.com` kann die Sentinel-Stufe ohne Browser-Login oder 2FA ausführen. Der private JSON-Schlüssel gehört **nicht** in Git - auch nicht vorübergehend -, weil Google öffentlich erkannte Schlüssel automatisch deaktivieren kann.

Auf Horeka liegen alle vier Dateien im lokalen, aber durch `.gitignore` ausgeschlossenen Ordner `.secrets/` der aktiven Git-Arbeitskopie. Der Ordner ist damit Teil der lokalen Projektstruktur, wird jedoch niemals nach GitHub übertragen. Der GEE-Schlüssel berechnet die Scores. Der Earth-Engine-Nutzer-Token startet die Legacy-Drive-Exporttasks mit dem Speicherkontingent von `biooton.kit@gmail.com`. Der separate Read-only-Drive- Token liest die fertigen TIFFs und spiegelt sie nach LSDF:

```
SECRETS_DIR="$(pwd)/.secrets"
install -d -m 700 "${SECRETS_DIR}"
install -m 600 bio-o-ton-gee-service-account.json \
  "${SECRETS_DIR}/bio-o-ton-gee-service-account.json"
install -m 600 credentials_bio-o-ton.json \
  "${SECRETS_DIR}/credentials_bio-o-ton.json"
install -m 600 token_bio-o-ton.json \
  "${SECRETS_DIR}/token_bio-o-ton.json"
install -m 600 earthengine-export-token-bio-o-ton.json \
  "${SECRETS_DIR}/earthengine-export-token-bio-o-ton.json"
```

`config.horeka.json` verwendet relative `.secrets/`-Pfade im aktiven Checkout. Die Befehle deshalb aus dessen Root ausfuehren. Optional lassen sie sich pro Job überschreiben:

```
export BIOOTON_GEE_SERVICE_ACCOUNT_KEY=/geschuetzter/pfad/service-account.json
export BIOOTON_DRIVE_CREDENTIALS=/geschuetzter/pfad/credentials.json
export BIOOTON_DRIVE_TOKEN=/geschuetzter/pfad/token.json
export BIOOTON_GEE_EXPORT_TOKEN=/geschuetzter/pfad/earthengine-export-token.json
```

Der Earth-Engine-Exporttoken wird einmalig lokal im angemeldeten Google-Konto erzeugt und danach wie oben nach LSDF kopiert:

```
python tools/create_gee_export_token.py \
  --output .secrets/earthengine-export-token-bio-o-ton.json
```

Der gespeicherte Refresh-Token erlaubt die späteren tmux-/Slurm-Läufe ohne Browser und ohne erneute 2FA, solange Google ihn nicht widerruft. Der Drive-Ordner `S2`, der Earth-Engine-Exporttoken und der separate Drive-Token muessen zum Drive-Konto `biooton.kit@gmail.com` passen. Der Service-Account wird nur fuer die Score-Abfragen verwendet und benoetigt kein eigenes Drive- Speicherkontingent. Lokal und auf Horeka liegen die Dateien jeweils im git-ignorierten Ordner `.secrets/` der Arbeitskopie. Vor einem Ersetzen oder Neu-Klonen von `scripts_horeka` muss dieser Ordner gesichert und anschliessend wieder in die neue Arbeitskopie kopiert werden.

Schritt 4.1 verarbeitet alle neuen Sentinel-IDs in aufeinanderfolgenden 500-ID-Pipeline-Batches. Die Score-Abfrage bleibt intern bei den stabil getesteten 100 IDs pro GEE-Request. Pro Batch sind höchstens 20 Drive-Exporttasks gleichzeitig aktiv; erst nach deren Abschluss beginnt der nächste Batch. Bestehende Drive- oder LSDF-TIFFs werden auch mit `--force` nicht erneut exportiert oder überschrieben.

Der reproduzierbare Score-Regressionslauf zieht standardmäßig 100 deterministisch ausgewählte Referenzzeilen ab 2026 aus der bestehenden `S2_Scores.csv`, berechnet sie mit derselben `dc_best`-Expression neu und verlangt Score-Gleichheit mit absoluter Toleranz `1e-12` sowie einen exakt gleichen `best_index`:

```
bash run_sentinel_score_validation.sh
```

Der vollständige Einzelzeilenbericht wird nach `outputs/step_4_1_sentinel2_download/score_validation_100.csv` geschrieben. Schon eine Abweichung ergibt Exitcode 1; die Referenz-Score-Datei wird ausschließlich gelesen.

14 GitHub-Workflow

GITHUB_WORKFLOW.md

Die aktuelle Arbeitskopie liegt auf Horeka unter `/home/hk-project-pai00063/jk3038/Bio-O-Ton/Data_automatisation_skripts/bio_o_ton_pipeline_git`. Der Pfad kann je nach Mount auch unter LSDF erreichbar sein. Befehle im Root des bestehenden Checkouts ausfuehren; keine vorhandenen Ordner umbenennen oder neu initialisieren.

Aktualisieren

```
bash update_horeka_from_git.sh main
```

Das Skript aktualisiert per Fast-Forward und stoppt bei lokalen Aenderungen. Ein normaler Submit fuehrt keinen automatischen Pull aus. `run_horeka.sh --update` fordert das Update ausdruecklich an. Aktiven produktiven Code nicht waehrend eines Laufs austauschen: die beim Submit gespeicherte Befehlszeile verweist weiterhin auf Dateien im Checkout, deren Inhalt sich sonst aendern kann.

Getrennte Laeufe

Nach Update und Tests zuerst den Kern starten:

```
bash slurm_add_new_ids.sh
```

Nach dessen Abschluss Bioakustik separat starten:

```
bash slurm_bioacoustics.sh
```

Alternativ dienen `bash run_horeka.sh add_new_ids` und `bash run_horeka.sh bioacoustics` als tmux-/Hybrid-Startwege.

Lokale Arbeit

Aenderungen und Tests zuerst lokal pruefen. Vor einem Commit `git diff` und die Dateiliste kontrollieren. Secrets, Modellgewichte, Umgebungen und produktive Outputs gehoeren nicht in Git; `.gitignore` schliesst sie aus. Remote: `https://github.com/FKistner85/Bioton_automatisation.git`. Ein bestehendes Repository benoetigt weder `git init` noch einen neuen Remote.

Die relativen `.secrets/`- und `bacpipe/model_checkpoints/`-Pfade beziehen sich auf den aktiven Checkout. Auf einem anderen Cluster Umgebungen neu aufbauen und die Datenpfade explizit pruefen; siehe Horeka-2-Vorbereitung.

15 Nomenclature Migration

NOMENCLATURE_MIGRATION.md

Stand: 2026-07-23

Ziel: Neue Skript- und Toolnamen beschreiben das fachliche Produkt `formation_status` statt eine personenbezogene Herkunft wie `susi`.

Neue Standardnamen

Alter Name	Neuer Standardname	Status
`scripts/Step_2_4_generate_susi_10m_products.py`	`scripts/Step_2_4_generate_10m_formation_status_products.py`	Alter Name bleibt Wrapper
`tools/sanity_check_susi_compatibility.py`	`tools/compare_formation_status_products.py`	Alter Name bleibt Wrapper
`submit_bio_o_ton_horeka.sh susi_compare`	`submit_bio_o_ton_horeka.sh formation_compare`	Alter Mode bleibt Alias

Entfernte Shell-Aliase

Diese alten Shell-Dateien wurden entfernt, damit der HoreKa-Ordner nicht mehr mehrere Einstiegspunkte fuer denselben Zweck enthaelt:

- `run_susi_sanity_check.sh`

- `slurm_compare_susi.sh`

Bitte stattdessen verwenden:

```
bash run_formation_status_comparison.sh
bash slurm_compare_formation_status.sh
```

Noch bewusst vorhandene `susi`-Vorkommen

Diese Vorkommen bleiben vorerst, weil sie Output-Kompatibilitaet oder historische Vergleichbarkeit sichern:

- Config-Keys wie `susi_10m_products` und `susi_compatible_outputs`

- Output-Ordner wie `outputs/step_2_1_susi_compatible`

- Output-Ordner wie `outputs/step_2_4_susi_10m`

- Doku/Audit-Dateien mit historischem Vergleichsbezug, z.B.

`SUSI_HOREKA_AENDERUNGSUEBERSICHT_2026-07-23.md`

Empfehlung fuer eine spaetere Breaking-Change-Version:

- Config-Keys nach `formation_status_10m_products` und

`formation_status_compatible_outputs` migrieren.

- Output-Ordner nur nach erfolgreichem Datensatzvergleich umbenennen.

- Alte Pfade fuer mindestens einen Release-Zyklus als Symlink oder

Kompatibilitaetsalias erhalten.

16 Pipeline-Steuerung (DE)

Readmes/pipeline_control/README_DE.md

Aktueller Betrieb (2026-09-23)

`add_new_ids` und `from_scratch` planen nur den Kern. `bioacoustics` plant ausschliesslich Step 6 und sein Master-/Validierungsende. Der konkrete `run_plan.json` und der Launcher bestimmen die auszufuehrenden Jobs. Bioakustik verwendet vorbereitete Metadaten, Inventar und Master statt der Step-1-Aenderungsliste. Beide Phasen teilen den Pipeline-Lock. `afterany` erlaubt Diagnose nach Fehlschlaegen; harte `afterok`-Kanten (insbesondere Step 6) blockieren abhaengige Arbeit. Solche Jobs koennen `DependencyNeverSatisfied` melden; nicht blind erneut submitten.

Slurm-Submitfehler

`AssocGrpSubmitJobsLimit` bedeutet, dass das gemeinsame Submit-Limit der Slurm-Association erreicht ist. Es koennen bereits Teiljobs eingereiht sein. Erst `squeue` und `sacct` pruefen; keine zweite Pipeline darueber starten und den Lock nicht bei aktiven Jobs entfernen. Die begrenzten Worker reduzieren die Zahl der gleichzeitig eingereihten Array-Elemente.

Zweck

Erzeugt vor dem Slurm-DAG einen deterministischen Run-Plan und verhindert parallele schreibende Gesamtlaeufe.

Tools

`tools/plan_pipeline_run.py`, `tools/pipeline_lock.py`, `tools/run_with_manifest.py`, `submit_bio_o_ton_horeka.sh`

Inputs

- Dawn-Chorus-CSV und Step-1-Domaenenfingerprints
- letzte Mastertabelle
- globale Step-States und konfigurierte Outputs

Outputs

- `outputs/step_0_control/run_plans/<run_id>/run_plan.json`
- `outputs/step_0_control/run_plans/<run_id>/*_ids.csv`
- `outputs/step_0_control/full_rebuild/current.json`
- `outputs/step_0_control/full_rebuild/<generation_id>/completed_steps/*.json`
- `outputs/step_0_control/pipeline.lock/owner.json`
- `outputs/step_0_manifests/<step>/<step_run_id>.json`

Verhalten

`add_new_ids` plant neue, geaenderte und problematische IDs. `from_scratch` plant alle aktuellen IDs und alle globalen Kernschritte. Slurm-Abhaengigkeiten verwenden je nach Zweig `afterany` oder `afterok`; blockierte Jobs muessen geprueft werden. Erfolgreiche Vollauf-Steps erhalten atomare Marker; ein neuer Submit reiht nur Steps ohne Marker erneut ein.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`in_progress`	`not resume`	plan_pipeline_run.py:469
`complete`	`Ausgabe in main`	run_with_manifest.py:117
`failed`	`Ausgabe in main`	run_with_manifest.py:117
`starting`	`Ausgabe in main`	horeka_controller.py:180

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`ValueError: Bioacoustics is disabled in this configuration.`	`not config.get('bioacoustics', {}).get('enabled', True)`	plan_pipeline_run.py:522
`FileNotFoundError: Run the core pipeline first; missing prerequisite: {path}`	`not path.is_file()`	plan_pipeline_run.py:528
`ValueError: Prepared metadata and master IDs differ; finish the core pipeline first.`	`ids != master_ids`	plan_pipeline_run.py:534
`ValueError: Require positive counts and 0 <= worker-index < worker-count`	`worker_count < 1 or task_count < 1 or (not 0 <= worker_index < worker_count)`	run_array_worker.py:13
`ValueError: A child command is required`	`not command`	run_array_worker.py:20
`ValueError: Unknown pipeline phase: {phase}`	`phase not in {'core', 'bioacoustics', 'all'}`	pipeline_phase.py:16
`ValueError: POINT_ASSIGNMENT_INCOMPLETE: {len(gaps)} recording IDs missing, duplicated or at different coordinates in {path}. Run Step 2.2 first; the existing master has not been replaced.`	`gaps`	input_consistency.py:54
`ValueError: SOURCE_POPULATION_REDUCED: source has {len(current)} IDs; {len(removed)} of {len(previous)} existing IDs would disappear. Restore the complete original source. For an intentional reduction set metadata_extraction.allow_large_source_reduction=true explicitly.`	`previous and len(removed) > max(10, len(previous) * 0.2) and (not settings.get('allow_large_source_reduction', False))`	input_consistency.py:73

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`args.mode == 'bioacoustics'`	plan_pipeline_run.py:564
`0`	`Ausgabe in main`	plan_pipeline_run.py:884
`3`	`except FileExistsError`	pipeline_lock.py:50
`0`	`Ausgabe in acquire`	pipeline_lock.py:62
`0`	`not path.exists()`	pipeline_lock.py:68
`4`	`not force and owner_run and (owner_run != run_id)`	pipeline_lock.py:76
`0`	`Ausgabe in release`	pipeline_lock.py:79
`0`	`args.command == 'status'`	pipeline_lock.py:95
`2`	`not args.run_id and (not args.force)`	pipeline_lock.py:98
`2`	`not command`	run_with_manifest.py:76
`submitted.returncode or 1`	`submitted.returncode != 0 or not submission_path.is_file()`	horeka_controller.py:233
`0`	`args.local_test`	horeka_controller.py:256
`130`	`except KeyboardInterrupt`	horeka_controller.py:313
`128 + interrupted`	`interrupted`	run_array_worker.py:54
`1`	`failed`	run_array_worker.py:57
`0`	`Ausgabe in run_worker`	run_array_worker.py:58
`run_stage_all(root, Path(sys.executable), variants, args.stage, force=args.force, ids_file=args.ids_file, max_workers=1)`	`config.get('lrt_variants', {}).get('input_dir')`	run_spatial_stage.py:22
`0`	`Ausgabe in main`	finalize_master.py:23

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

17 Pipeline Control (EN)

Readmes/pipeline_control/README_EN.md

Current Operation (2026-09-23)

`add_new_ids` and `from_scratch` plan the core only. `bioacoustics` plans Step 6 and its master/validation ending. The actual run plan and launcher determine submissions. Bioacoustics consumes prepared metadata, inventory and master rather than Step-1 deltas. Both phases share the lock. `afterany` permits diagnostics after failure; hard `afterok` edges (especially Step 6) block dependent work and can leave `DependencyNeverSatisfied` jobs. Inspect before resubmission.

Slurm submission failures

`AssocGrpSubmitJobsLimit` means the shared Slurm association submission limit has been reached. Some jobs may already be queued. Inspect `squeue` and `sacct` first; do not submit a second pipeline or remove the lock while jobs remain active. Bounded workers reduce the number of queued array elements.

Purpose

Builds a deterministic run plan before submitting the Slurm DAG and prevents concurrent workflows from writing the same products.

Tools

`tools/plan_pipeline_run.py`, `tools/pipeline_lock.py`, `tools/run_with_manifest.py`, `submit_bio_o_ton_horeka.sh`

Inputs

- Dawn Chorus CSV and Step 1 domain fingerprints
- previous master table
- global step states and configured outputs

Outputs

- `outputs/step_0_control/run_plans/<run_id>/run_plan.json`
- `outputs/step_0_control/run_plans/<run_id>/*_ids.csv`
- `outputs/step_0_control/full_rebuild/current.json`
- `outputs/step_0_control/full_rebuild/<generation_id>/completed_steps/*.json`
- `outputs/step_0_control/pipeline.lock/owner.json`
- `outputs/step_0_manifests/<step>/<step_run_id>.json`

Behaviour

`add_new_ids` plans new, changed and previously problematic IDs. `from_scratch` plans all current IDs and all global core steps. Slurm uses both `afterany` and `afterok`; inspect jobs blocked by failed dependencies. Successful full-run steps receive atomic markers; a new submission only queues steps whose markers are missing.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`in_progress`	`not resume`	plan_pipeline_run.py:469
`complete`	`Emitted by main`	run_with_manifest.py:117
`failed`	`Emitted by main`	run_with_manifest.py:117
`starting`	`Emitted by main`	horeka_controller.py:180

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`ValueError: Bioacoustics is disabled in this configuration.`	`not config.get('bioacoustics', {}).get('enabled', True)`	plan_pipeline_run.py:522
`FileNotFoundError: Run the core pipeline first; missing prerequisite: {path}`	`not path.is_file()`	plan_pipeline_run.py:528
`ValueError: Prepared metadata and master IDs differ; finish the core pipeline first.`	`ids != master_ids`	plan_pipeline_run.py:534
`ValueError: Require positive counts and 0 <= worker-index < worker-count`	`worker_count < 1 or task_count < 1 or (not 0 <= worker_index < worker_count)`	run_array_worker.py:13
`ValueError: A child command is required`	`not command`	run_array_worker.py:20
`ValueError: Unknown pipeline phase: {phase}`	`phase not in {'core', 'bioacoustics', 'all'}`	pipeline_phase.py:16
`ValueError: POINT_ASSIGNMENT_INCOMPLETE: {len(gaps)} recording IDs missing, duplicated or at different coordinates in {path}. Run Step 2.2 first; the existing master has not been replaced.`	`gaps`	input_consistency.py:54
`ValueError: SOURCE_POPULATION_REDUCED: source has {len(current)} IDs; {len(removed)} of {len(previous)} existing IDs would disappear. Restore the complete original source. For an intentional reduction set metadata_extraction.allow_large_source_reduction=true explicitly.`	`previous and len(removed) > max(10, len(previous) * 0.2) and (not settings.get('allow_large_source_reduction', False))`	input_consistency.py:73

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`args.mode == 'bioacoustics'`	plan_pipeline_run.py:564
`0`	`Emitted by main`	plan_pipeline_run.py:884
`3`	`except FileExistsError`	pipeline_lock.py:50
`0`	`Emitted by acquire`	pipeline_lock.py:62
`0`	`not path.exists()`	pipeline_lock.py:68
`4`	`not force and owner_run and (owner_run != run_id)`	pipeline_lock.py:76
`0`	`Emitted by release`	pipeline_lock.py:79
`0`	`args.command == 'status'`	pipeline_lock.py:95
`2`	`not args.run_id and (not args.force)`	pipeline_lock.py:98
`2`	`not command`	run_with_manifest.py:76
`submitted.returncode or 1`	`submitted.returncode != 0 or not submission_path.is_file()`	horeka_controller.py:233
`0`	`args.local_test`	horeka_controller.py:256
`130`	`except KeyboardInterrupt`	horeka_controller.py:313
`128 + interrupted`	`interrupted`	run_array_worker.py:54
`1`	`failed`	run_array_worker.py:57
`0`	`Emitted by run_worker`	run_array_worker.py:58
`run_stage_all(root, Path(sys.executable), variants, args.stage, force=args.force, ids_file=args.ids_file, max_workers=1)`	`config.get('lrt_variants', {}).get('input_dir')`	run_spatial_stage.py:22
`0`	`Emitted by main`	finalize_master.py:23

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

18 Step 1 Metadata Extraction (DE)

Readmes/step_1_metadata/README_DE.md

Zweck

Normalisiert Dawn-Chorus-IDs, Koordinaten und Zeitfelder.

Script

`scripts/Step_1_metadata_extraction.py`

Eingaben

- `PointData/dawn-chorus-soundscape.csv`

Outputs

- `outputs/step_1_metadata/dawnchorus_metadata_clean.csv`
- `outputs/step_1_metadata/dawnchorus_metadata_log.csv`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_1_metadata`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `dawn_chorus_csv`, `status_dir`, `metadata_extraction`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Step 1 verarbeitet ausschliesslich Zeilen, bei denen die konfigurierte Länderspalte dem konfigurierten Zielland entspricht (standardmaessig `country = Germany`). Fehlt die Länderspalte, bricht der Step ab. Wechselt ein zuvor verarbeitetes ID in ein anderes Land, wird es aus den Step-1-Outputs entfernt und kann bei der folgenden Mastertable-Aktualisierung nicht mehr erscheinen.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_1_metadata_extraction.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

Neue IDs werden inkrementell ergaenzt; from_scratch nutzt --force.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

German recording-time policy (2026-09-16)

`localtimes` is authoritative when nonempty: preserve its local clock and apply Europe/Berlin DST rules. Only missing local values use `datetime` as an instant and convert it to Europe/Berlin. A datetime without an offset is assumed UTC and explicitly flagged; no such rows occur in the audited September input. Malformed/nonexistent local clocks and unresolved autumn folds remain invalid; there is no silent fallback or one-hour shift. A valid local offset, otherwise a matching UTC reference, resolves the autumn fold. Conflicting UTC references and non-German local offsets are logged, while the local clock remains authoritative. `coordinate_check` is only a WGS84 / broad Germany rectangle plausibility check, not a country-border or timezone-polygon test.

The clean intermediate `datetime` retains the correct +01:00/+02:00 offset. The master `datetime_local` stores only the German clock, without an offset. Planner and extraction share fingerprints. A policy change invalidates metadata, weather and Sentinel planning. The first migration therefore revisits existing IDs, not only newly added IDs. A scoped extraction never acknowledges untouched IDs.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Code / Meldung	Bedeutung / Ausloeser	Quelle
`datetime_timezone_missing_assumed_utc`	datetime hat keinen Offset und wird als UTC interpretiert; Warnung bleibt sichtbar.	recording_time.py:56
`localtimes_unparseable`	Nichtleere localtimes ist unlesbar; kein stiller UTC-Fallback.	recording_time.py:64
`nonexistent_localtime`	Lokale Uhrzeit faellt in die uebersprungene Fruehlingsstunde; bleibt ungueltig.	recording_time.py:74
`ambiguous_localtime_unresolved`	Doppelte Herbststunde ohne eindeutigen Offset/UTC-Abgleich; Zeitpunkt bleibt ungueltig.	recording_time.py:91
`localtimes_offset_mismatch`	Mitgelieferter lokaler Offset passt nicht zu Europe/Berlin; lokale Uhrzeit bleibt massgeblich.	recording_time.py:93
`utc_reference_unavailable`	Keine nutzbare UTC-Referenz fuer den Gegencheck.	recording_time.py:95
`datetime_missing_or_unparseable`	Ohne lokale Zeit ist auch datetime nicht nutzbar.	recording_time.py:99
`utc_local_conflict`	UTC-Referenz widerspricht der lokalen Uhr; localtimes bleibt massgeblich.	recording_time.py:110

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`within_broad_germany_bounds`	`Ausgabe in build_outputs`	Step_1_metadata_extraction.py:115
`outside_broad_germany_bounds`	`Ausgabe in build_outputs`	Step_1_metadata_extraction.py:116
`invalid_wgs84_coordinates`	`Ausgabe in build_outputs`	Step_1_metadata_extraction.py:117
`invalid`	`Ausgabe in resolve_recording_time`	recording_time.py:111
`warning`	`Ausgabe in resolve_recording_time`	recording_time.py:111
`validated`	`Ausgabe in resolve_recording_time`	recording_time.py:111

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_1_metadata_extraction.py:52
`ValueError: Missing required Dawn Chorus column: id`	`'id' not in source.columns`	Step_1_metadata_extraction.py:155
`ValueError: Missing required Dawn Chorus country column: {country_column}`	`country_column not in source.columns`	Step_1_metadata_extraction.py:157
`FileNotFoundError: IDs file not found: {path}`	`not path.is_file()`	Step_1_metadata_extraction.py:256
`KeyError: No dawn_chorus_id/id column in {path}`	`Ausgabe in read_ids_file`	Step_1_metadata_extraction.py:265
`ValueError: This pipeline requires country Germany and timezone Europe/Berlin`	`timezone != DEFAULT_TIMEZONE or country_value != 'Germany'`	Step_1_metadata_extraction.py:332
`FileNotFoundError: Dawn Chorus CSV not found: {input_csv}`	`not input_csv.is_file()`	Step_1_metadata_extraction.py:344
`ValueError: Germany recordings require timezone Europe/Berlin`	`timezone != GERMAN_TIMEZONE`	recording_time.py:48
`ValueError: POINT_ASSIGNMENT_INCOMPLETE: {len(gaps)} recording IDs missing, duplicated or at different coordinates in {path}. Run Step 2.2 first; the existing master has not been replaced.`	`gaps`	input_consistency.py:54
`ValueError: SOURCE_POPULATION_REDUCED: source has {len(current)} IDs; {len(removed)} of {len(previous)} existing IDs would disappear. Restore the complete original source. For an intentional reduction set metadata_extraction.allow_large_source_reduction=true explicitly.`	`previous and len(removed) > max(10, len(previous) * 0.2) and (not settings.get('allow_large_source_reduction', False))`	input_consistency.py:73

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`1`	`except Exception`	Step_1_metadata_extraction.py:445
`0`	`Ausgabe in main`	Step_1_metadata_extraction.py:483

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

19 Step 1 Metadata Extraction (EN)

Readmes/step_1_metadata/README_EN.md

Purpose

Normalises Dawn Chorus IDs, coordinates and time fields.

Script

`scripts/Step_1_metadata_extraction.py`

Inputs

- `PointData/dawn-chorus-soundscape.csv`

Outputs

- `outputs/step_1_metadata/dawnchorus_metadata_clean.csv`
- `outputs/step_1_metadata/dawnchorus_metadata_log.csv`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_1_metadata`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `dawn_chorus_csv`, `status_dir`, `metadata_extraction`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Step 1 processes only rows whose configured country column matches the configured target country (by default, `country = Germany`). The step fails if the country column is absent. If a previously processed ID changes to another country, it is removed from the Step 1 outputs and therefore cannot appear in the next master-table update.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_1_metadata_extraction.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

New IDs are added incrementally; `from_scratch` uses `--force`.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

German recording-time policy (2026-09-16)

`localtimes` is authoritative when nonempty: preserve its local clock and apply Europe/Berlin DST rules. Only missing local values use `datetime` as an instant and convert it to Europe/Berlin. A datetime without an offset is assumed UTC and explicitly flagged; no such rows occur in the audited September input. Malformed/nonexistent local clocks and unresolved autumn folds remain invalid; there is no silent fallback or one-hour shift. A valid local offset, otherwise a matching UTC reference, resolves the autumn fold. Conflicting UTC references and non-German local offsets are logged, while the local clock remains authoritative. `coordinate_check` is only a WGS84 / broad Germany rectangle plausibility check, not a country-border or timezone-polygon test.

The clean intermediate `datetime` retains the correct +01:00/+02:00 offset. The master `datetime_local` stores only the German clock, without an offset. Planner and extraction share fingerprints. A policy change invalidates metadata, weather and Sentinel planning. The first migration therefore revisits existing IDs, not only newly added IDs. A scoped extraction never acknowledges untouched IDs.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

Code / Message	Meaning / Trigger	Source
`datetime_timezone_missing_assumed_utc`	datetime has no offset and is interpreted as UTC, with a warning.	recording_time.py:56
`localtimes_unparseable`	Nonempty localtimes cannot be parsed; no silent UTC fallback.	recording_time.py:64
`nonexistent_localtime`	Local clock falls in the skipped spring hour and remains invalid.	recording_time.py:74
`ambiguous_localtime_unresolved`	Repeated autumn hour cannot be resolved by offset/UTC; timestamp stays invalid.	recording_time.py:91
`localtimes_offset_mismatch`	Supplied local offset disagrees with Europe/Berlin; local clock remains authoritative.	recording_time.py:93
`utc_reference_unavailable`	No usable UTC reference for cross-checking.	recording_time.py:95
`datetime_missing_or_unparseable`	With no local clock supplied, datetime is also unusable.	recording_time.py:99
`utc_local_conflict`	UTC reference conflicts with local clock; localtimes remains authoritative.	recording_time.py:110

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`within_broad_germany_bounds`	`Emitted by build_outputs`	Step_1_metadata_extraction.py:115
`outside_broad_germany_bounds`	`Emitted by build_outputs`	Step_1_metadata_extraction.py:116
`invalid_wgs84_coordinates`	`Emitted by build_outputs`	Step_1_metadata_extraction.py:117
`invalid`	`Emitted by resolve_recording_time`	recording_time.py:111
`warning`	`Emitted by resolve_recording_time`	recording_time.py:111
`validated`	`Emitted by resolve_recording_time`	recording_time.py:111

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_1_metadata_extraction.py:52
`ValueError: Missing required Dawn Chorus column: id`	`'id' not in source.columns`	Step_1_metadata_extraction.py:155
`ValueError: Missing required Dawn Chorus country column: {country_column}`	`country_column not in source.columns`	Step_1_metadata_extraction.py:157
`FileNotFoundError: IDs file not found: {path}`	`not path.is_file()`	Step_1_metadata_extraction.py:256
`KeyError: No dawn_chorus_id/id column in {path}`	Emitted by `read_ids_file`	Step_1_metadata_extraction.py:265
`ValueError: This pipeline requires country Germany and timezone Europe/Berlin`	`timezone != DEFAULT_TIMEZONE or country_value != 'Germany'`	Step_1_metadata_extraction.py:332
`FileNotFoundError: Dawn Chorus CSV not found: {input_csv}`	`not input_csv.is_file()`	Step_1_metadata_extraction.py:344
`ValueError: Germany recordings require timezone Europe/Berlin`	`timezone != GERMAN_TIMEZONE`	recording_time.py:48
`ValueError: POINT_ASSIGNMENT_INCOMPLETE: {len(gaps)} recording IDs missing, duplicated or at different coordinates in {path}. Run Step 2.2 first; the existing master has not been replaced.`	`gaps`	input_consistency.py:54
`ValueError: SOURCE_POPULATION_REDUCED: source has {len(current)} IDs; {len(removed)} of {len(previous)} existing IDs would disappear. Restore the complete original source. For an intentional reduction set metadata_extraction.allow_large_source_reduction=true explicitly.`	`previous and len(removed) > max(10, len(previous) * 0.2) and (not settings.get('allow_large_source_reduction', False))`	input_consistency.py:73

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`1`	`except Exception`	Step_1_metadata_extraction.py:445
`0`	Emitted by `main`	Step_1_metadata_extraction.py:483

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

20 Step 2_0 LRT Cleaning (DE)

Readmes/step_2_0_lrt_cleaning/README_DE.md

Zweck

Bereinigt LRT-Polygone, normalisiert Codes, Status und Formation.

Script

`scripts/Step_2_0_clean_lrts.py`

Eingaben

- `Biodiversity_data/Bundeslander/All_Bundeslander/All_Bundeslander_<suffix>.gpkg`

Outputs

- `outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg`
- `outputs/step_2_variants/<suffix>/step_2_0/state.json`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_2_0_lrt_cleaning`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `lrt_cleaning`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_2_0_clean_lrts.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

State/Fingerprints entscheiden, ob ein Rebuild noetig ist.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`conservation_status`	`Ausgabe in module`	Step_2_0_clean_lrts.py:34
`Conservation_status`	`Ausgabe in module`	Step_2_0_clean_lrts.py:34
`CONSERVATION_STATUS`	`Ausgabe in module`	Step_2_0_clean_lrts.py:34
`conservationstatus`	`Ausgabe in module`	Step_2_0_clean_lrts.py:34
`ConservationStatus`	`Ausgabe in module`	Step_2_0_clean_lrts.py:34

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_2_0_clean_lrts.py:52
`KeyError: Missing 'lrt_cleaning' section in config.json.`	`not isinstance(section, dict)`	Step_2_0_clean_lrts.py:59
`ValueError: 'lrt_cleaning.source_gpkgs' must be a non-empty list.`	`not isinstance(section['source_gpkgs'], list) or not section['source_gpkgs']`	Step_2_0_clean_lrts.py:69
`FileNotFoundError: Input file not found: {path}`	`not path.is_file()`	Step_2_0_clean_lrts.py:76
`ValueError: Required column '{canonical}' not found in {source}. Available columns: {list(gdf.columns)}`	`found is None`	Step_2_0_clean_lrts.py:133
`ValueError: No layers found in {gpkg_path}`	`not layers`	Step_2_0_clean_lrts.py:163
`ValueError: Layer has no CRS: {gpkg_path.name}:{layer_name}`	`layer.crs is None`	Step_2_0_clean_lrts.py:186
`ValueError: No usable LRT layers found in {gpkg_path}`	`not prepared`	Step_2_0_clean_lrts.py:203
`RuntimeError: No formation group was processed successfully.`	`not results`	Step_2_0_clean_lrts.py:369
`RuntimeError: Atomic GPKG validation failed: expected {len(output)} rows, found {written_rows}.`	`written_rows != len(output)`	Step_2_0_clean_lrts.py:536

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`should_skip(output_gpkg, state_file, expected_state, args.force)`	Step_2_0_clean_lrts.py:469
`0`	`Ausgabe in main`	Step_2_0_clean_lrts.py:549
`1`	`except Exception`	Step_2_0_clean_lrts.py:553

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

21 Step 2_0 LRT Cleaning (EN)

Readmes/step_2_0_lrt_cleaning/README_EN.md

Purpose

Cleans LRT polygons and normalises code, status and formation fields.

Script

`scripts/Step_2_0_clean_lrts.py`

Inputs

- `Biodiversity_data/Bundeslander/All_Bundeslander/All_Bundeslander_<suffix>.gpkg`

Outputs

- `outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg`
- `outputs/step_2_variants/<suffix>/step_2_0/state.json`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_2_0_lrt_cleaning`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `lrt_cleaning`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_2_0_clean_lrts.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

State/Fingerprints entscheiden, ob ein Rebuild noetig ist.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`conservation_status`	`Emitted by module`	Step_2_0_clean_lrts.py:34
`Conservation_status`	`Emitted by module`	Step_2_0_clean_lrts.py:34
`CONSERVATION_STATUS`	`Emitted by module`	Step_2_0_clean_lrts.py:34
`conservationstatus`	`Emitted by module`	Step_2_0_clean_lrts.py:34
`ConservationStatus`	`Emitted by module`	Step_2_0_clean_lrts.py:34

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_2_0_clean_lrts.py:52
`KeyError: Missing 'lrt_cleaning' section in config.json.`	`not isinstance(section, dict)`	Step_2_0_clean_lrts.py:59
`ValueError: 'lrt_cleaning.source_gpkgs' must be a non-empty list.`	`not isinstance(section['source_gpkgs'], list) or not section['source_gpkgs']`	Step_2_0_clean_lrts.py:69
`FileNotFoundError: Input file not found: {path}`	`not path.is_file()`	Step_2_0_clean_lrts.py:76
`ValueError: Required column '{canonical}' not found in {source}. Available columns: {list(gdf.columns)}`	`found is None`	Step_2_0_clean_lrts.py:133
`ValueError: No layers found in {gpkg_path}`	`not layers`	Step_2_0_clean_lrts.py:163
`ValueError: Layer has no CRS: {gpkg_path.name}:{layer_name}`	`layer.crs is None`	Step_2_0_clean_lrts.py:186
`ValueError: No usable LRT layers found in {gpkg_path}`	`not prepared`	Step_2_0_clean_lrts.py:203
`RuntimeError: No formation group was processed successfully.`	`not results`	Step_2_0_clean_lrts.py:369
`RuntimeError: Atomic GPKG validation failed: expected {len(output)} rows, found {written_rows}.`	`written_rows != len(output)`	Step_2_0_clean_lrts.py:536

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`should_skip(output_gpkg, state_file, expected_state, args.force)`	Step_2_0_clean_lrts.py:469
`0`	`Emitted by main`	Step_2_0_clean_lrts.py:549
`1`	`except Exception`	Step_2_0_clean_lrts.py:553

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

22 Step 2_1 100m Formation Status (DE)

Readmes/step_2_1_100m_formation_status/README_DE.md

Zweck

Verschneidet LRTs mit dem 100m-Grid und erzeugt Majority- und Formation-Status-Produkte.

Script

`scripts/Step_2_1_merge_lrts_and_grid.py`

Eingaben

- `InspireGrid/Vector_Data/grid.gpkg`
- `outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg`

Outputs

- `outputs/step_2_variants/<suffix>/step_2_1/*`
- `outputs/step_2_variants/<suffix>/step_2_1_susi_compatible/*`

Die Susi-kompatible Matrix speichert alle Formation- und LRT-Anteile als ganzzahlige Prozentwerte mit Faktor 100 (`10000 = 100 Prozent`). Formation- Totalsummen enthalten A/B/C/K. Der Majority-Status wird nur aus A/B/C bestimmt; `majority_disputed` bedeutet `majority_delta <= 200` und damit maximal zwei Prozentpunkte Abstand.

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_2_1_100m_formation`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `lrt_grid_merge`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_2_1_merge_lrts_and_grid.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

Chunk-Checkpoints und State-Datei ermoeglichen Wiederaufnahme.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`majority_formation_status`	`Ausgabe in build_summary`	Step_2_1_merge_lrts_and_grid.py:584

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`RuntimeError: Temporary GeoPackage is empty: {temporary}`	`not temporary.is_file() or temporary.stat().st_size == 0`	Step_2_1_merge_lrts_and_grid.py:67
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_2_1_merge_lrts_and_grid.py:73
`KeyError: Missing 'lrt_grid_merge' section in config.json.`	`not isinstance(section, dict)`	Step_2_1_merge_lrts_and_grid.py:80
`FileNotFoundError: Input file not found: {path}`	`not path.is_file()`	Step_2_1_merge_lrts_and_grid.py:95
`ValueError: Grid ID column '{grid_id_column}' not found. Available columns: {list(grid.columns)}`	`grid_id_column not in grid.columns`	Step_2_1_merge_lrts_and_grid.py:161
`ValueError: Grid and LRT data must both have a CRS.`	`grid.crs is None or lrt.crs is None`	Step_2_1_merge_lrts_and_grid.py:182
`RuntimeError: Step 2_1 worker was not initialised.`	`_WORKER_GRID is None or _WORKER_LRT is None or _WORKER_GRID_ID is None`	Step_2_1_merge_lrts_and_grid.py:211
`RuntimeError: No intersections found between grid and LRT.`	`not parts`	Step_2_1_merge_lrts_and_grid.py:446
`ValueError: Could not derive 100 m coordinates from grid_id. Examples: {bad}`	`ids.isna().any(axis=None)`	Step_2_1_merge_lrts_and_grid.py:772

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`should_skip(output_paths, state_file, expected_state, args.force)`	Step_2_1_merge_lrts_and_grid.py:1196
`0`	`Ausgabe in main`	Step_2_1_merge_lrts_and_grid.py:1271
`1`	`except Exception`	Step_2_1_merge_lrts_and_grid.py:1275

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

23 Step 2_1 100m Formation Status (EN)

Readmes/step_2_1_100m_formation_status/README_EN.md

Purpose

Overlays LRTs with the 100m grid and creates majority and formation-status products.

Script

`scripts/Step_2_1_merge_lrts_and_grid.py`

Inputs

- `InspireGrid/Vector_Data/grid.gpkg`
- `outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg`

Outputs

- `outputs/step_2_variants/<suffix>/step_2_1/*`
- `outputs/step_2_variants/<suffix>/step_2_1_susi_compatible/*`

The Susi-compatible matrix stores all formation and LRT shares as integer percentages with a factor of 100 (`10000 = 100 percent`). Formation totals include A/B/C/K. Majority status is selected from A/B/C only; `majority_disputed` means `majority_delta <= 200`, i.e. no more than two percentage points apart.

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_2_1_100m_formation`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `lrt_grid_merge`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_2_1_merge_lrts_and_grid.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

Chunk checkpoints and the state file allow a run to resume.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`majority_formation_status`	`Emitted by build_summary`	Step_2_1_merge_lrts_and_grid.py:584

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`RuntimeError: Temporary GeoPackage is empty: {temporary}`	`not temporary.is_file() or temporary.stat().st_size == 0`	Step_2_1_merge_lrts_and_grid.py:67
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_2_1_merge_lrts_and_grid.py:73
`KeyError: Missing 'lrt_grid_merge' section in config.json.`	`not isinstance(section, dict)`	Step_2_1_merge_lrts_and_grid.py:80
`FileNotFoundError: Input file not found: {path}`	`not path.is_file()`	Step_2_1_merge_lrts_and_grid.py:95
`ValueError: Grid ID column '{grid_id_column}' not found. Available columns: {list(grid.columns)}`	`grid_id_column not in grid.columns`	Step_2_1_merge_lrts_and_grid.py:161
`ValueError: Grid and LRT data must both have a CRS.`	`grid.crs is None or lrt.crs is None`	Step_2_1_merge_lrts_and_grid.py:182
`RuntimeError: Step 2_1 worker was not initialised.`	`_WORKER_GRID is None or _WORKER_LRT is None or _WORKER_GRID_ID is None`	Step_2_1_merge_lrts_and_grid.py:211
`RuntimeError: No intersections found between grid and LRT.`	`not parts`	Step_2_1_merge_lrts_and_grid.py:446
`ValueError: Could not derive 100 m coordinates from grid_id. Examples: {bad}`	`ids.isna().any(axis=None)`	Step_2_1_merge_lrts_and_grid.py:772

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`should_skip(output_paths, state_file, expected_state, args.force)`	Step_2_1_merge_lrts_and_grid.py:1196
`0`	`Emitted by main`	Step_2_1_merge_lrts_and_grid.py:1271
`1`	`except Exception`	Step_2_1_merge_lrts_and_grid.py:1275

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

24 Step 2_2 Point Assignment (DE)

Readmes/step_2_2_point_assignment/README_DE.md

Zweck

Ordnet jeden Dawn-Chorus-Punkt der INSPIRE-100m-Gridzelle und LRT-Polygonen zu. Die Grid-ID wird auch ohne Majority Formation ausgegeben; eine vorhandene Majority Formation bleibt ein separates Attribut.

Script

`scripts/Step_2_2_assign_points_to_lrt_grid.py`

Eingaben

- `outputs/step_1_metadata/dawnchorus_metadata_clean.csv`
- `outputs/step_2_variants/<suffix>/step_2_1/LRT_Grid_Majority_<suffix>.csv`

Outputs

- `outputs/step_2_variants/<suffix>/step_2_2/DawnChorus_LRT_Grid_Assignment_<suffix>.csv`
- `outputs/step_2_variants/<suffix>/step_2_2/DawnChorus_LRT_Polygon_Matches_<suffix>.csv`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_2_2_point_assignment`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `point_lrt_assignment`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_2_2_assign_points_to_lrt_grid.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

Bei unveraenderten Spatial Inputs werden nur neue IDs verarbeitet.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`processed`	Ausgabe in build_outputs	Step_2_2_assign_points_to_lrt_grid.py:505
`missing_coordinates`	Ausgabe in build_outputs	Step_2_2_assign_points_to_lrt_grid.py:506
`outside_majority_grid`	Ausgabe in build_outputs	Step_2_2_assign_points_to_lrt_grid.py:510
`inside_majority_grid_outside_lrt`	Ausgabe in build_outputs	Step_2_2_assign_points_to_lrt_grid.py:515
`inside_lrt_polygon`	Ausgabe in build_outputs	Step_2_2_assign_points_to_lrt_grid.py:520

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_2_2_assign_points_to_lrt_grid.py:56
`KeyError: Missing 'point_lrt_assignment' section in config.json.`	`not isinstance(section, dict)`	Step_2_2_assign_points_to_lrt_grid.py:63
`FileNotFoundError: Input file not found: {path}`	`not path.is_file()`	Step_2_2_assign_points_to_lrt_grid.py:91
`ValueError: Grid ID column '{grid_id_column}' not found. Available columns: {list(grid.columns)}`	`grid_id_column not in grid.columns`	Step_2_2_assign_points_to_lrt_grid.py:189
`ValueError: The INSPIRE grid contains no cells.`	`grid.empty`	Step_2_2_assign_points_to_lrt_grid.py:203
`ValueError: The cleaned LRT layer has no CRS.`	`lrt.crs is None`	Step_2_2_assign_points_to_lrt_grid.py:239
`KeyError: No dawn_chorus_id/id column in {path}`	`Ausgabe in read_ids_file`	Step_2_2_assign_points_to_lrt_grid.py:545
`ValueError: POINT_ASSIGNMENT_INCOMPLETE: {len(gaps)} recording IDs missing, duplicated or at different coordinates in {path}. Run Step 2.2 first; the existing master has not been replaced.`	`gaps`	input_consistency.py:54
`ValueError: SOURCE_POPULATION_REDUCED: source has {len(current)} IDs; {len(removed)} of {len(previous)} existing IDs would disappear. Restore the complete original source. For an intentional reduction set metadata_extraction.allow_large_source_reduction=true explicitly.`	`previous and len(removed) > max(10, len(previous) * 0.2) and (not settings.get('allow_large_source_reduction', False))`	input_consistency.py:73

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`not target_ids`	Step_2_2_assign_points_to_lrt_grid.py:694
`0`	`Ausgabe in main`	Step_2_2_assign_points_to_lrt_grid.py:786
`1`	`except Exception`	Step_2_2_assign_points_to_lrt_grid.py:790

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

25 Step 2_2 Point Assignment (EN)

Readmes/step_2_2_point_assignment/README_EN.md

Purpose

Assigns every Dawn Chorus point to its INSPIRE 100 m grid cell and LRT polygons. The grid ID is emitted even without a majority formation; majority formation remains a separate attribute.

Script

`scripts/Step_2_2_assign_points_to_lrt_grid.py`

Inputs

- `outputs/step_1_metadata/dawnchorus_metadata_clean.csv`
- `outputs/step_2_variants/<suffix>/step_2_1/LRT_Grid_Majority_<suffix>.csv`

Outputs

- `outputs/step_2_variants/<suffix>/step_2_2/DawnChorus_LRT_Grid_Assignment_<suffix>.csv`
- `outputs/step_2_variants/<suffix>/step_2_2/DawnChorus_LRT_Polygon_Matches_<suffix>.csv`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_2_2_point_assignment`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `point_lrt_assignment`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_2_2_assign_points_to_lrt_grid.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

When spatial inputs are unchanged, only affected/new IDs are processed.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`processed`	`Emitted by build_outputs`	Step_2_2_assign_points_to_lrt_grid.py:505
`missing_coordinates`	`Emitted by build_outputs`	Step_2_2_assign_points_to_lrt_grid.py:506
`outside_majority_grid`	`Emitted by build_outputs`	Step_2_2_assign_points_to_lrt_grid.py:510
`inside_majority_grid_outside_lrt`	`Emitted by build_outputs`	Step_2_2_assign_points_to_lrt_grid.py:515
`inside_lrt_polygon`	`Emitted by build_outputs`	Step_2_2_assign_points_to_lrt_grid.py:520

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_2_2_assign_points_to_lrt_grid.py:56
`KeyError: Missing 'point_lrt_assignment' section in config.json.`	`not isinstance(section, dict)`	Step_2_2_assign_points_to_lrt_grid.py:63
`FileNotFoundError: Input file not found: {path}`	`not path.is_file()`	Step_2_2_assign_points_to_lrt_grid.py:91
`ValueError: Grid ID column '{grid_id_column}' not found. Available columns: {list(grid.columns)}`	`grid_id_column not in grid.columns`	Step_2_2_assign_points_to_lrt_grid.py:189
`ValueError: The INSPIRE grid contains no cells.`	`grid.empty`	Step_2_2_assign_points_to_lrt_grid.py:203
`ValueError: The cleaned LRT layer has no CRS.`	`lrt.crs is None`	Step_2_2_assign_points_to_lrt_grid.py:239
`KeyError: No dawn_chorus_id/id column in {path}`	`Emitted by read_ids_file`	Step_2_2_assign_points_to_lrt_grid.py:545
`ValueError: POINT_ASSIGNMENT_INCOMPLETE: {len(gaps)} recording IDs missing, duplicated or at different coordinates in {path}. Run Step 2.2 first; the existing master has not been replaced.`	`gaps`	input_consistency.py:54
`ValueError: SOURCE_POPULATION_REDUCED: source has {len(current)} IDs; {len(removed)} of {len(previous)} existing IDs would disappear. Restore the complete original source. For an intentional reduction set metadata_extraction.allow_large_source_reduction=true explicitly.`	`previous and len(removed) > max(10, len(previous) * 0.2) and (not settings.get('allow_large_source_reduction', False))`	input_consistency.py:73

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`not target_ids`	Step_2_2_assign_points_to_lrt_grid.py:694
`0`	`Emitted by main`	Step_2_2_assign_points_to_lrt_grid.py:786
`1`	`except Exception`	Step_2_2_assign_points_to_lrt_grid.py:790

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

26 Step 2_3 Grid Aggregation (DE)

Readmes/step_2_3_grid_aggregation/README_DE.md

Zweck

Aggregiert 100m-Gridprodukte auf groessere Raster.

Script

`scripts/Step_2_3_generate_remaining_grid_products.py`

Eingaben

- `outputs/step_2_variants/<suffix>/step_2_1/majority_formation_grid_<suffix>.parquet`

Outputs

- `outputs/step_2_variants/<suffix>/step_2_3/*.csv`
- `outputs/step_2_variants/<suffix>/step_2_3/state.json`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_2_3_grid_aggregation`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `lrt_grid_aggregation`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_2_3_generate_remaining_grid_products.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

State/Fingerprint-basierter Skip bei unveraenderten Inputs.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`KeyError: Missing 'lrt_grid_aggregation' section.`	`not isinstance(config.get('lrt_grid_aggregation'), dict)`	Step_2_3_generate_remaining_grid_products.py:32
`KeyError: Missing 'lrt_grid_merge' section.`	`not isinstance(config.get('lrt_grid_merge'), dict)`	Step_2_3_generate_remaining_grid_products.py:34
`RuntimeError: Worker source was not initialised.`	`_BASE is None`	Step_2_3_generate_remaining_grid_products.py:141

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`previous == expected_state`	Step_2_3_generate_remaining_grid_products.py:402
`0`	`Ausgabe in main`	Step_2_3_generate_remaining_grid_products.py:436
`1`	`except Exception`	Step_2_3_generate_remaining_grid_products.py:440

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

27 Step 2_3 Grid Aggregation (EN)

Readmes/step_2_3_grid_aggregation/README_EN.md

Purpose

Aggregates 100m grid products to coarser grid resolutions.

Script

`scripts/Step_2_3_generate_remaining_grid_products.py`

Inputs

- `outputs/step_2_variants/<suffix>/step_2_1/majority_formation_grid_<suffix>.parquet`

Outputs

- `outputs/step_2_variants/<suffix>/step_2_3/*.csv`
- `outputs/step_2_variants/<suffix>/step_2_3/state.json`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_2_3_grid_aggregation`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `lrt_grid_aggregation`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_2_3_generate_remaining_grid_products.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

State and input fingerprints allow unchanged products to be skipped.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

No fixed values in this category in the associated source.

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`KeyError: Missing 'lrt_grid_aggregation' section.`	`not isinstance(config.get('lrt_grid_aggregation'), dict)`	Step_2_3_generate_remaining_grid_products.py:32
`KeyError: Missing 'lrt_grid_merge' section.`	`not isinstance(config.get('lrt_grid_merge'), dict)`	Step_2_3_generate_remaining_grid_products.py:34
`RuntimeError: Worker source was not initialised.`	`_BASE is None`	Step_2_3_generate_remaining_grid_products.py:141

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`previous == expected_state`	Step_2_3_generate_remaining_grid_products.py:402
`0`	`Emitted by main`	Step_2_3_generate_remaining_grid_products.py:436
`1`	`except Exception`	Step_2_3_generate_remaining_grid_products.py:440

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

28 Step 2_4 10m Formation Status (DE)

Readmes/step_2_4_10m_formation_status/README_DE.md

Zweck

Erzeugt checkpointfaehige 10m-Formation-Status-Produkte.

Script

`scripts/Step_2_4_generate_10m_formation_status_products.py`

Eingaben

- `outputs/step_2_variants/<suffix>/step_2_1_susi_compatible/Formation_Status_Grid_withLRTCode.parquet`
- `outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg`

Es wird kein separates originales INSPIRE-10m-Grid eingelesen. Wie in Susis `3_10mgrid_prep.py` werden die 100 10m-Zellen je 100m-INSPIRE-ID in EPSG:3035 deterministisch abgeleitet: `x0=E100*100`, `y0=N100*100`, anschliessend `grid_id_10=10mN(N100*10+dy)E(E100*10+dx)` fuer `dx,dy=0..9`.

Outputs

- `outputs/step_2_variants/<suffix>/step_2_4_susi_10m/*`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_2_4_10m_formation`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `susi_10m_products`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_2_4_generate_10m_formation_status_products.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

Parquet-Parts und _batch_status erlauben Wiederaufnahme nach Timeout.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`in_progress`	`reset_required`	Step_2_4_generate_10m_formation_status_products.py:614
`complete`	`Ausgabe in main`	Step_2_4_generate_10m_formation_status_products.py:707

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_2_4_generate_10m_formation_status_products.py:43
`KeyError: Missing 'susi_10m_products' section in config.`	`'susi_10m_products' not in config`	Step_2_4_generate_10m_formation_status_products.py:47
`FileNotFoundError: Input file not found: {path}`	`not path.is_file()`	Step_2_4_generate_10m_formation_status_products.py:53
`ValueError: Invalid 100 m grid_id: {grid_id}`	`match is None`	Step_2_4_generate_10m_formation_status_products.py:101
`RuntimeError: Temporary final parquet is empty: {temporary}`	`not temporary.is_file() or temporary.stat().st_size == 0`	Step_2_4_generate_10m_formation_status_products.py:307
`RuntimeError: Step 2_4 worker was not initialised.`	`_WORKER_LRT is None or _WORKER_SETTINGS is None`	Step_2_4_generate_10m_formation_status_products.py:467
`FileNotFoundError: 10 m grid selection CSV not found: {path}`	`not path.is_file()`	Step_2_4_generate_10m_formation_status_products.py:503
`KeyError: 10 m grid selection CSV needs a 'grid_100m_id' or 'grid_id' column.`	`column is None`	Step_2_4_generate_10m_formation_status_products.py:510
`ValueError: 10 m grid selection CSV contains no usable grid IDs.`	`not selected`	Step_2_4_generate_10m_formation_status_products.py:516
`FileNotFoundError: Missing source parquet: {source}`	`not source.is_file()`	Step_2_4_generate_10m_formation_status_products.py:545
`FileNotFoundError: Missing LRT GPKG: {lrt_gpkg}`	`not lrt_gpkg.is_file()`	Step_2_4_generate_10m_formation_status_products.py:547
`ValueError: No selected 100 m grid IDs occur in the Step 2.1 product.`	`not grid_ids`	Step_2_4_generate_10m_formation_status_products.py:636

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`same_inputs and previous_state.get('status') == 'complete' and final_parquet.is_file() and (final_parquet.stat().st_size > 0) and (not args.force)`	Step_2_4_generate_10m_formation_status_products.py:593
`0`	`Ausgabe in main`	Step_2_4_generate_10m_formation_status_products.py:738
`1`	`except Exception`	Step_2_4_generate_10m_formation_status_products.py:748

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

29 Step 2_4 10m Formation Status (EN)

Readmes/step_2_4_10m_formation_status/README_EN.md

Purpose

Creates checkpointed 10m formation-status products.

Script

`scripts/Step_2_4_generate_10m_formation_status_products.py`

Inputs

- `outputs/step_2_variants/<suffix>/step_2_1_susi_compatible/Formation_Status_Grid_withLRTCode.parquet`
- `outputs/step_2_variants/<suffix>/step_2_0/lrt_<suffix>.gpkg`

No separate original INSPIRE 10m grid is read. As in Susi's `3_10mgrid_prep.py`, the 100 10m cells per 100m INSPIRE ID are derived deterministically in EPSG:3035: `x0=E100*100`, `y0=N100*100`, then `grid_id_10=10mN(N100*10+dy)E(E100*10+dx)` for `dx,dy=0..9`.

Outputs

- `outputs/step_2_variants/<suffix>/step_2_4_susi_10m/*`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_2_4_10m_formation`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `susi_10m_products`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_2_4_generate_10m_formation_status_products.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

Parquet parts and `_batch_status` allow processing to resume after a timeout.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`in_progress`	`reset_required`	Step_2_4_generate_10m_formation_status_products.py:614
`complete`	`Emitted by main`	Step_2_4_generate_10m_formation_status_products.py:707

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_2_4_generate_10m_formation_status_products.py:43
`KeyError: Missing 'susi_10m_products' section in config.`	`'susi_10m_products' not in config`	Step_2_4_generate_10m_formation_status_products.py:47
`FileNotFoundError: Input file not found: {path}`	`not path.is_file()`	Step_2_4_generate_10m_formation_status_products.py:53
`ValueError: Invalid 100 m grid_id: {grid_id}`	`match is None`	Step_2_4_generate_10m_formation_status_products.py:101
`RuntimeError: Temporary final parquet is empty: {temporary}`	`not temporary.is_file() or temporary.stat().st_size == 0`	Step_2_4_generate_10m_formation_status_products.py:307
`RuntimeError: Step 2_4 worker was not initialised.`	`_WORKER_LRT is None or _WORKER_SETTINGS is None`	Step_2_4_generate_10m_formation_status_products.py:467
`FileNotFoundError: 10 m grid selection CSV not found: {path}`	`not path.is_file()`	Step_2_4_generate_10m_formation_status_products.py:503
`KeyError: 10 m grid selection CSV needs a 'grid_100m_id' or 'grid_id' column.`	`column is None`	Step_2_4_generate_10m_formation_status_products.py:510
`ValueError: 10 m grid selection CSV contains no usable grid IDs.`	`not selected`	Step_2_4_generate_10m_formation_status_products.py:516
`FileNotFoundError: Missing source parquet: {source}`	`not source.is_file()`	Step_2_4_generate_10m_formation_status_products.py:545
`FileNotFoundError: Missing LRT GPKG: {lrt_gpkg}`	`not lrt_gpkg.is_file()`	Step_2_4_generate_10m_formation_status_products.py:547
`ValueError: No selected 100 m grid IDs occur in the Step 2.1 product.`	`not grid_ids`	Step_2_4_generate_10m_formation_status_products.py:636

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`same_inputs and previous_state.get('status') == 'complete' and final_parquet.is_file() and (final_parquet.stat().st_size > 0) and (not args.force)`	Step_2_4_generate_10m_formation_status_products.py:593
`0`	`Emitted by main`	Step_2_4_generate_10m_formation_status_products.py:738
`1`	`except Exception`	Step_2_4_generate_10m_formation_status_products.py:748

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

30 Step 2 Varianten - LRT-Sensitivitaetsanalyse

Readmes/step_2_variants/README_DE.md

Zweck

Alle GeoPackages aus `Biodiversity_data/Bundeslander/All_Bundeslander/All_Bundeslander_*.gpkg` werden als getrennte fachliche Varianten verarbeitet. Kein Datensatz ueberschreibt einen anderen. Der Dateisuffix nach `All_Bundeslander_` ist die stabile Varianten-ID.

Die Primärvariante ist derzeit `no_K_post2017`. Nur ihre Formation-Felder werden in der kompakten ID-Mastertabelle verwendet. Alle Varianten stehen zusaetzlich in einer normalisierten Vergleichstabelle.

Verarbeitung pro Variante

```
Eingangs-GPKG
  -> Step 2_0 LRT-Bereinigung
  -> Step 2_1 100m-Formation und Statusmatrix
  -> Step 2_2 Recording-Zuordnung
  -> Step 2_3 groebere Rasterprodukte
  -> Step 2_4 10m-Formation und Statusmatrix
  -> Step 7_1 Varianten-Mastertabelle
  -> Step 7_0 kompakte Haupt-Mastertabelle
```

Step 2_2, 2_3 und 2_4 beginnen nach Abschluss der Step-2_1-Arraystufe. Ein fehlender Vorgaengeroutput laesst nur den betroffenen Task mit einem klaren Fehler enden. Die Abhaengigkeit `afterany` verhindert dauerhaft wartende `DependencyNeverSatisfied`-Jobs. Auf Horeka laufen verschiedene Varianten als Slurm-Array parallel. Lokal wird die Parallelitaet durch `local_max_parallel_variants` begrenzt.

Ausgaben

Alle variantenbezogenen Dateien liegen unter:

```
outputs/step_2_variants/<suffix>/step_2_0/
outputs/step_2_variants/<suffix>/step_2_1/
outputs/step_2_variants/<suffix>/step_2_1_susi_compatible/
outputs/step_2_variants/<suffix>/step_2_2/
outputs/step_2_variants/<suffix>/step_2_3/
outputs/step_2_variants/<suffix>/step_2_4_susi_10m/
```

Zentrale Produkte:

```
outputs/step_2_variants/variant_index.json
outputs/Bio_O_Ton_Formation_Variants.csv
outputs/Bio_O_Ton_Formation_Variants.parquet
outputs/Bio_O_Ton_Formation_Variants_summary.json
```

Die Varianten-Mastertabelle hat genau eine Zeile pro `dawn_chorus_id` und `lrt_variant`. Sie enthaelt 100m-/10m-Majority, Conservation Status, Majority-Werte, Disputed-Flags und Produktstatus.

Inkrementelles Verhalten

Jede Variante besitzt eigene States und Checkpoints. Eine geaenderte Eingangsdatei invalidiert nur ihren Zweig. Neue Recording-IDs erfordern nur Step 2_2 und das anschliessende Mastertable-Update, solange die raeumlichen Produkte unveraendert sind. `from_scratch` uebergibt `--force`; Step 2_1 und Step 2_4 koennen nach einem Zeitlimit an ihren Chunk-Checkpoints fortsetzen.

Start

Horeka:

```
bash submit_step2_variants_horeka.sh add_new_ids
bash submit_step2_variants_horeka.sh from_scratch
```

Lokal:

```
powershell -ExecutionPolicy Bypass -File .\scripts_local_run\run_step2_variants_local.ps1 -Mode
add_new_ids -SkipEnvironmentSetup
```

Der Variantenlauf darf nicht gleichzeitig mit einem anderen schreibenden Pipeline-Lauf ausgefuehrt werden.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`complete`	`Ausgabe in build_variant_rows`	Step_7_1_update_formation_variant_table.py:294
`partial`	`Ausgabe in build_variant_rows`	Step_7_1_update_formation_variant_table.py:294

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`ValueError: Cannot derive a variant suffix from {path.name}`	`not suffix`	step2_variants.py:60
`TypeError: 'lrt_variants' must be a JSON object.`	`not isinstance(settings, dict)`	step2_variants.py:67
`NotADirectoryError: LRT variant input directory not found: {input_dir}`	`not input_dir.is_dir()`	step2_variants.py:73
`FileNotFoundError: No LRT variant GeoPackages match {input_dir / pattern}`	`not sources`	step2_variants.py:79
`ValueError: Duplicate LRT variant suffix: {suffix}`	`key in seen`	step2_variants.py:86
`ValueError: 'lrt_variants.primary_suffix' must not be empty.`	`not primary`	step2_variants.py:169
`ValueError: Configured primary LRT variant is missing: {primary}. Available: {', '.join(sorted(available))}`	`primary not in available`	step2_variants.py:171
`ValueError: --task-index must be between 0 and {len(variants) - 1}`	`not 0 <= args.task_index < len(variants)`	step2_variants.py:316
`ValueError: --stage is required with --task-index`	`args.stage is None`	step2_variants.py:318
`ValueError: Select --stage, --all-stages, --prepare-only or --task-count`	`not stages`	step2_variants.py:330
`KeyError: Missing public_lrt_cleaning section in config.`	`'public_lrt_cleaning' not in config`	Step_2_5_clean_public_lrts.py:34
`RuntimeError: No public LRT polygons remained after cleaning.`	`not results`	Step_2_5_clean_public_lrts.py:159
`KeyError: Missing public_lrt_grid_merge section in config.`	`'public_lrt_grid_merge' not in config`	Step_2_6_merge_public_lrts_and_grid.py:32
`KeyError: Metadata missing columns: {sorted(missing)}`	`missing`	Step_7_1_update_formation_variant_table.py:170

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`args.task_count`	step2_variants.py:309
`0`	`args.prepare_only`	step2_variants.py:311
`0`	`Ausgabe in main`	step2_variants.py:343
`0`	`should_skip(output_gpkg, state_file, expected_state, args.force)`	Step_2_5_clean_public_lrts.py:127
`0`	`Ausgabe in main`	Step_2_5_clean_public_lrts.py:188
`1`	`except Exception`	Step_2_5_clean_public_lrts.py:191
`0`	`should_skip(output_paths, state_file, expected_state, args.force)`	Step_2_6_merge_public_lrts_and_grid.py:120
`0`	`Ausgabe in main`	Step_2_6_merge_public_lrts_and_grid.py:176
`1`	`except Exception`	Step_2_6_merge_public_lrts_and_grid.py:179
`0`	`not rebuilt and output_csv.is_file() and output_parquet.is_file()`	Step_7_1_update_formation_variant_table.py:373
`0`	`Ausgabe in main`	Step_7_1_update_formation_variant_table.py:462
`1`	`except Exception`	Step_7_1_update_formation_variant_table.py:465

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

31 Step 2 Variants - LRT Sensitivity Analysis

Readmes/step_2_variants/README_EN.md

All `All_Bundeslander_*.gpkg` files are processed as isolated variants. The suffix following `All_Bundeslander_` is the stable variant identifier, and no variant overwrites another one.

Each branch executes Steps 2_0 through 2_4 and writes to `outputs/step_2_variants/<suffix>/`. The configured primary variant is `no_K_post2017`; its fields feed the compact ID-level master table. Step 7_1 additionally writes `outputs/Bio_O_Ton_Formation_Variants.parquet` and CSV with one row per `dawn_chorus_id` and `lrt_variant` for sensitivity comparisons.

Horeka uses one Slurm array task per variant and stage. Stage barriers use `afterany`, so a failed variant produces an explicit downstream error instead of leaving `DependencyNeverSatisfied` jobs behind:

```
bash submit_step2_variants_horeka.sh add_new_ids
bash submit_step2_variants_horeka.sh from_scratch
```

Local execution uses the same Python steps and isolated configuration files:

```
powershell -ExecutionPolicy Bypass -File .\scripts_local_run\run_step2_variants_local.ps1 -Mode
add_new_ids -SkipEnvironmentSetup
```

States and checkpoints are variant-specific. A changed source only invalidates its own branch. Do not run this workflow concurrently with another pipeline writer.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`complete`	`Emitted by build_variant_rows`	Step_7_1_update_formation_variant_table.py:294
`partial`	`Emitted by build_variant_rows`	Step_7_1_update_formation_variant_table.py:294

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`ValueError: Cannot derive a variant suffix from {path.name}`	`not suffix`	step2_variants.py:60
`TypeError: 'lrt_variants' must be a JSON object.`	`not isinstance(settings, dict)`	step2_variants.py:67
`NotADirectoryError: LRT variant input directory not found: {input_dir}`	`not input_dir.is_dir()`	step2_variants.py:73
`FileNotFoundError: No LRT variant GeoPackages match {input_dir / pattern}`	`not sources`	step2_variants.py:79
`ValueError: Duplicate LRT variant suffix: {suffix}`	`key in seen`	step2_variants.py:86
`ValueError: 'lrt_variants.primary_suffix' must not be empty.`	`not primary`	step2_variants.py:169
`ValueError: Configured primary LRT variant is missing: {primary}. Available: {', '.join(sorted(available))}`	`primary not in available`	step2_variants.py:171
`ValueError: --task-index must be between 0 and {len(variants) - 1}`	`not 0 <= args.task_index < len(variants)`	step2_variants.py:316
`ValueError: --stage is required with --task-index`	`args.stage is None`	step2_variants.py:318
`ValueError: Select --stage, --all-stages, --prepare-only or --task-count`	`not stages`	step2_variants.py:330
`KeyError: Missing public_lrt_cleaning section in config.`	`'public_lrt_cleaning' not in config`	Step_2_5_clean_public_lrts.py:34
`RuntimeError: No public LRT polygons remained after cleaning.`	`not results`	Step_2_5_clean_public_lrts.py:159
`KeyError: Missing public_lrt_grid_merge section in config.`	`'public_lrt_grid_merge' not in config`	Step_2_6_merge_public_lrts_and_grid.py:32
`KeyError: Metadata missing columns: {sorted(missing)}`	`missing`	Step_7_1_update_formation_variant_table.py:170

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`args.task_count`	step2_variants.py:309
`0`	`args.prepare_only`	step2_variants.py:311
`0`	`Emitted by main`	step2_variants.py:343
`0`	`should_skip(output_gpkg, state_file, expected_state, args.force)`	Step_2_5_clean_public_lrts.py:127
`0`	`Emitted by main`	Step_2_5_clean_public_lrts.py:188
`1`	`except Exception`	Step_2_5_clean_public_lrts.py:191
`0`	`should_skip(output_paths, state_file, expected_state, args.force)`	Step_2_6_merge_public_lrts_and_grid.py:120
`0`	`Emitted by main`	Step_2_6_merge_public_lrts_and_grid.py:176
`1`	`except Exception`	Step_2_6_merge_public_lrts_and_grid.py:179
`0`	`not rebuilt and output_csv.is_file() and output_parquet.is_file()`	Step_7_1_update_formation_variant_table.py:373
`0`	`Emitted by main`	Step_7_1_update_formation_variant_table.py:462
`1`	`except Exception`	Step_7_1_update_formation_variant_table.py:465

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

32 Step 3 Media Inventory And Download (DE)

Readmes/step_3_media/README_DE.md

Zweck

Prueft und ergaenzt Audio- und Bilddateien.

Script

`scripts/Step_3_0_a_audio_inventory.py, Step_3_0_b_photo_inventory.py, Step_3_1_a_audio_download.py, Step_3_1_b_photo_download.py`

Eingaben

- `PointData/SoundRecordings`
- `PointData/Images_SoundRecordings`
- `PointData/dawn-chorus-soundscape.csv`

Outputs

- `outputs/step_3_0_*/*`
- `outputs/step_3_1_*/*`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_3_0_audio_inventory`, `step_3_0_photo_inventory`, `step_3_1_audio_download`, `step_3_1_photo_download`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `audio_inventory`, `photo_inventory`, `audio_download`, `photo_download`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_3_0_a_audio_inventory.py --config config.horeka.json`
- `python scripts/Step_3_0_b_photo_inventory.py --config config.horeka.json`
- `python scripts/Step_3_1_a_audio_download.py --config config.horeka.json`
- `python scripts/Step_3_1_b_photo_download.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

Inventare und Retry-Logs verhindern doppelte Downloads.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Code / Meldung	Bedeutung / Ausloeser	Quelle
`WinError 53`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_a_audio_inventory.py:92
`WinError 64`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_a_audio_inventory.py:92
`WinError 121`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_a_audio_inventory.py:92
`WinError 995`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_a_audio_inventory.py:92
`WinError 1203`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_a_audio_inventory.py:92
`no_audio_stream_found`	Kein Audiostream in der Datei gefunden.	Step_3_0_a_audio_inventory.py:218
`multiple_audio_streams:{len(audio_streams)}`	Datei enthaelt mehrere Audiostreams.	Step_3_0_a_audio_inventory.py:238
`invalid_or_missing_sample_rate`	Samplingrate fehlt oder ist nicht positiv.	Step_3_0_a_audio_inventory.py:275
`invalid_or_missing_channel_count`	Audiokanalzahl fehlt oder ist nicht positiv.	Step_3_0_a_audio_inventory.py:278
`no_audio_frames_decoded`	Decoder lieferte keine Audioframes.	Step_3_0_a_audio_inventory.py:303
`duration_unavailable`	Audiodauer konnte nicht bestimmt werden.	Step_3_0_a_audio_inventory.py:318
`invalid_duration:{duration}`	Ermittelte Audiodauer ist nicht positiv.	Step_3_0_a_audio_inventory.py:321
`pyav_decode_failed:{type(exc).__name__}:{exc}`	PyAV konnte Audio nicht vollstaendig decodieren.	Step_3_0_a_audio_inventory.py:327
`filename_does_not_match_<id>_audio_pattern`	Audio-Dateiname entspricht nicht dem erwarteten ID-Muster.	Step_3_0_a_audio_inventory.py:361
`source_stat_failed:{type(exc).__name__}:{exc}`	Quelldatei konnte nicht per Dateisystem-Stat abgefragt werden.	Step_3_0_a_audio_inventory.py:369
`empty_file`	Vorhandene Datei hat null Bytes.	Step_3_0_a_audio_inventory.py:399
`duration_not_{target}s:observed={duration}s,allowed={target - tolerance}-{target + tolerance}s`	Audiodauer weicht vom Ziel ab; nur Hinweis, setzt allein keine Audio-Issue-Flag. Analysefilter separat waehlen.	Step_3_0_a_audio_inventory.py:416
`WinError 53`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_b_photo_inventory.py:46
`WinError 64`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_b_photo_inventory.py:46
`WinError 121`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_b_photo_inventory.py:46
`WinError 995`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_b_photo_inventory.py:46
`WinError 1203`	Vom Inventar als Windows-/Netztransportproblem eingestuft; Mount/Verbindung pruefen statt Dateidefekt anzunehmen.	Step_3_0_b_photo_inventory.py:46
`filename_does_not_match_<id>_photo_pattern`	Foto-Dateiname entspricht nicht dem erwarteten ID-Muster.	Step_3_0_b_photo_inventory.py:139
`copy_failed:{type(exc).__name__}:{exc}`	Dateikopie ist fehlgeschlagen.	Step_3_0_b_photo_inventory.py:158

Code / Meldung	Bedeutung / Ausloeser	Quelle
`copy_size_mismatch`	Dateigroessen von Quelle und Kopie stimmen nicht ueberein.	Step_3_0_b_photo_inventory.py:165
`copy_verification_failed:{type(exc).__name__}:{exc}`	Die Kontrolle nach dem Kopieren ist fehlgeschlagen.	Step_3_0_b_photo_inventory.py:167
`destination_missing_after_copy`	Zieldatei fehlt nach dem Kopierversuch.	Step_3_0_b_photo_inventory.py:169
`image_verify_failed:{type(exc).__name__}:{exc}`	Pillow-Dateipruefung fehlgeschlagen.	Step_3_0_b_photo_inventory.py:189
`pixel_load_failed:{type(exc).__name__}:{exc}`	Bildheader lesbar, aber Pixel konnten nicht geladen werden.	Step_3_0_b_photo_inventory.py:197
`invalid_dimensions:{width}x{height}`	Raster-/Bildbreite oder -hoehe ist ungueltig.	Step_3_0_b_photo_inventory.py:200
`no_audio_stream_found`	Kein Audiostream in der Datei gefunden.	Step_3_1_a_audio_download.py:172
`multiple_audio_streams:{len(streams)}`	Datei enthaelt mehrere Audiostreams.	Step_3_1_a_audio_download.py:189
`duration_unavailable`	Audiodauer konnte nicht bestimmt werden.	Step_3_1_a_audio_download.py:232
`no_audio_frames_decoded`	Decoder lieferte keine Audioframes.	Step_3_1_a_audio_download.py:245
`pyav_decode_failed:{type(exc).__name__}:{exc}`	PyAV konnte Audio nicht vollstaendig decodieren.	Step_3_1_a_audio_download.py:259
`HTTPError:{exc.code}:{exc.reason}`	HTTP-Download fehlgeschlagen; Suffix enthaelt Status und Servermeldung. Retry-Regeln beachten.	Step_3_1_a_audio_download.py:330
`{type(exc).__name__}:{exc}`	Originaler Exception-Typ und Meldung; kein eigener statischer Fehlercode.	Step_3_1_a_audio_download.py:337
`duration_not_{target}s:observed={duration}s,allowed={target - tolerance}-{target + tolerance}s`	Audiodauer weicht vom Ziel ab; nur Hinweis, setzt allein keine Audio-Issue-Flag. Analysefilter separat waehlen.	Step_3_1_a_audio_download.py:356
`missing_audio_url`	Fuer den Audio-Download fehlt eine URL.	Step_3_1_a_audio_download.py:448
`HTTPError:{exc.code}:{exc.reason}`	HTTP-Download fehlgeschlagen; Suffix enthaelt Status und Servermeldung. Retry-Regeln beachten.	Step_3_1_b_photo_download.py:127
`{type(exc).__name__}:{exc}`	Originaler Exception-Typ und Meldung; kein eigener statischer Fehlercode.	Step_3_1_b_photo_download.py:130
`image_verify_failed:{type(exc).__name__}:{exc}`	Pillow-Dateipruefung fehlgeschlagen.	Step_3_1_b_photo_download.py:149
`pixel_load_failed:{type(exc).__name__}:{exc}`	Bildheader lesbar, aber Pixel konnten nicht geladen werden.	Step_3_1_b_photo_download.py:157
`missing_photo_url`	Fuer den Foto-Download fehlt eine URL.	Step_3_1_b_photo_download.py:203

Statuswerte (nicht alle sind Fehler)

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`FileNotFoundError: Config file not found: {path}`	`not path.is_file()`	Step_3_0_a_audio_inventory.py:117
`TypeError: 'audio_inventory' must be a JSON object.`	`not isinstance(section, dict)`	Step_3_0_a_audio_inventory.py:125
`NotADirectoryError: Audio source directory not found: {source_dir}`	`not source_dir.is_dir()`	Step_3_0_a_audio_inventory.py:133
`ConnectionError: LSDF transport failed while listing {path}: {exc}`	`is_transport_error(exc)`	Step_3_0_a_audio_inventory.py:152
`FileNotFoundError: Dawn Chorus metadata CSV not found: {metadata_csv}`	`not metadata_csv.is_file()`	Step_3_0_a_audio_inventory.py:172
`KeyError: Missing metadata columns in {metadata_csv}: {sorted(missing)}`	`missing`	Step_3_0_a_audio_inventory.py:179
`ValueError: target_duration_seconds must be greater than zero.`	`target <= 0`	Step_3_0_a_audio_inventory.py:636
`ValueError: duration_tolerance_seconds must not be negative.`	`tolerance < 0`	Step_3_0_a_audio_inventory.py:641
`ConnectionError: Audio inventory stopped because the LSDF transport failed ({len(transport_rows)} files in the current validation block). Sample: {sample}`	`transport_rows`	Step_3_0_a_audio_inventory.py:788
`FileNotFoundError: Config file not found: {path}`	`not path.is_file()`	Step_3_0_b_photo_inventory.py:58
`TypeError: 'photo_inventory' must be a JSON object.`	`not isinstance(section, dict)`	Step_3_0_b_photo_inventory.py:63
`NotADirectoryError: Photo source directory not found: {source_dir}`	`not source_dir.is_dir()`	Step_3_0_b_photo_inventory.py:69
`FileNotFoundError: Dawn Chorus metadata CSV not found: {metadata_csv}`	`not metadata_csv.is_file()`	Step_3_0_b_photo_inventory.py:96
`KeyError: Missing metadata columns in {metadata_csv}: {sorted(missing)}`	`missing`	Step_3_0_b_photo_inventory.py:103
`NotADirectoryError: Photo {label} directory not found: {raw}`	`not create_if_missing`	Step_3_0_b_photo_inventory.py:122
`ConnectionError: Photo inventory stopped because the LSDF transport failed. Affected rows: {len(transport_rows)}; sample: {transport_rows[0].get('issues', '')}`	`transport_rows`	Step_3_0_b_photo_inventory.py:375
`TypeError: 'audio_download' must be an object.`	`not isinstance(section, dict)`	Step_3_1_a_audio_download.py:99
`OSError: Downloaded file is empty.`	`temporary.stat().st_size == 0`	Step_3_1_a_audio_download.py:323
`OSError: Downloaded file is empty.`	`temporary.stat().st_size == 0`	Step_3_1_b_photo_download.py:122

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`args.list_only`	Step_3_0_a_audio_inventory.py:686
`0`	`Ausgabe in main`	Step_3_0_a_audio_inventory.py:972
`1`	`except Exception`	Step_3_0_a_audio_inventory.py:979
`0`	`args.list_only`	Step_3_0_b_photo_inventory.py:331
`0`	`Ausgabe in main`	Step_3_0_b_photo_inventory.py:423
`1`	`except Exception`	Step_3_0_b_photo_inventory.py:426
`0`	`Ausgabe in main`	Step_3_1_a_audio_download.py:906
`1`	`except Exception`	Step_3_1_a_audio_download.py:913
`0`	`Ausgabe in main`	Step_3_1_b_photo_download.py:430
`1`	`except Exception`	Step_3_1_b_photo_download.py:433
`1`	`failures or not image_source_usable or (not image_output_usable)`	step3_path_preflight.py:153
`0`	`Ausgabe in main`	step3_path_preflight.py:156
`1`	`except Exception`	step3_path_preflight.py:159

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

33 Step 3 Media Inventory And Download (EN)

Readmes/step_3_media/README_EN.md

Purpose

Validates and fills audio and photo files.

Script

`scripts/Step_3_0_a_audio_inventory.py, Step_3_0_b_photo_inventory.py, Step_3_1_a_audio_download.py, Step_3_1_b_photo_download.py`

Inputs

- `PointData/SoundRecordings`
- `PointData/Images_SoundRecordings`
- `PointData/dawn-chorus-soundscape.csv`

Outputs

- `outputs/step_3_0_*/*`
- `outputs/step_3_1_*/*`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_3_0_audio_inventory`, `step_3_0_photo_inventory`, `step_3_1_audio_download`, `step_3_1_photo_download`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `audio_inventory`, `photo_inventory`, `audio_download`, `photo_download`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_3_0_a_audio_inventory.py --config config.horeka.json`
- `python scripts/Step_3_0_b_photo_inventory.py --config config.horeka.json`
- `python scripts/Step_3_1_a_audio_download.py --config config.horeka.json`
- `python scripts/Step_3_1_b_photo_download.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

Inventories and persistent retry logs prevent duplicate downloads.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

Code / Message	Meaning / Trigger	Source
`WinError 53`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_a_audio_inventory.py:92
`WinError 64`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_a_audio_inventory.py:92
`WinError 121`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_a_audio_inventory.py:92
`WinError 995`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_a_audio_inventory.py:92
`WinError 1203`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_a_audio_inventory.py:92
`no_audio_stream_found`	No audio stream found in the file.	Step_3_0_a_audio_inventory.py:218
`multiple_audio_streams:{len(audio_streams)}`	File contains multiple audio streams.	Step_3_0_a_audio_inventory.py:238
`invalid_or_missing_sample_rate`	Sample rate is missing or nonpositive.	Step_3_0_a_audio_inventory.py:275
`invalid_or_missing_channel_count`	Audio channel count is missing or nonpositive.	Step_3_0_a_audio_inventory.py:278
`no_audio_frames_decoded`	Decoder returned no audio frames.	Step_3_0_a_audio_inventory.py:303
`duration_unavailable`	Audio duration could not be determined.	Step_3_0_a_audio_inventory.py:318
`invalid_duration:{duration}`	Determined audio duration is not positive.	Step_3_0_a_audio_inventory.py:321
`pyav_decode_failed:{type(exc).__name__}:{exc}`	PyAV could not decode the audio successfully.	Step_3_0_a_audio_inventory.py:327
`filename_does_not_match_<id>_audio_pattern`	Audio filename does not match the expected ID pattern.	Step_3_0_a_audio_inventory.py:361
`source_stat_failed:{type(exc).__name__}:{exc}`	Filesystem stat failed for the source file.	Step_3_0_a_audio_inventory.py:369
`empty_file`	Existing file has zero bytes.	Step_3_0_a_audio_inventory.py:399
`duration_not_{target}s:observed={duration}s,allowed={target - tolerance}-{target + tolerance}s`	Audio duration differs from target; informational only and alone does not set an audio issue flag. Apply analysis filters separately.	Step_3_0_a_audio_inventory.py:416
`WinError 53`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_b_photo_inventory.py:46
`WinError 64`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_b_photo_inventory.py:46
`WinError 121`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_b_photo_inventory.py:46
`WinError 995`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_b_photo_inventory.py:46

Code / Message	Meaning / Trigger	Source
`WinError 1203`	Classified by inventory as a Windows/network transport problem; inspect mount/connectivity rather than assuming corrupt media.	Step_3_0_b_photo_inventory.py:46
`filename_does_not_match_<id>_photo_pattern`	Photo filename does not match the expected ID pattern.	Step_3_0_b_photo_inventory.py:139
`copy_failed:{type(exc).__name__}:{exc}`	File copy failed.	Step_3_0_b_photo_inventory.py:158
`copy_size_mismatch`	Source and copied file sizes differ.	Step_3_0_b_photo_inventory.py:165
`copy_verification_failed:{type(exc).__name__}:{exc}`	Verification after copying failed.	Step_3_0_b_photo_inventory.py:167
`destination_missing_after_copy`	Destination is missing after the copy attempt.	Step_3_0_b_photo_inventory.py:169
`image_verify_failed:{type(exc).__name__}:{exc}`	Pillow image verification failed.	Step_3_0_b_photo_inventory.py:189
`pixel_load_failed:{type(exc).__name__}:{exc}`	Image pixels could not be loaded after header inspection.	Step_3_0_b_photo_inventory.py:197
`invalid_dimensions:{width}x{height}`	Raster/image width or height is invalid.	Step_3_0_b_photo_inventory.py:200
`no_audio_stream_found`	No audio stream found in the file.	Step_3_1_a_audio_download.py:172
`multiple_audio_streams:{len(streams)}`	File contains multiple audio streams.	Step_3_1_a_audio_download.py:189
`duration_unavailable`	Audio duration could not be determined.	Step_3_1_a_audio_download.py:232
`no_audio_frames_decoded`	Decoder returned no audio frames.	Step_3_1_a_audio_download.py:245
`pyav_decode_failed:{type(exc).__name__}:{exc}`	PyAV could not decode the audio successfully.	Step_3_1_a_audio_download.py:259
`HTTPError:{exc.code}:{exc.reason}`	HTTP download failed; suffix contains status and server message. Follow retry policy.	Step_3_1_a_audio_download.py:330
`{type(exc).__name__}:{exc}`	Original exception type and message; not a separate fixed code.	Step_3_1_a_audio_download.py:337
`duration_not_{target}s:observed={duration}s,allowed={target - tolerance}-{target + tolerance}s`	Audio duration differs from target; informational only and alone does not set an audio issue flag. Apply analysis filters separately.	Step_3_1_a_audio_download.py:356
`missing_audio_url`	Audio download URL is missing.	Step_3_1_a_audio_download.py:448
`HTTPError:{exc.code}:{exc.reason}`	HTTP download failed; suffix contains status and server message. Follow retry policy.	Step_3_1_b_photo_download.py:127
`{type(exc).__name__}:{exc}`	Original exception type and message; not a separate fixed code.	Step_3_1_b_photo_download.py:130
`image_verify_failed:{type(exc).__name__}:{exc}`	Pillow image verification failed.	Step_3_1_b_photo_download.py:149
`pixel_load_failed:{type(exc).__name__}:{exc}`	Image pixels could not be loaded after header inspection.	Step_3_1_b_photo_download.py:157
`missing_photo_url`	Photo download URL is missing.	Step_3_1_b_photo_download.py:203

Status Values (Not All Are Errors)

No fixed values in this category in the associated source.

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`FileNotFoundError: Config file not found: {path}`	`not path.is_file()`	Step_3_0_a_audio_inventory.py:117
`TypeError: 'audio_inventory' must be a JSON object.`	`not isinstance(section, dict)`	Step_3_0_a_audio_inventory.py:125
`NotADirectoryError: Audio source directory not found: {source_dir}`	`not source_dir.is_dir()`	Step_3_0_a_audio_inventory.py:133
`ConnectionError: LSDF transport failed while listing {path}: {exc}`	`is_transport_error(exc)`	Step_3_0_a_audio_inventory.py:152
`FileNotFoundError: Dawn Chorus metadata CSV not found: {metadata_csv}`	`not metadata_csv.is_file()`	Step_3_0_a_audio_inventory.py:172
`KeyError: Missing metadata columns in {metadata_csv}: {sorted(missing)}`	`missing`	Step_3_0_a_audio_inventory.py:179
`ValueError: target_duration_seconds must be greater than zero.`	`target <= 0`	Step_3_0_a_audio_inventory.py:636
`ValueError: duration_tolerance_seconds must not be negative.`	`tolerance < 0`	Step_3_0_a_audio_inventory.py:641
`ConnectionError: Audio inventory stopped because the LSDF transport failed ({len(transport_rows)} files in the current validation block). Sample: {sample}`	`transport_rows`	Step_3_0_a_audio_inventory.py:788
`FileNotFoundError: Config file not found: {path}`	`not path.is_file()`	Step_3_0_b_photo_inventory.py:58
`TypeError: 'photo_inventory' must be a JSON object.`	`not isinstance(section, dict)`	Step_3_0_b_photo_inventory.py:63
`NotADirectoryError: Photo source directory not found: {source_dir}`	`not source_dir.is_dir()`	Step_3_0_b_photo_inventory.py:69
`FileNotFoundError: Dawn Chorus metadata CSV not found: {metadata_csv}`	`not metadata_csv.is_file()`	Step_3_0_b_photo_inventory.py:96
`KeyError: Missing metadata columns in {metadata_csv}: {sorted(missing)}`	`missing`	Step_3_0_b_photo_inventory.py:103
`NotADirectoryError: Photo {label} directory not found: {raw}`	`not create_if_missing`	Step_3_0_b_photo_inventory.py:122
`ConnectionError: Photo inventory stopped because the LSDF transport failed. Affected rows: {len(transport_rows)}; sample: {transport_rows[0].get('issues', '')}`	`transport_rows`	Step_3_0_b_photo_inventory.py:375
`TypeError: 'audio_download' must be an object.`	`not isinstance(section, dict)`	Step_3_1_a_audio_download.py:99
`OSError: Downloaded file is empty.`	`temporary.stat().st_size == 0`	Step_3_1_a_audio_download.py:323
`OSError: Downloaded file is empty.`	`temporary.stat().st_size == 0`	Step_3_1_b_photo_download.py:122

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`args.list_only`	Step_3_0_a_audio_inventory.py:686
`0`	`Emitted by main`	Step_3_0_a_audio_inventory.py:972
`1`	`except Exception`	Step_3_0_a_audio_inventory.py:979
`0`	`args.list_only`	Step_3_0_b_photo_inventory.py:331
`0`	`Emitted by main`	Step_3_0_b_photo_inventory.py:423
`1`	`except Exception`	Step_3_0_b_photo_inventory.py:426
`0`	`Emitted by main`	Step_3_1_a_audio_download.py:906
`1`	`except Exception`	Step_3_1_a_audio_download.py:913
`0`	`Emitted by main`	Step_3_1_b_photo_download.py:430
`1`	`except Exception`	Step_3_1_b_photo_download.py:433
`1`	`failures or not image_source_usable or (not image_output_usable)`	step3_path_preflight.py:153
`0`	`Emitted by main`	step3_path_preflight.py:156
`1`	`except Exception`	step3_path_preflight.py:159

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

34 Step 4 Sentinel-2 Mirror And Inventory (DE)

Readmes/step_4_sentinel2/README_DE.md

Zweck

Spiegelt externe Sentinel-2-Drive-Dateien und inventarisiert GeoTIFFs. Die Score-Tabelle wird ueber die konfigurierte ID-Spalte `DC_id` verbunden; gaengige Alternativen wie `id` und `dawn_chorus_id` werden ebenfalls erkannt.

Script

`scripts/Step_4_1_Sentinel2_download.py, scripts/Step_4_0_Sentinel2_inventory.py`

Eingaben

- `Google Drive token.json`
- `PointData/S2`
- `PointData/S2_Scores.csv`

Outputs

- `PointData/S2/*.tif`
- `outputs/step_4_0_Sentinel2_inventory/*`
- `outputs/step_4_1_sentinel2_download/*`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_4_1_sentinel2_mirror`, `step_4_0_sentinel2_inventory`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `sentinel2_download`, `sentinel2_inventory`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_4_1_Sentinel2_download.py --config config.horeka.json`
- `python scripts/Step_4_0_Sentinel2_inventory.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

Drive-Log, Dateigroesse und mtime steuern inkrementelle Verarbeitung.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Code / Meldung	Bedeutung / Ausloeser	Quelle
`score_rows_with_invalid_id:{invalid_id_count}`	Scoredatei enthaelt Zeilen ohne gueltige ID.	Step_4_0_Sentinel2_inventory.py:223
`duplicate_score_rows:{row_count}`	Mehrere Score-Zeilen fuer dieselbe ID.	Step_4_0_Sentinel2_inventory.py:276
`non_numeric_or_missing_scores:{missing_numeric_count}`	Scorewerte sind nicht numerisch oder fehlen.	Step_4_0_Sentinel2_inventory.py:281
`scores_outside_0_1:{invalid_range_count}`	Numerische Sentinel-Scores liegen ausserhalb 0 bis 1.	Step_4_0_Sentinel2_inventory.py:286
`quality_score_missing`	Fuer die Aufnahme liegt kein Score vor.	Step_4_0_Sentinel2_inventory.py:291
`no_valid_quality_score`	Kein gueltiger Sentinel-Score in den vorhandenen Zeilen.	Step_4_0_Sentinel2_inventory.py:294
`too_cloudy_sentinel_not_available`	Sentinel-Score ist genau 0: zu bewoelkt, Sentinel-Daten nicht nutzbar; eine Datei kann trotzdem vorhanden sein.	Step_4_0_Sentinel2_inventory.py:297
`unexpected_raster_driver:{dataset.driver}`	Rasterformat ist nicht GTiff.	Step_4_0_Sentinel2_inventory.py:362
`invalid_dimensions:{dataset.width}x{dataset.height}`	Raster-/Bildbreite oder -hoehe ist ungueltig.	Step_4_0_Sentinel2_inventory.py:367
`no_raster_bands`	Raster enthaelt keine Baender.	Step_4_0_Sentinel2_inventory.py:372
`missing_crs`	Raster hat kein Koordinatenreferenzsystem.	Step_4_0_Sentinel2_inventory.py:375
`identity_geotransform`	Raster besitzt nur die Identitaetstransformation; Georeferenzierung pruefen.	Step_4_0_Sentinel2_inventory.py:378
`non_finite_bounds`	Rastergrenzen enthalten NaN oder unendliche Werte.	Step_4_0_Sentinel2_inventory.py:391
`invalid_bounds`	Rastergrenzen haben keine gueltige raeumliche Ausdehnung.	Step_4_0_Sentinel2_inventory.py:397
`raster_read_failed:{type(exc).__name__}:{exc}`	Raster konnte nicht geoeffnet oder gelesen werden.	Step_4_0_Sentinel2_inventory.py:452
`all_pixels_nodata`	Alle geprueften Rasterpixel sind NoData.	Step_4_0_Sentinel2_inventory.py:474
`contains_nan_values:{nan_values}`	Raster enthaelt NaN-Werte.	Step_4_0_Sentinel2_inventory.py:477
`contains_infinite_values:{infinite_values}`	Raster enthaelt unendliche Werte.	Step_4_0_Sentinel2_inventory.py:482
`constant_raster:value={global_min}`	Alle gueltigen Rasterwerte sind identisch; Plausibilitaet pruefen.	Step_4_0_Sentinel2_inventory.py:487
`filename_does_not_contain_id_before_tif_extension`	TIFF-Dateiname liefert keine erwartete Aufnahme-ID.	Step_4_0_Sentinel2_inventory.py:528
`source_stat_failed:{type(exc).__name__}:{exc}`	Quelldatei konnte nicht per Dateisystem-Stat abgefragt werden.	Step_4_0_Sentinel2_inventory.py:536
`empty_file`	Vorhandene Datei hat null Bytes.	Step_4_0_Sentinel2_inventory.py:578
`duplicate_tif_files_for_id:{len(id_rows)}`	Mehrere TIFF-Dateien sind derselben Aufnahme zugeordnet.	Step_4_0_Sentinel2_inventory.py:952
`quality_score_missing`	Fuer die Aufnahme liegt kein Score vor.	Step_4_0_Sentinel2_inventory.py:973

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
skipped_existing_immutable	dc_id in immutable_ids	sentinel2_gee.py:666
completed_existing_task	state == 'COMPLETED'	sentinel2_gee.py:683
ready	Ausgabe in run_gee_drive_exports	sentinel2_gee.py:721
score_only	score_only	sentinel2_gee.py:748
score_only	bool(getattr(args, 'score_only', False))	sentinel2_gee.py:815
skipped_existing	ok	sentinel2_gee.py:837
downloaded	ok	sentinel2_gee.py:847
validation_failed	not (ok)	sentinel2_gee.py:850
download_failed	except Exception	sentinel2_gee.py:854

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`FileNotFoundError: Config file not found: {path}`	`not path.is_file()`	Step_4_0_Sentinel2_inventory.py:105
`TypeError: 'sentinel2_inventory' must be a JSON object.`	`not isinstance(section, dict)`	Step_4_0_Sentinel2_inventory.py:113
`NotADirectoryError: Sentinel-2 source directory not found: {source_dir}`	`not source_dir.is_dir()`	Step_4_0_Sentinel2_inventory.py:124
`ValueError: Missing Dawn Chorus ID column in Sentinel-2 score CSV. Tried {[candidate for candidate in candidates if candidate]}. Available columns: {columns}`	Ausgabe in resolve_score_id_column	Step_4_0_Sentinel2_inventory.py:167
`FileNotFoundError: Sentinel-2 score CSV not found: {score_csv}`	`not score_csv.is_file()`	Step_4_0_Sentinel2_inventory.py:185
`ValueError: Missing required columns in Sentinel-2 score CSV: {sorted(missing_columns)}. Available columns: {df.columns.tolist()}`	`missing_columns`	Step_4_0_Sentinel2_inventory.py:199
`KeyError: Missing sentinel2_download section in config.`	`'sentinel2_download' not in config`	Step_4_1_Sentinel2_download.py:48
`KeyError: No ID column found in Sentinel-2 metadata CSV: {metadata_csv}`	`id_column is None`	Step_4_1_Sentinel2_download.py:133
`TimeoutError: Completed GEE exports not visible in Drive folder {folder_id}: {sample}`	`time.monotonic() >= deadline`	Step_4_1_Sentinel2_download.py:192
`RuntimeError: Google Drive credentials/token missing or interactive auth disabled`	`service is None`	Step_4_1_Sentinel2_download.py:442
`RuntimeError: --score-only requires sentinel2_download.gee_drive_export_enabled=true`	`args.score_only`	Step_4_1_Sentinel2_download.py:500
`KeyError: Missing sentinel2_cleaning section in config.`	`'sentinel2_cleaning' not in config`	Step_4_2_clean_sentinel2.py:20
`KeyError: No Dawn Chorus ID column found in {path}`	`id_column is None`	sentinel2_gee.py:78
`KeyError: Metadata must contain lat and lon/lng columns: {path}`	`'lat' not in frame or not ('lon' in frame or 'lng' in frame)`	sentinel2_gee.py:80
`KeyError: Metadata must contain datetime or datetime_utc: {path}`	`datetime_column is None`	sentinel2_gee.py:86
`KeyError: Score file must contain an ID column and score: {path}`	`column is None or 'score' not in frame`	sentinel2_gee.py:153
`KeyError: Incoming score rows must contain DC_id`	`'DC_id' not in update`	sentinel2_gee.py:198
`requests.HTTPError: Earth Engine HTTP {response.status_code}`	`response.status_code in {429, 500, 502, 503, 504}`	sentinel2_gee.py:332
`RuntimeError: GEE download failed for {target.name}: {last_error}`	Ausgabe in _download_image	sentinel2_gee.py:342
`GeeUnavailable: GEE service-account key not found: {key_path}`	`not key_path.is_file()`	sentinel2_gee.py:379
`GeeUnavailable: No GEE service-account key configured and stored Earth Engine credentials are unavailable`	`except Exception`	sentinel2_gee.py:390

Code / Meldung	Bedeutung / Ausloeser	Quelle
GeeUnavailable: No user OAuth token configured for legacy Earth Engine Drive exports	not token_value	sentinel2_gee.py:402
GeeUnavailable: GEE export OAuth token not found: {token_path}	not token_path.is_file()	sentinel2_gee.py:407
GeeUnavailable: Cannot read GEE export OAuth token: {token_path}	except (OSError, json.JSONDecodeError)	sentinel2_gee.py:414
GeeUnavailable: Invalid GEE export OAuth token: {token_path}	not isinstance(token_payload, dict)	sentinel2_gee.py:416
GeeUnavailable: GEE export OAuth token lacks required scopes: {missing}	granted and (not required_scopes.issubset(granted))	sentinel2_gee.py:440
GeeUnavailable: GEE export OAuth token has no refresh token	not credentials.refresh_token	sentinel2_gee.py:443
RuntimeError: Earth Engine score response has no DC_id column	'DC_id' not in scored_chunk	sentinel2_gee.py:491
GeeUnavailable: earthengine-api is not installed	except ImportError	sentinel2_gee.py:584
TimeoutError: Timed out waiting for legacy Sentinel-2 Drive exports (queued={len(queue)}, active={len(running)})	time.monotonic() >= export_deadline	sentinel2_gee.py:733
RuntimeError: {failed} legacy Sentinel-2 Drive export task(s) failed	failed	sentinel2_gee.py:757
GeeUnavailable: Direct GEE raster download is disabled until parity with the legacy export is verified	not bool(settings.get('gee_direct_download_verified', False))	sentinel2_gee.py:773
GeeUnavailable: earthengine-api is not installed	except ImportError	sentinel2_gee.py:779
RuntimeError: Earth Engine score response has no DC_id column	'DC_id' not in scored_chunk	sentinel2_gee.py:796
RuntimeError: Earth Engine score response has no DC_id column	'DC_id' not in scored	sentinel2_gee.py:806
RuntimeError: {failed} Sentinel-2 GEE tile downloads failed	failed	sentinel2_gee.py:857
FileNotFoundError: {label} missing: {path}	not path.is_file()	sentinel_credentials_preflight.py:16
RuntimeError: {label} is not valid JSON: {path}	except Exception	sentinel_credentials_preflight.py:20
RuntimeError: {label} must contain a JSON object: {path}	not isinstance(value, dict)	sentinel_credentials_preflight.py:22
RuntimeError: Missing Sentinel credential setting: {setting}	not value	sentinel_credentials_preflight.py:35
RuntimeError: GEE service-account key has the wrong structure	service_key.get('type') != 'service_account' or not service_key.get('client_email')	sentinel_credentials_preflight.py:84
RuntimeError: GEE user export token has no refresh_token	not export_token.get('refresh_token')	sentinel_credentials_preflight.py:88
RuntimeError: GEE user export token lacks Earth Engine or Drive scope	export_scopes and (not required_export_scopes.issubset(export_scopes))	sentinel_credentials_preflight.py:95
RuntimeError: Drive OAuth client has the wrong structure	not any((key in drive_credentials for key in ('installed', 'web')))	sentinel_credentials_preflight.py:101

Code / Meldung	Bedeutung / Ausloeser	Quelle
`RuntimeError: Drive read-only token has no refresh_token`	`not drive_token.get('refresh_token')`	sentinel_credentials_preflight.py:104

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`Ausgabe in main`	Step_4_0_Sentinel2_inventory.py:1239
`1`	`except Exception`	Step_4_0_Sentinel2_inventory.py:1246
`0`	`acquisition_mode in {'auto', 'gee_direct'}`	Step_4_1_Sentinel2_download.py:370
`0`	`args.score_only`	Step_4_1_Sentinel2_download.py:489
`0`	`Ausgabe in main`	Step_4_1_Sentinel2_download.py:670
`1`	`except Exception`	Step_4_1_Sentinel2_download.py:680
`0`	`Ausgabe in main`	Step_4_2_clean_sentinel2.py:86
`1`	`except Exception`	Step_4_2_clean_sentinel2.py:89
`0`	`not bool(settings.get('gee_drive_export_enabled', False))`	sentinel_credentials_preflight.py:55
`0`	`Ausgabe in main`	sentinel_credentials_preflight.py:112

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

Sentinel-Score genau `0`: `sentinel_has_issues=True`, Fehlercode `too_cloudy_sentinel_not_available` (zu bewoelkt; Sentinel-Daten nicht nutzbar). Zahlenwert und Dateiverfuegbarkeit bleiben erhalten. Positive niedrige Scores allein loesen diesen Code nicht aus.

35 Step 4 Sentinel-2 Mirror And Inventory (EN)

Readmes/step_4_sentinel2/README_EN.md

Purpose

Mirrors external Sentinel-2 Drive files and inventories GeoTIFFs.

Script

`scripts/Step_4_1_Sentinel2_download.py, scripts/Step_4_0_Sentinel2_inventory.py`

Inputs

- `Google Drive token.json`
- `PointData/S2`
- `PointData/S2_Scores.csv`

The score-table identifier is configured through `sentinel2_inventory.score_id_column`. The inventory also recognises the case-insensitive aliases `id`, `DC_id`, `dawn_chorus_id`, and `recording_id`.

Outputs

- `PointData/S2/*.tif`
- `outputs/step_4_0_Sentinel2_inventory/*`
- `outputs/step_4_1_sentinel2_download/*`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_4_1_sentinel2_mirror`, `step_4_0_sentinel2_inventory`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `sentinel2_download`, `sentinel2_inventory`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_4_1_Sentinel2_download.py --config config.horeka.json`
- `python scripts/Step_4_0_Sentinel2_inventory.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

The Drive log, file size and modification time control incremental processing.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

Code / Message	Meaning / Trigger	Source
`score_rows_with_invalid_id:{invalid_id_count}`	Score file contains rows with invalid IDs.	Step_4_0_Sentinel2_inventory.py:223
`duplicate_score_rows:{row_count}`	Multiple score rows for the same ID.	Step_4_0_Sentinel2_inventory.py:276
`non_numeric_or_missing_scores:{missing_numeric_count}`	Score values are nonnumeric or missing.	Step_4_0_Sentinel2_inventory.py:281
`scores_outside_0_1:{invalid_range_count}`	Numeric Sentinel scores lie outside 0 to 1.	Step_4_0_Sentinel2_inventory.py:286
`quality_score_missing`	No score is available for the recording.	Step_4_0_Sentinel2_inventory.py:291
`no_valid_quality_score`	No valid Sentinel score among existing rows.	Step_4_0_Sentinel2_inventory.py:294
`too_cloudy_sentinel_not_available`	Sentinel score is exactly 0: too cloudy, Sentinel data unavailable for use; a file may still exist.	Step_4_0_Sentinel2_inventory.py:297
`unexpected_raster_driver:{dataset.driver}`	Raster driver is not GTiff.	Step_4_0_Sentinel2_inventory.py:362
`invalid_dimensions:{dataset.width}x{dataset.height}`	Raster/image width or height is invalid.	Step_4_0_Sentinel2_inventory.py:367
`no_raster_bands`	Raster contains no bands.	Step_4_0_Sentinel2_inventory.py:372
`missing_crs`	Raster has no coordinate reference system.	Step_4_0_Sentinel2_inventory.py:375
`identity_geotransform`	Raster has an identity transform; check georeferencing.	Step_4_0_Sentinel2_inventory.py:378
`non_finite_bounds`	Raster bounds contain NaN or infinite values.	Step_4_0_Sentinel2_inventory.py:391
`invalid_bounds`	Raster bounds do not define a valid extent.	Step_4_0_Sentinel2_inventory.py:397
`raster_read_failed:{type(exc).__name__}:{exc}`	Raster could not be opened or read.	Step_4_0_Sentinel2_inventory.py:452
`all_pixels_nodata`	All inspected raster pixels are NoData.	Step_4_0_Sentinel2_inventory.py:474
`contains_nan_values:{nan_values}`	Raster contains NaN values.	Step_4_0_Sentinel2_inventory.py:477
`contains_infinite_values:{infinite_values}`	Raster contains infinite values.	Step_4_0_Sentinel2_inventory.py:482
`constant_raster:value={global_min}`	All valid raster values are identical; check plausibility.	Step_4_0_Sentinel2_inventory.py:487
`filename_does_not_contain_id_before_tif_extension`	TIFF filename does not supply the expected recording ID.	Step_4_0_Sentinel2_inventory.py:528
`source_stat_failed:{type(exc).__name__}:{exc}`	Filesystem stat failed for the source file.	Step_4_0_Sentinel2_inventory.py:536
`empty_file`	Existing file has zero bytes.	Step_4_0_Sentinel2_inventory.py:578
`duplicate_tif_files_for_id:{len(id_rows)}`	Multiple TIFF files are assigned to the same recording.	Step_4_0_Sentinel2_inventory.py:952
`quality_score_missing`	No score is available for the recording.	Step_4_0_Sentinel2_inventory.py:973

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`skipped_existing_immutable`	`dc_id in immutable_ids`	sentinel2_gee.py:666
`completed_existing_task`	`state == 'COMPLETED'`	sentinel2_gee.py:683
`ready`	`Emitted by run_gee_drive_exports`	sentinel2_gee.py:721
`score_only`	`score_only`	sentinel2_gee.py:748
`score_only`	`bool(getattr(args, 'score_only', False))`	sentinel2_gee.py:815
`skipped_existing`	`ok`	sentinel2_gee.py:837
`downloaded`	`ok`	sentinel2_gee.py:847
`validation_failed`	`not (ok)`	sentinel2_gee.py:850
`download_failed`	`except Exception`	sentinel2_gee.py:854

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`FileNotFoundError: Config file not found: {path}`	`not path.is_file()`	Step_4_0_Sentinel2_inventory.py:105
`TypeError: 'sentinel2_inventory' must be a JSON object.`	`not isinstance(section, dict)`	Step_4_0_Sentinel2_inventory.py:113
`NotADirectoryError: Sentinel-2 source directory not found: {source_dir}`	`not source_dir.is_dir()`	Step_4_0_Sentinel2_inventory.py:124
`ValueError: Missing Dawn Chorus ID column in Sentinel-2 score CSV. Tried {[candidate for candidate in candidates if candidate]}. Available columns: {columns}`	Emitted by resolve_score_id_column	Step_4_0_Sentinel2_inventory.py:167
`FileNotFoundError: Sentinel-2 score CSV not found: {score_csv}`	`not score_csv.is_file()`	Step_4_0_Sentinel2_inventory.py:185
`ValueError: Missing required columns in Sentinel-2 score CSV: {sorted(missing_columns)}. Available columns: {df.columns.tolist()}`	`missing_columns`	Step_4_0_Sentinel2_inventory.py:199
`KeyError: Missing sentinel2_download section in config.`	`'sentinel2_download' not in config`	Step_4_1_Sentinel2_download.py:48
`KeyError: No ID column found in Sentinel-2 metadata CSV: {metadata_csv}`	`id_column is None`	Step_4_1_Sentinel2_download.py:133
`TimeoutError: Completed GEE exports not visible in Drive folder {folder_id}: {sample}`	`time.monotonic() >= deadline`	Step_4_1_Sentinel2_download.py:192
`RuntimeError: Google Drive credentials/token missing or interactive auth disabled`	`service is None`	Step_4_1_Sentinel2_download.py:442
`RuntimeError: --score-only requires sentinel2_download.gee_drive_export_enabled=true`	`args.score_only`	Step_4_1_Sentinel2_download.py:500
`KeyError: Missing sentinel2_cleaning section in config.`	`'sentinel2_cleaning' not in config`	Step_4_2_clean_sentinel2.py:20
`KeyError: No Dawn Chorus ID column found in {path}`	`id_column is None`	sentinel2_gee.py:78
`KeyError: Metadata must contain lat and lon/lng columns: {path}`	`'lat' not in frame or not ('lon' in frame or 'lng' in frame)`	sentinel2_gee.py:80
`KeyError: Metadata must contain datetime or datetime_utc: {path}`	`datetime_column is None`	sentinel2_gee.py:86
`KeyError: Score file must contain an ID column and score: {path}`	`column is None or 'score' not in frame`	sentinel2_gee.py:153
`KeyError: Incoming score rows must contain DC_id`	`'DC_id' not in update`	sentinel2_gee.py:198
`requests.HTTPError: Earth Engine HTTP {response.status_code}`	`response.status_code in {429, 500, 502, 503, 504}`	sentinel2_gee.py:332
`RuntimeError: GEE download failed for {target.name}: {last_error}`	Emitted by _download_image	sentinel2_gee.py:342
`GeeUnavailable: GEE service-account key not found: {key_path}`	`not key_path.is_file()`	sentinel2_gee.py:379
`GeeUnavailable: No GEE service-account key configured and stored Earth Engine credentials are unavailable`	`except Exception`	sentinel2_gee.py:390

Code / Message	Meaning / Trigger	Source
`GeeUnavailable: No user OAuth token configured for legacy Earth Engine Drive exports`	`not token_value`	sentinel2_gee.py:402
`GeeUnavailable: GEE export OAuth token not found: {token_path}`	`not token_path.is_file()`	sentinel2_gee.py:407
`GeeUnavailable: Cannot read GEE export OAuth token: {token_path}`	`except (OSError, json.JSONDecodeError)`	sentinel2_gee.py:414
`GeeUnavailable: Invalid GEE export OAuth token: {token_path}`	`not isinstance(token_payload, dict)`	sentinel2_gee.py:416
`GeeUnavailable: GEE export OAuth token lacks required scopes: {missing}`	`granted and (not required_scopes.issubset(granted))`	sentinel2_gee.py:440
`GeeUnavailable: GEE export OAuth token has no refresh token`	`not credentials.refresh_token`	sentinel2_gee.py:443
`RuntimeError: Earth Engine score response has no DC_id column`	`'DC_id' not in scored_chunk`	sentinel2_gee.py:491
`GeeUnavailable: earthengine-api is not installed`	`except ImportError`	sentinel2_gee.py:584
`TimeoutError: Timed out waiting for legacy Sentinel-2 Drive exports (queued={len(queue)}, active={len(running)})`	`time.monotonic() >= export_deadline`	sentinel2_gee.py:733
`RuntimeError: {failed} legacy Sentinel-2 Drive export task(s) failed`	`failed`	sentinel2_gee.py:757
`GeeUnavailable: Direct GEE raster download is disabled until parity with the legacy export is verified`	`not bool(settings.get('gee_direct_download_verified', False))`	sentinel2_gee.py:773
`GeeUnavailable: earthengine-api is not installed`	`except ImportError`	sentinel2_gee.py:779
`RuntimeError: Earth Engine score response has no DC_id column`	`'DC_id' not in scored_chunk`	sentinel2_gee.py:796
`RuntimeError: Earth Engine score response has no DC_id column`	`'DC_id' not in scored`	sentinel2_gee.py:806
`RuntimeError: {failed} Sentinel-2 GEE tile downloads failed`	`failed`	sentinel2_gee.py:857
`FileNotFoundError: {label} missing: {path}`	`not path.is_file()`	sentinel_credentials_preflight.py:16
`RuntimeError: {label} is not valid JSON: {path}`	`except Exception`	sentinel_credentials_preflight.py:20
`RuntimeError: {label} must contain a JSON object: {path}`	`not isinstance(value, dict)`	sentinel_credentials_preflight.py:22
`RuntimeError: Missing Sentinel credential setting: {setting}`	`not value`	sentinel_credentials_preflight.py:35
`RuntimeError: GEE service-account key has the wrong structure`	`service_key.get('type') != 'service_account' or not service_key.get('client_email')`	sentinel_credentials_preflight.py:84
`RuntimeError: GEE user export token has no refresh_token`	`not export_token.get('refresh_token')`	sentinel_credentials_preflight.py:88
`RuntimeError: GEE user export token lacks Earth Engine or Drive scope`	`export_scopes and (not required_export_scopes.issubset(export_scopes))`	sentinel_credentials_preflight.py:95
`RuntimeError: Drive OAuth client has the wrong structure`	`not any((key in drive_credentials for key in ('installed', 'web')))`	sentinel_credentials_preflight.py:101

Code / Message	Meaning / Trigger	Source
`RuntimeError: Drive read-only token has no refresh_token`	`not drive_token.get('refresh_token')`	sentinel_credentials_preflight.py:104

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`Emitted by main`	Step_4_0_Sentinel2_inventory.py:1239
`1`	`except Exception`	Step_4_0_Sentinel2_inventory.py:1246
`0`	`acquisition_mode in {'auto', 'gee_direct'}`	Step_4_1_Sentinel2_download.py:370
`0`	`args.score_only`	Step_4_1_Sentinel2_download.py:489
`0`	`Emitted by main`	Step_4_1_Sentinel2_download.py:670
`1`	`except Exception`	Step_4_1_Sentinel2_download.py:680
`0`	`Emitted by main`	Step_4_2_clean_sentinel2.py:86
`1`	`except Exception`	Step_4_2_clean_sentinel2.py:89
`0`	`not bool(settings.get('gee_drive_export_enabled', False))`	sentinel_credentials_preflight.py:55
`0`	`Emitted by main`	sentinel_credentials_preflight.py:112

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

Sentinel quality score exactly `0` sets `sentinel_has_issues=True` and the issue code `too_cloudy_sentinel_not_available` (too cloudy; Sentinel data unavailable for use). The numeric score and physical file-presence flag are retained. Positive low scores alone do not trigger this code.

36 Step 5_2 HOSTRADA Weather Per Recording (DE)

Readmes/step_5_2_weather/README_DE.md

Zweck

Laedt/cached HOSTRADA und extrahiert Wetterzeitreihen pro Recording.

Script

`scripts/Step_5_2_download_weather_data.py`

Eingaben

- `outputs/step_1_metadata/dawnchorus_metadata_clean.csv`
- `DWD HOSTRADA`

Outputs

- `PointData/Weather/Hostrada/weather_<id>.csv`
- `outputs/step_5_2_weather_download/*`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_5_1_weather_inventory`, `step_5_2_weather_download`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `weather_inventory`, `weather_download`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python scripts/Step_5_2_download_weather_data.py --config config.horeka.json`

Batch- und Parallelisierungslogik

`SLURM_CPUS_PER_TASK` begrenzt die Parallelitaet innerhalb eines Jobs. Der Orchestrator teilt grosse ID-Mengen deterministisch auf bis zu acht `bio_step52`-Array-Tasks auf; standardmaessig laufen maximal vier Tasks gleichzeitig. Kleine inkrementelle Mengen bleiben ein einzelner Task. Gemeinsam genutzte Monats-NetCDFs besitzen einen LSDF-kompatiblen Download-Lock, damit parallele Tasks keine unvollstaendigen Cachedateien erzeugen. Nach dem Array bestaetigt `bio_step52verify`, dass fuer jede angeforderte, noch in den Metadaten vorhandene ID eine nichtleere Wetter-CSV geschrieben wurde.

Checkpoint/Resume

Nichtleere `weather_<id>.csv` und `_recording_status` werden wiederverwendet. Bei `--ids-file` ist diese ID-Liste verbindlich; alte Probleme ausserhalb des Run-Plans werden nicht unbeabsichtigt erneut verarbeitet. Fortschritt und ETA werden pro Shard in `progress_shard_<n>.json` gespeichert.

Qualitaetskontrolle

Das Wetterfenster umfasst `preceding_days` vollstaendige lokale Kalendertage vor der Aufnahme plus den gesamten Aufnahmetag in `input_timezone`. Bei zehn Vortagen sind das normalerweise 264 Stunden, bei einer Zeitumstellung 263 oder 265 Stunden. Download und Inventar verwenden dieselben Kalendergrenzen. Die CSV speichert weiterhin lokale Uhrzeiten ohne Offset. Das Inventar erwartet deshalb die fehlende Fruehjahrsstunde beziehungsweise die doppelte Herbststunde; zusaetzliche Duplikate und fehlende Messwerte bleiben Fehler. `weather_inventory.expected_rows` ist nur der Ersatzwert ohne Aufnahmezeit; sonst ergibt sich die Zeilenzahl aus dem konkreten Fenster.

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Code / Meldung	Bedeutung / Ausloeser	Quelle
`filename_does_not_match_weather_id_pattern`	Wetter-Dateiname entspricht nicht dem erwarteten ID-Muster.	Step_5_1_Weather_inventory.py:205
`invalid_metadata_datetime`	Aufnahmezeit aus Metadaten ist fuer das Wetterfenster ungueltig.	Step_5_1_Weather_inventory.py:216
`empty_file`	Vorhandene Datei hat null Bytes.	Step_5_1_Weather_inventory.py:245
`unexpected_row_count`	Zeilenzahl entspricht nicht dem erwarteten Wetterfenster.	Step_5_1_Weather_inventory.py:270
`missing_required_column`	Mindestens eine konfigurierte Pflichtspalte fehlt.	Step_5_1_Weather_inventory.py:277
`missing_value`	Wetterwerte fehlen; Hinweis bleibt im Code, setzt allein keine Issue-Flag. Benoetigte Variablen/Stunden pruefen.	Step_5_1_Weather_inventory.py:286
`unparseable_datetime`	Zeitstempel in der Datendatei sind nicht parsebar.	Step_5_1_Weather_inventory.py:294
`duplicate_datetime`	Mehr Zeitstempelduplikate als im lokalen DST-Fenster zulaessig.	Step_5_1_Weather_inventory.py:302
`unexpected_time_interval`	Zeitstempel einschliesslich ihrer Haeufigkeit entsprechen nicht dem vollstaendigen erwarteten lokalen Wetterfenster. Korrekte Sommer-/Winterzeitwechsel sind erlaubt; Code allein unterscheidet fehlende, zusaetzliche und verschobene Stunden nicht.	Step_5_1_Weather_inventory.py:316
`unexpected_time_window`	Zeitstempel decken nicht das berechnete lokale Wetterfenster ab.	Step_5_1_Weather_inventory.py:330
`missing_datetime_column`	Wetter-CSV enthaelt keine datetime-Spalte.	Step_5_1_Weather_inventory.py:334
`implausible_value`	Mindestens ein Wert verletzt konfigurierte Plausibilitaetsgrenzen.	Step_5_1_Weather_inventory.py:338
`read_error:{type(exc).__name__}`	Datendatei konnte nicht gelesen werden; Exception-Typ steht im Suffix.	Step_5_1_Weather_inventory.py:341
`missing_file`	Erwartete Datei fehlt.	Step_5_1_Weather_inventory.py:373

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`out_of_bounds`	`bounds_checked and (not bounds_ok)`	Step_5_2_download_weather_data.py:422
`upstream_unavailable`	`unavailable_source_months`	Step_5_2_download_weather_data.py:481
`ok`	`Ausgabe in process_recording`	Step_5_2_download_weather_data.py:482
`failed`	`except Exception`	Step_5_2_download_weather_data.py:485

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_5_1_Weather_inventory.py:76
`KeyError: Missing 'weather_inventory' section in config.`	`not isinstance(config.get('weather_inventory'), dict)`	Step_5_1_Weather_inventory.py:80
`ValueError: Metadata ID column '{id_column}' not found in {metadata_csv}.`	`id_column not in metadata.columns`	Step_5_1_Weather_inventory.py:133
`NotADirectoryError: Weather directory not found: {directory}`	`not directory.is_dir()`	Step_5_1_Weather_inventory.py:514
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_5_2_download_weather_data.py:98
`KeyError: Missing 'weather_download' section in config.json.`	`not isinstance(section, dict)`	Step_5_2_download_weather_data.py:104
`RuntimeError: Step 5_2 worker was not initialised.`	`_worker_transformer is None or _worker_preceding_days is None or _worker_input_timezone is None or (_worker_cache_dir is None) or (_worker_download_settings is None)`	Step_5_2_download_weather_data.py:532
`ValueError: Input CSV missing required column(s): {missing}.`	`missing`	Step_5_2_download_weather_data.py:579
`ValueError: --task-count must be at least 1.`	`args.task_count < 1`	Step_5_2_download_weather_data.py:812
`ValueError: --task-index must satisfy 0 <= index < task-count.`	`args.task_index < 0 or args.task_index >= args.task_count`	Step_5_2_download_weather_data.py:814

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`args.list_only`	Step_5_1_Weather_inventory.py:572
`0`	`Ausgabe in main`	Step_5_1_Weather_inventory.py:676
`1`	`except Exception`	Step_5_1_Weather_inventory.py:679
`1`	`missing`	Step_5_2_download_weather_data.py:847
`2`	`unavailable`	Step_5_2_download_weather_data.py:854
`0`	`args.verify_shards`	Step_5_2_download_weather_data.py:855
`1 if processed_failed or (bool(section.get('fail_on_upstream_unavailable', False)) and processed_upstream_unavailable) else 0`	`Ausgabe in main`	Step_5_2_download_weather_data.py:1269
`1`	`except Exception`	Step_5_2_download_weather_data.py:1288

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

37 Step 5_2 HOSTRADA Weather Per Recording (EN)

Readmes/step_5_2_weather/README_EN.md

Purpose

Downloads/caches HOSTRADA data and extracts weather time series per recording.

Script

`scripts/Step_5_2_download_weather_data.py`

Inputs

- `outputs/step_1_metadata/dawnchorus_metadata_clean.csv`
- `DWD HOSTRADA`

Outputs

- `PointData/Weather/Hostrada/weather_<id>.csv`
- `outputs/step_5_2_weather_download/*`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_5_1_weather_inventory`, `step_5_2_weather_download`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `weather_inventory`, `weather_download`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python scripts/Step_5_2_download_weather_data.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits parallelism inside one job. The orchestrator deterministically distributes large ID sets across up to eight `bio_step52` array tasks, with at most four tasks running concurrently by default. Small incremental sets remain a single task. Shared monthly NetCDF files use an LSDF-safe download lock. `bio_step52verify` subsequently checks that every requested ID still present in metadata has a non-empty weather CSV.

Checkpoint/Resume

Non-empty `weather_<id>.csv` files and `_recording_status` are reused. When an `--ids-file` is supplied it is authoritative, preventing unrelated historical problems from re-entering the worklist. Progress and ETA are written per shard to `progress_shard_<n>.json`.

Quality Control

The weather window includes `preceding_days` complete local calendar days before the recording plus the complete recording day in `input_timezone`. Ten preceding days normally give 264 hourly observations, or 263/265 across daylight-saving transitions. Download and inventory share the same calendar boundaries. The CSV retains local times without offsets. Inventory therefore expects the missing spring hour and repeated autumn hour, while still rejecting extra duplicates and missing observations. `weather_inventory.expected_rows` is only a fallback without a recording timestamp; otherwise the actual window sets the expected row count.

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

Code / Message	Meaning / Trigger	Source
`filename_does_not_match_weather_id_pattern`	Weather filename does not match the expected ID pattern.	Step_5_1_Weather_inventory.py:205
`invalid_metadata_datetime`	Metadata recording time is invalid for the weather window.	Step_5_1_Weather_inventory.py:216
`empty_file`	Existing file has zero bytes.	Step_5_1_Weather_inventory.py:245
`unexpected_row_count`	Row count does not match the expected weather window.	Step_5_1_Weather_inventory.py:270
`missing_required_column`	At least one configured required column is missing.	Step_5_1_Weather_inventory.py:277
`missing_value`	Weather values are missing; the code is informational and alone does not set an issue flag. Check required variables/hours.	Step_5_1_Weather_inventory.py:286
`unparseable_datetime`	Timestamps in the data file cannot be parsed.	Step_5_1_Weather_inventory.py:294
`duplicate_datetime`	More duplicate timestamps than allowed by the local DST window.	Step_5_1_Weather_inventory.py:302
`unexpected_time_interval`	Timestamps and their multiplicities do not match the complete expected local weather window. Correct DST transitions are allowed; this code alone does not distinguish missing, extra or shifted hours.	Step_5_1_Weather_inventory.py:316
`unexpected_time_window`	Timestamps do not cover the calculated local weather window.	Step_5_1_Weather_inventory.py:330
`missing_datetime_column`	Weather CSV has no datetime column.	Step_5_1_Weather_inventory.py:334
`implausible_value`	At least one value violates configured plausibility limits.	Step_5_1_Weather_inventory.py:338
`read_error:{type(exc).__name__}`	Data file could not be read; suffix identifies exception type.	Step_5_1_Weather_inventory.py:341
`missing_file`	Expected file is missing.	Step_5_1_Weather_inventory.py:373

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`out_of_bounds`	`bounds_checked and (not bounds_ok)`	Step_5_2_download_weather_data.py:422
`upstream_unavailable`	`unavailable_source_months`	Step_5_2_download_weather_data.py:481
`ok`	`Emitted by process_recording`	Step_5_2_download_weather_data.py:482
`failed`	`except Exception`	Step_5_2_download_weather_data.py:485

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_5_1_Weather_inventory.py:76
`KeyError: Missing 'weather_inventory' section in config.`	`not isinstance(config.get('weather_inventory'), dict)`	Step_5_1_Weather_inventory.py:80
`ValueError: Metadata ID column '{id_column}' not found in {metadata_csv}.`	`id_column not in metadata.columns`	Step_5_1_Weather_inventory.py:133
`NotADirectoryError: Weather directory not found: {directory}`	`not directory.is_dir()`	Step_5_1_Weather_inventory.py:514
`FileNotFoundError: Config file not found: {config_path}`	`not config_path.is_file()`	Step_5_2_download_weather_data.py:98
`KeyError: Missing 'weather_download' section in config.json.`	`not isinstance(section, dict)`	Step_5_2_download_weather_data.py:104
`RuntimeError: Step 5_2 worker was not initialised.`	`_worker_transformer is None or _worker_preceding_days is None or _worker_input_timezone is None or (_worker_cache_dir is None) or (_worker_download_settings is None)`	Step_5_2_download_weather_data.py:532
`ValueError: Input CSV missing required column(s): {missing}.`	`missing`	Step_5_2_download_weather_data.py:579
`ValueError: --task-count must be at least 1.`	`args.task_count < 1`	Step_5_2_download_weather_data.py:812
`ValueError: --task-index must satisfy 0 <= index < task-count.`	`args.task_index < 0 or args.task_index >= args.task_count`	Step_5_2_download_weather_data.py:814

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`args.list_only`	Step_5_1_Weather_inventory.py:572
`0`	`Emitted by main`	Step_5_1_Weather_inventory.py:676
`1`	`except Exception`	Step_5_1_Weather_inventory.py:679
`1`	`missing`	Step_5_2_download_weather_data.py:847
`2`	`unavailable`	Step_5_2_download_weather_data.py:854
`0`	`args.verify_shards`	Step_5_2_download_weather_data.py:855
`1 if processed_failed or (bool(section.get('fail_on_upstream_unavailable', False)) and processed_upstream_unavailable) else 0`	`Emitted by main`	Step_5_2_download_weather_data.py:1269
`1`	`except Exception`	Step_5_2_download_weather_data.py:1288

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

38 Step 5_3 HOSTRADA Monatsdaten (DE)

Readmes/step_5_3_hostrada_monthly/README_DE.md

Spiegelt die konfigurierten DWD-HOSTRADA-Monats-NetCDFs fuer Ta, Rh, Radiation, CloudCover, Winddirection und Windspeed.

Input: DWD Open Data URLs aus `config.horeka.json`.

Outputs:

- `outputs/step_5_3_hostrada_monthly_download/netcdf/<Variable>/*.nc`

- `outputs/step_5_3_hostrada_monthly_download/download_log.csv`

Vorhandene nichtleere Dateien werden uebersprungen. Step 5_4 haengt von diesem Step ab.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Code / Meldung	Bedeutung / Ausloeser	Quelle
`{type(exc).__name__}:{exc}`	Originaler Exception-Typ und Meldung; kein eigener statischer Fehlercode.	Step_5_3_download_hostrada_monthly.py:106

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`failed`	`except Exception`	Step_5_3_download_hostrada_monthly.py:105

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`KeyError: Missing hostrada_monthly_download section in config.`	`'hostrada_monthly_download' not in config`	Step_5_3_download_hostrada_monthly.py:24

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`1 if failed else 0`	`Ausgabe in main`	Step_5_3_download_hostrada_monthly.py:202
`1`	`except Exception`	Step_5_3_download_hostrada_monthly.py:205

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

39 Step 5_3 HOSTRADA Monthly Data (EN)

Readmes/step_5_3_hostrada_monthly/README_EN.md

Mirrors configured DWD HOSTRADA monthly NetCDF files for Ta, Rh, Radiation, CloudCover, Winddirection and Windspeed.

Input: DWD Open Data URLs from `config.horeka.json`.

Outputs:

- `outputs/step_5_3_hostrada_monthly_download/netcdf/<Variable>/*.nc`

- `outputs/step_5_3_hostrada_monthly_download/download_log.csv`

Existing non-empty files are skipped. Step 5_4 depends on this step.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

Code / Message	Meaning / Trigger	Source
`{type(exc).__name__}:{exc}`	Original exception type and message; not a separate fixed code.	Step_5_3_download_hostrada_monthly.py:106

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`failed`	`except Exception`	Step_5_3_download_hostrada_monthly.py:105

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`KeyError: Missing hostrada_monthly_download section in config.`	`'hostrada_monthly_download' not in config`	Step_5_3_download_hostrada_monthly.py:24

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`1 if failed else 0`	`Emitted by main`	Step_5_3_download_hostrada_monthly.py:202
`1`	`except Exception`	Step_5_3_download_hostrada_monthly.py:205

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

40 Step 5_4 HOSTRADA 100m Raster (DE)

Readmes/step_5_4_hostrada_rasters/README_DE.md

Der Hauptworkflow startet Step 5_4 als Slurm-Array: Bis zu zwei Slurm-Worker bearbeiten mehrere deterministische Variable/Jahr-Aufgaben nacheinander; jede logische Aufgabe verarbeitet genau eine Kombination. Unvollstaendige Jahre (weniger als zwoelf Monats-NetCDFs) werden ohne Fehler uebersprungen. Nach dem Array verifiziert ein kurzer Einzeljob die Tile-Statusdateien aller vollstaendigen Quelljahre, bevor Step 5_5 die Raster-QC ausfuehrt.

`tools/run_hostrada_raster_all.py` ist der Array-Dispatcher und ruft die Rasterlogik aus `scripts/Step_5_4_prepare_hostrada_rasters.py` auf.

Input:

- `outputs/step_5_3_hostrada_monthly_download/netcdf/<Variable>/*.nc`

Outputs:

- `outputs/step_5_4_hostrada_raster_products/Hostrada_<Variable>/*.tif`

- `.../_tile_status/<year>/*.json`

- `.../_force_state/<Variable>_<year>.json`

Im Vollauf werden alte Tile-Statusdateien pro Generation genau einmal invalidiert. Ein `in_progress`-Status ueberlebt neue Slurm-Submits, sodass nach einem Timeout fertiggestellte Tiles weiterverwendet werden.

Standardressourcen im Orchestrator: zwei parallele Array-Tasks mit je 8 CPUs und 32 GB RAM. Diese Werte lassen sich ohne Codeaenderung anpassen:

```
BIOOTON_STEP54_CPUS=8 \
BIOOTON_STEP54_MEMORY=32G \
BIOOTON_STEP54_MAX_CONCURRENT_TASKS=2 \
bash slurm_add_new_ids.sh
```

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`in_progress`	`Ausgabe in prepare_force_resume`	Step_5_4_prepare_hostrada_rasters.py:142
`complete`	`Ausgabe in finish_force_resume`	Step_5_4_prepare_hostrada_rasters.py:171

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`KeyError: Missing hostrada_raster_products section in config.`	`'hostrada_raster_products' not in config`	Step_5_4_prepare_hostrada_rasters.py:48
`ValueError: Unsafe tile status directory: {status_dir}`	`resolved_output not in resolved_status.parents`	Step_5_4_prepare_hostrada_rasters.py:137
`FileNotFoundError: Missing HOSTRADA file: {path}`	`not path.exists()`	Step_5_4_prepare_hostrada_rasters.py:223
`ValueError: Task index {args.task_index} outside 0..{len(jobs) - 1}`	`args.task_index < 0 or args.task_index >= len(jobs)`	run_hostrada_raster_all.py:131

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`Ausgabe in main`	Step_5_4_prepare_hostrada_rasters.py:388
`1`	`except Exception`	Step_5_4_prepare_hostrada_rasters.py:398
`0`	`args.task_count`	run_hostrada_raster_all.py:112
`1 if incomplete else 0`	`args.verify_array`	run_hostrada_raster_all.py:120
`0`	`not task_is_complete(input_dir, variable, year)`	run_hostrada_raster_all.py:140
`0`	`Ausgabe in main`	run_hostrada_raster_all.py:171

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

41 Step 5_4 HOSTRADA 100m Rasters (EN)

Readmes/step_5_4_hostrada_rasters/README_EN.md

The main workflow launches Step 5_4 as a Slurm array: up to two Slurm workers process multiple deterministic variable/year tasks sequentially; each logical task processes one combination. Incomplete years (fewer than twelve monthly NetCDF files) are skipped without error. A short single-job verification checks tile states for all complete source years before Step 5_5 runs raster QC.

`tools/run_hostrada_raster_all.py` is the array dispatcher and calls the raster logic in `scripts/Step_5_4_prepare_hostrada_rasters.py`.

Input:

- `outputs/step_5_3_hostrada_monthly_download/netcdf/<Variable>/*.nc`

Outputs:

- `outputs/step_5_4_hostrada_raster_products/Hostrada_<Variable>/*.tif`
- `.../_tile_status/<year>/*.json`
- `.../_force_state/<Variable>_<year>.json`

A full run invalidates old tile status once per generation. The persistent `in_progress` state survives a new Slurm submission, so completed tiles are reused after a timeout.

The orchestrator defaults to two parallel array tasks with 8 CPUs and 32 GB RAM each. Adjust them without code changes:

```
BIOOTON_STEP54_CPUS=8 \
BIOOTON_STEP54_MEMORY=32G \
BIOOTON_STEP54_MAX_CONCURRENT_TASKS=2 \
bash slurm_add_new_ids.sh
```

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`in_progress`	`Emitted by prepare_force_resume`	Step_5_4_prepare_hostrada_rasters.py:142
`complete`	`Emitted by finish_force_resume`	Step_5_4_prepare_hostrada_rasters.py:171

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`KeyError: Missing hostrada_raster_products section in config.`	`'hostrada_raster_products' not in config`	Step_5_4_prepare_hostrada_rasters.py:48
`ValueError: Unsafe tile status directory: {status_dir}`	`resolved_output not in resolved_status.parents`	Step_5_4_prepare_hostrada_rasters.py:137
`FileNotFoundError: Missing HOSTRADA file: {path}`	`not path.exists()`	Step_5_4_prepare_hostrada_rasters.py:223
`ValueError: Task index {args.task_index} outside 0..{len(jobs) - 1}`	`args.task_index < 0 or args.task_index >= len(jobs)`	run_hostrada_raster_all.py:131

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`Emitted by main`	Step_5_4_prepare_hostrada_rasters.py:388
`1`	`except Exception`	Step_5_4_prepare_hostrada_rasters.py:398
`0`	`args.task_count`	run_hostrada_raster_all.py:112
`1 if incomplete else 0`	`args.verify_array`	run_hostrada_raster_all.py:120
`0`	`not task_is_complete(input_dir, variable, year)`	run_hostrada_raster_all.py:140
`0`	`Emitted by main`	run_hostrada_raster_all.py:171

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

42 Step 5_5 HOSTRADA Raster-QC (DE)

Readmes/step_5_5_hostrada_raster_qc/README_DE.md

Prueft alle Rastervariablen rekursiv auf Lesbarkeit, Dimensionen, CRS, NoData und konstante beziehungsweise vollstaendig fehlende Zeilen/Spalten.

Input:

- `outputs/step_5_4_hostrada_raster_products/**/*.tif`

Outputs:

- `outputs/step_5_5_hostrada_raster_quality_check/hostrada_raster_quality.csv`
- `hostrada_raster_quality.json`
- `hostrada_raster_quality.md`
- `state.json`

Unveraenderte fehlerfreie Raster werden anhand Dateigroesse und mtime wiederverwendet. Alte Fehler werden erneut geprueft.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`ONLY_NODATA`	`valid.size == 0`	Step_5_5_check_hostrada_raster_products.py:104
`OK`	`not (valid.size == 0)`	Step_5_5_check_hostrada_raster_products.py:107

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`KeyError: Missing hostrada_raster_quality_check section in config.`	`'hostrada_raster_quality_check' not in config`	Step_5_5_check_hostrada_raster_products.py:26
`FileNotFoundError: No TIFF files found in {input_dir}`	`not tif_files`	Step_5_5_check_hostrada_raster_products.py:204

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`Ausgabe in main`	Step_5_5_check_hostrada_raster_products.py:297
`1`	`except Exception`	Step_5_5_check_hostrada_raster_products.py:300

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

43 Step 5_5 HOSTRADA Raster QC (EN)

Readmes/step_5_5_hostrada_raster_qc/README_EN.md

Recursively checks all raster variables for readability, dimensions, CRS, NoData and constant or entirely missing rows and columns.

Input:

- `outputs/step_5_4_hostrada_raster_products/**/*.tif`

Outputs:

- `outputs/step_5_5_hostrada_raster_quality_check/hostrada_raster_quality.csv`
- `hostrada_raster_quality.json`
- `hostrada_raster_quality.md`
- `state.json`

Unchanged clean rasters are reused by file size and mtime. Previous issues are checked again.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`ONLY_NODATA`	`valid.size == 0`	Step_5_5_check_hostrada_raster_products.py:104
`OK`	`not (valid.size == 0)`	Step_5_5_check_hostrada_raster_products.py:107

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`KeyError: Missing hostrada_raster_quality_check section in config.`	`'hostrada_raster_quality_check' not in config`	Step_5_5_check_hostrada_raster_products.py:26
`FileNotFoundError: No TIFF files found in {input_dir}`	`not tif_files`	Step_5_5_check_hostrada_raster_products.py:204

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`Emitted by main`	Step_5_5_check_hostrada_raster_products.py:297
`1`	`except Exception`	Step_5_5_check_hostrada_raster_products.py:300

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

44 Step 6 - Bioakustische Embeddings und Arteninferenz

Readmes/step_6_bioacoustics/README_DE.md

Aktueller Betrieb (2026-09-23)

Separater Start nach dem Kernlauf: `bash slurm_bioacoustics.sh` oder `bash run_horeka.sh bioacoustics`. Step 6.1 gleicht mit `--reconcile-all` alle aktuellen Metadaten-IDs ab; entfernte IDs bleiben nicht in der Worklist. Unveraenderte Work-Keys nutzen Checkpoints. Standard: 64 logische Shards je Modell, bis zu vier Worker mit je 4 CPUs, 24 GB und 12 Stunden. `require_all_models_complete=true` macht den vorgeschalteten Verify-Gate fuer alle sechs Modelle verbindlich; die anschliessende ID-QC unterscheidet weiterhin `required` und optionale Modelle. Das Master-Ende aendert nur Bioakustikfelder.

Zweck

Step 6 verarbeitet technisch valide Dawn-Chorus-Audiodateien mit Bacpipe. Er erzeugt modellbezogene Embeddings, segmentweise Artenvorhersagen, Deutschland-/Saison-Plausibilitaetskennzeichen, Aufnahme-Aggregate und einen kompakten QC-Status pro `dawn_chorus_id`.

Grundlage ist die offizielle Bacpipe-API. `run_pretrained_classifier`, Device und Klassifikatorschwelle werden explizit gesetzt und sind Teil des Modellfingerprints.

Die threshold- und Top-k-begrenzten Modellvorhersagen vor der Plausibilitaetspruefung bleiben erhalten. Plausibilitaetsfilter markieren Vorhersagen; das kanonische Rohvorhersageprodukt aus Step 6_3 wird nicht ueberschrieben.

Abhaengigkeiten

- Step 1 liefert den korrigierten Aufnahmezeitpunkt.

- Step 3_0_a post liefert das Audio-Inventar nach dem Download.

- Step 3_1_a liefert vorhandene Original-Audiodateien.

- Bacpipe laeuft in `.venv_bacpipe` mit Python 3.11.

- CPU-Jobs verwenden standardmaessig HoreKa `cpuonly`; eine GPU ist nicht

erforderlich.

Teilschritte und Outputs

Step 6_0: Modell-Preflight

`scripts/Step_6_0_bioacoustic_model_preflight.py` prueft Bacpipe-Version, Torch/CUDA, Modelle und Taxonomiereferenz. Vor der Modellinitialisierung ruft er explizit Bacpipes `ensure_models_exist` auf. Damit werden fehlende Checkpoint-Dateien einmalig aus `vskode/bacpipe_models` nach `bacpipe/model_checkpoints/` relativ zum aktiven Checkout geladen und bei Folgejobs wiederverwendet. Pflichtmodelle muessen danach initialisierbar sein; optionale Modelle werden ebenfalls bereitgestellt. In der HoreKa-Konfiguration muessen alle sechs Modelle initialisierbar sein, bevor Step 6_1 oder Step 6_2 startet. Der Preflight wird fuer jeden Lauf mit geplanter Bioakustik-Inferenz erneut ausgefuehrt; ein veraltetes Registry-JSON kann ihn nicht mehr ueberspringen.

Ein vorhandener Ordner gilt nicht automatisch als gueltiger Checkpoint. Bei typischen Fehlern wie fehlenden Keras-Dateien, abgeschnittenen PyTorch-Archiven oder ungueltigen ZIP-Zentralverzeichnissen verschiebt der Preflight den betroffenen Modellbaum nach `bacpipe/model_checkpoints/_quarantine/<zeit>_<modell>/`, laedt ihn einmal neu und initialisiert das Modell erneut. Die Reparatur wird im Modellregister protokolliert; die defekten Dateien bleiben zur Diagnose erhalten.

`outputs/step_6_0_bioacoustic_model_preflight/model_registry.json`

Step 6_1: Worklist

`scripts/Step_6_1_prepare_bioacoustic_worklist.py` waehlt nur probe-, decode- und dauer-validierte Audios. Pro ID wird genau eine valide Datei verwendet.

```
outputs/step_6_1_bioacoustic_worklist/worklist.csv
outputs/step_6_1_bioacoustic_worklist/worklist.parquet
outputs/step_6_1_bioacoustic_worklist/rejected_audio.csv
outputs/step_6_1_bioacoustic_worklist/state.json
```

Step 6_2: Embeddings und native Modelloutputs

`scripts/Step_6_2_generate_bioacoustic_embeddings.py` wird als Slurm-Array mit bis zu vier Slurm-Workern ausgefuehrt. Die 64 logischen Shards je Modell bleiben erhalten; jeder Worker bearbeitet mehrere Modell-/Shard-Aufgaben nacheinander. Jeder Task schreibt nach kleinen Batches atomare Parquet-Checkpoints. Native Klassenscores werden bereits hier auf den konfigurierten Mindestscore und Top-k pro Segment begrenzt, damit keine unkontrolliert grossen Zwischenprodukte entstehen.

Nach dem Array prueft ein eigener Verify-Job jeden erwarteten Pflichtmodell- Shard-Status und Worklist-Schluessel. Probleme optionaler Modelle blockieren nur dann nicht, wenn `require_all_models_complete` ausgeschaltet ist. Die aktuelle Horeka-Konfiguration setzt diesen Wert auf `true`: alle sechs Modelle muessen abschliessen. Erst dieser Verify-Job markiert Step 6_2 als vollstaendig abgeschlossen. Einzelne Array-Tasks koennen daher einen unvollstaendigen Full-Rebuild nicht versehentlich als fertig kennzeichnen. Bei einem Timeout bleiben deren Checkpoints erhalten; ein Folgelauf berechnet nur fehlende oder fehlgeschlagene Worklist-Eintraege neu. Die Verifikation schreibt zusaetzlich eine kompakte Aufschluesselung je Modell nach `step_6_2_bioacoustic_state/verification.json`. Step 6_3 bis Step 6_6 verwenden harte `afterok`-Abhaengigkeiten und starten nicht, wenn die Verifikation fehlschlaegt.

Die HoreKa-Standardwerte sind 64 Shards pro Modell, vier CPUs und 24 GB RAM pro Array-Task sowie 12 Stunden Walltime. Drei Minuten vor dem Walltime sendet Slurm `SIGUSR1`; der Task speichert seinen letzten Batch und markiert den Shard als `interrupted`.

```
outputs/step_6_2_bioacoustic_embeddings/model=<modell>/*.parquet
outputs/step_6_2_bioacoustic_native_predictions/model=<modell>/*.parquet
outputs/step_6_2_bioacoustic_state/model=<modell>/shard=<n>.json
```

Step 6_3: Vorhersagen normalisieren

`scripts/Step_6_3_normalise_species_predictions.py` vereinheitlicht Klassifikatorausgaben, wendet den konfigurierten Mindestscore an und behaelt Top-k Vorhersagen pro Segment.

```
outputs/step_6_3_species_predictions_raw/model=<modell>/predictions.parquet
```

Step 6_4: Deutschland und Saison

`scripts/Step_6_4_filter_germany_taxonomy.py` verwendet:

```
reference_data/germany_species_allowlist.csv
```

Die ausgelieferte Datei ist bewusst nur ein Template. Bis eine fachlich freigegebene Referenz eingetragen ist, lautet der Status `not_evaluated`.

```
outputs/step_6_4_species_predictions_germany/model=<modell>/predictions.parquet
```

Step 6_5: Aufnahme-Aggregation

`scripts/Step_6_5_aggregate_bioacoustic_results.py` aggregiert Segmente und Modelle. Modellkonfidenzen werden nicht miteinander gemittelt.

```
outputs/step_6_5_bioacoustic_recording_summary/recording_summary.csv
outputs/step_6_5_bioacoustic_recording_summary/recording_summary.parquet
outputs/step_6_5_bioacoustic_recording_summary/recording_species.parquet
```

Step 6_6: Qualitaetskontrolle

`scripts/Step_6_6_bioacoustic_quality_control.py` vergleicht erwartete und abgeschlossene Modelle. Pflichtmodelle bestimmen die Bioakustik-Readiness; die QC in Step 6_6 orientiert sich an `required`. Der vorgeschaltete Verify-Gate kann mit `require_all_models_complete=true` strenger sein.

```
outputs/step_6_6_bioacoustic_quality_control/bioacoustic_qc_compact.csv
outputs/step_6_6_bioacoustic_quality_control/bioacoustic_qc_detailed.csv
outputs/step_6_6_bioacoustic_quality_control/state.json
```

Modelle

Pflichtmodelle sind `birdnet`, `perch_v2`, `audioprotopnet` und `convnext_birdset`. `insect66` und `naturebeats` sind optionale Embedding-Modelle. Fledermausmodelle sind fuer normale Smartphone-Aufnahmen nicht aktiviert, weil deren Samplingrate typische Ultraschallrufe nicht zuverlaessig abbildet.

Die Modellgewichte werden im Preflight explizit bereitgestellt und danach instanziiert. Schlaegt die Initialisierung eines Pflichtmodells fehl, startet das Inferenzarray nicht. Ein fehlgeschlagener Registry-Status wird im naechsten `bioacoustics`-Lauf erneut versucht.

Resume

Der Verarbeitungsschluessel enthaelt Audiofingerprint, Modellfingerprint, Bacpipe-/Preprocessing-Version und Modellparameter. Abgeschlossene IDs werden pro Modell/Shard gespeichert. Nach einem Timeout laufen nur offene IDs weiter; Inference-Fehler bleiben offen und werden beim naechsten Lauf erneut versucht.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Code / Meldung	Bedeutung / Ausloeser	Quelle
`bacpipe_import_failed:{type(exc).__name__}:{exc}`	Bacpipe konnte nicht importiert werden.	Step_6_0_bioacoustic_model_preflight.py:117
`bacpipe_version_mismatch:expected={expected_version}:actual={bacpipe_version}`	Installierte und konfigurierte Bacpipe-Version unterscheiden sich.	Step_6_0_bioacoustic_model_preflight.py:121
`torch_import_failed:{type(exc).__name__}:{exc}`	PyTorch konnte nicht importiert werden.	Step_6_0_bioacoustic_model_preflight.py:137
`model_checkpoint_dir_not_accessible:{checkpoint_dir}`	Checkpoint-Verzeichnis hat nicht die benoetigten Zugriffsrechte.	Step_6_0_bioacoustic_model_preflight.py:144
`bacpipe_ensure_models_exist_unavailable`	Bacpipe bietet die erwartete Checkpoint-Bereitstellung nicht an.	Step_6_0_bioacoustic_model_preflight.py:165
`{type(repair_exc).__name__}:{repair_exc}`	Originaler Exception-Typ und Meldung; kein eigener statischer Fehlercode.	Step_6_0_bioacoustic_model_preflight.py:223
`required_model_initialisation_failed:{name}`	Pflichtmodell bzw. unter require_all_models jedes Modell kann nicht initialisiert werden.	Step_6_0_bioacoustic_model_preflight.py:232
`inventory_issue`	Worklist verwirft Audio; Detailgruende aus dem Inventar fehlen.	Step_6_1_prepare_bioacoustic_worklist.py:86
`duplicate_valid_audio_for_id`	Weitere valide Audiodatei fuer dieselbe ID; nur die ausgewaehlte Datei kommt in die Worklist.	Step_6_1_prepare_bioacoustic_worklist.py:103
`missing_state`	Erwartete Shard-State-Datei fehlt oder ist nicht nutzbar.	Step_6_2_generate_bioacoustic_embeddings.py:116
`state={state.get('status', 'unknown')}`	Shard-State hat nicht den erwarteten Status complete.	Step_6_2_generate_bioacoustic_embeddings.py:121
`failed_ids={len(state_failed)}`	Shard-State enthaelt fehlgeschlagene IDs.	Step_6_2_generate_bioacoustic_embeddings.py:125
`task_error`	Shard-State enthaelt einen Fehler des gesamten Tasks.	Step_6_2_generate_bioacoustic_embeddings.py:127
`shard_count_mismatch`	Gespeicherte und konfigurierte Shardzahl unterscheiden sich.	Step_6_2_generate_bioacoustic_embeddings.py:129
`missing_rows={shard_missing}`	Erwartete Work-Keys sind im Shard nicht abgeschlossen.	Step_6_2_generate_bioacoustic_embeddings.py:146
`required_models_incomplete`	Mindestens ein Pflichtmodell ist fuer die Aufnahme nicht abgeschlossen.	Step_6_6_bioacoustic_quality_control.py:91
`model_inference_failed`	Mindestens ein Modell meldet einen Inferenzfehler fuer die ID.	Step_6_6_bioacoustic_quality_control.py:93

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
started	checkpoint_error_is_repairable(exc)	Step_6_0_bioacoustic_model_preflight.py:195
repaired	checkpoint_error_is_repairable(exc)	Step_6_0_bioacoustic_model_preflight.py:213
failed	except Exception	Step_6_0_bioacoustic_model_preflight.py:222
failed	Ausgabe in main	Step_6_0_bioacoustic_model_preflight.py:243
has_warnings	Ausgabe in main	Step_6_0_bioacoustic_model_preflight.py:243
validated	Ausgabe in main	Step_6_0_bioacoustic_model_preflight.py:243
rejected	Ausgabe in build_worklist	Step_6_1_prepare_bioacoustic_worklist.py:85
rejected	not duplicate_audio.empty	Step_6_1_prepare_bioacoustic_worklist.py:102
complete	Ausgabe in main	Step_6_1_prepare_bioacoustic_worklist.py:222
complete	Ausgabe in verify_shards	Step_6_2_generate_bioacoustic_embeddings.py:150
incomplete	Ausgabe in verify_shards	Step_6_2_generate_bioacoustic_embeddings.py:150
failed	Ausgabe in verify_shards	Step_6_2_generate_bioacoustic_embeddings.py:184
raw_model_prediction	Ausgabe in normalise_model	Step_6_3_normalise_species_predictions.py:68
complete	Ausgabe in main	Step_6_3_normalise_species_predictions.py:105
not_evaluated	allowlist.empty	Step_6_4_filter_germany_taxonomy.py:120
accepted	Ausgabe in apply_filter	Step_6_4_filter_germany_taxonomy.py:147
flagged	Ausgabe in apply_filter	Step_6_4_filter_germany_taxonomy.py:148
not_evaluated	Ausgabe in apply_filter	Step_6_4_filter_germany_taxonomy.py:153
complete	Ausgabe in main	Step_6_4_filter_germany_taxonomy.py:197
complete	Ausgabe in main	Step_6_5_aggregate_bioacoustic_results.py:121
validated	Ausgabe in build_qc	Step_6_6_bioacoustic_quality_control.py:94
partial	Ausgabe in build_qc	Step_6_6_bioacoustic_quality_control.py:94
failed	Ausgabe in build_qc	Step_6_6_bioacoustic_quality_control.py:94
complete	Ausgabe in main	Step_6_6_bioacoustic_quality_control.py:149

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`AttributeError: bacpipe.ensure_models_exist is unavailable`	`not callable(ensure_models)`	Step_6_0_bioacoustic_model_preflight.py:177
`FileNotFoundError: Audio inventory not found: {inventory_path}`	`not inventory_path.is_file()`	Step_6_1_prepare_bioacoustic_worklist.py:151
`FileNotFoundError: Bioacoustic model registry not found: {registry_path}`	`not registry_path.is_file()`	Step_6_1_prepare_bioacoustic_worklist.py:162
`RuntimeError: Bioacoustic model preflight registry has failed status.`	`registry.get('status') == 'failed'`	Step_6_1_prepare_bioacoustic_worklist.py:167
`ValueError: Unexpected empty embedding shape: {array.shape}`	`array.ndim != 2 or not array.size`	Step_6_2_generate_bioacoustic_embeddings.py:245
`ValueError: Classifier model returned no interpretable class-score output.`	`bool(work.get('classifier_available')) and (not native_rows)`	Step_6_2_generate_bioacoustic_embeddings.py:624
`KeyError: Taxonomy allowlist missing columns: {sorted(missing)}`	`missing`	Step_6_4_filter_germany_taxonomy.py:77
`KeyError: Missing required 'bioacoustics' configuration section.`	`not isinstance(section, dict)`	bioacoustics_common.py:63
`ValueError: Each bioacoustic model needs a non-empty name.`	`not name`	bioacoustics_common.py:75
`ValueError: Bioacoustic model names must be unique.`	`len(names) != len(set(names))`	bioacoustics_common.py:85
`KeyError: Missing bioacoustics.{key} in configuration.`	`not value`	bioacoustics_common.py:93
`ValueError: bioacoustics.model_checkpoint_dir must not be empty.`	`not raw`	bioacoustics_common.py:104
`ValueError: Bacpipe 1.3.1 requires model_checkpoint_dir to end in 'bacpipe/model_checkpoints'; got: {checkpoint_dir}`	`checkpoint_dir.name.lower() != 'model_checkpoints' or checkpoint_dir.parent.name.lower() != 'bacpipe'`	bioacoustics_common.py:118
`RuntimeError: Could not set bacpipe {attribute}={value}: {exc}`	`except Exception`	bioacoustics_common.py:180

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`1 if issues else 0`	`Ausgabe in main`	Step_6_0_bioacoustic_model_preflight.py:276
`0`	`Ausgabe in main`	Step_6_1_prepare_bioacoustic_worklist.py:237
`1`	`except Exception`	Step_6_1_prepare_bioacoustic_worklist.py:240
`2`	`not worklist_path.is_file()`	Step_6_2_generate_bioacoustic_embeddings.py:83
`2`	`blocking_models`	Step_6_2_generate_bioacoustic_embeddings.py:205
`0`	`Ausgabe in verify_shards`	Step_6_2_generate_bioacoustic_embeddings.py:207
`0`	`model_index >= len(models)`	Step_6_2_generate_bioacoustic_embeddings.py:433
`0`	`pending.empty`	Step_6_2_generate_bioacoustic_embeddings.py:501
`1`	`except Exception`	Step_6_2_generate_bioacoustic_embeddings.py:507
`2`	`except Exception`	Step_6_2_generate_bioacoustic_embeddings.py:528
`3`	`stop_requested`	Step_6_2_generate_bioacoustic_embeddings.py:661
`0 if not failed_by_id else 2`	`Ausgabe in main`	Step_6_2_generate_bioacoustic_embeddings.py:676
`0`	`Ausgabe in main`	Step_6_3_normalise_species_predictions.py:115
`1`	`except Exception`	Step_6_3_normalise_species_predictions.py:118
`0`	`Ausgabe in main`	Step_6_4_filter_germany_taxonomy.py:214
`1`	`except Exception`	Step_6_4_filter_germany_taxonomy.py:217
`0`	`Ausgabe in main`	Step_6_5_aggregate_bioacoustic_results.py:132
`1`	`except Exception`	Step_6_5_aggregate_bioacoustic_results.py:135
`0`	`Ausgabe in main`	Step_6_6_bioacoustic_quality_control.py:161
`1`	`except Exception`	Step_6_6_bioacoustic_quality_control.py:164

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

45 Step 6 - Bioacoustic Embeddings and Species Inference

Readmes/step_6_bioacoustics/README_EN.md

Current Operation (2026-09-23)

Start separately after the core: `bash slurm_bioacoustics.sh` or `bash run_horeka.sh bioacoustics`. Step 6.1 reconciles all current metadata IDs using `--reconcile-all`, removing deleted IDs from the worklist. Unchanged work keys reuse checkpoints. Defaults: 64 logical shards per model, at most four workers with 4 CPUs, 24 GB and 12 hours each. `require_all_models_complete=true` makes the preceding verification gate require all six models, while per-ID QC still distinguishes required and optional models. The final master update changes bioacoustic fields only.

Step 6 processes technically valid Dawn Chorus audio with Bacpipe. It creates model-specific embeddings, segment-level species predictions, Germany/season plausibility flags, recording summaries and compact QC per `dawn_chorus_id`.

The implementation follows the official Bacpipe API. `run_pretrained_classifier`, device and classifier threshold are set explicitly and are included in the model fingerprint.

The substeps are:

- `6_0`: provision checkpoint files, then validate Bacpipe, CUDA and the model registry.

- `6_1`: select valid audio and create audio/model fingerprints.

- `6_2`: run checkpointed CPU arrays over model x deterministic ID shard,

followed by a verification gate over every required-model shard and work key; optional-model gaps remain warnings only when require_all_models_complete is false.

- `6_3`: normalise classifier outputs and retain thresholded segment top-k.

- `6_4`: harmonise taxonomy and add Germany/season plausibility.

- `6_5`: aggregate segments and model support per recording and species.

- `6_6`: reconcile expected/completed models and write per-ID QC.

Outputs use matching `outputs/step_6_<n>_*` directories. Exact paths and thresholds are defined in the `bioacoustics` section of `config.horeka.json`.

Required models are `birdnet`, `perch_v2`, `audioprotopnet` and `convnext_birdset`. `insect66` and `naturebeats` are optional embedding models. The Germany allowlist is shipped as a header-only template under `reference_data/germany_species_allowlist.csv`; predictions remain `not_evaluated` until a scientifically reviewed reference is supplied.

Native classifier output is thresholded and limited to top-k per segment before it is written. The preflight explicitly calls Bacpipe's `ensure_models_exist` for every configured model and persists checkpoints under the configured `bacpipe/model_checkpoints/` directory relative to the active checkout. It then instantiates the models before the inference array can start. A failed required-model registry is automatically retried by the next `bioacoustics` run.

An existing directory is not assumed to be a valid checkpoint. On common missing-file, truncated PyTorch archive, or invalid ZIP errors, the preflight moves the affected model tree to `bacpipe/model_checkpoints/_quarantine/<time>_<model>/`, downloads it once again, and retries model initialisation. The repair is recorded in the model registry while the damaged files remain available for diagnosis.

Resume is keyed by audio fingerprint, model fingerprint and preprocessing version. Every model/shard state records completed and failed IDs. A timeout continues at the last checkpoint, while failed IDs are retried in later runs. Only the verification gate writes the completed Step 6_2 marker, so one successful array task cannot prematurely complete a full rebuild.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

Code / Message	Meaning / Trigger	Source
`bacpipe_import_failed:{type(exc).__name__}:{exc}`	Bacpipe could not be imported.	Step_6_0_bioacoustic_model_preflight.py:117
`bacpipe_version_mismatch:expected={expected_version}:actual={bacpipe_version}`	Installed and configured Bacpipe versions differ.	Step_6_0_bioacoustic_model_preflight.py:121
`torch_import_failed:{type(exc).__name__}:{exc}`	PyTorch could not be imported.	Step_6_0_bioacoustic_model_preflight.py:137
`model_checkpoint_dir_not_accessible:{checkpoint_dir}`	Checkpoint directory lacks required access permissions.	Step_6_0_bioacoustic_model_preflight.py:144
`bacpipe_ensure_models_exist_unavailable`	Expected Bacpipe checkpoint provisioning API is unavailable.	Step_6_0_bioacoustic_model_preflight.py:165
`{type(repair_exc).__name__}:{repair_exc}`	Original exception type and message; not a separate fixed code.	Step_6_0_bioacoustic_model_preflight.py:223
`required_model_initialisation_failed:{name}`	Required model, or any model under require_all_models, cannot initialise.	Step_6_0_bioacoustic_model_preflight.py:232
`inventory_issue`	Worklist rejects audio without more specific inventory details.	Step_6_1_prepare_bioacoustic_worklist.py:86
`duplicate_valid_audio_for_id`	Extra valid audio for an ID; only the selected file enters the worklist.	Step_6_1_prepare_bioacoustic_worklist.py:103
`missing_state`	Expected shard state file is missing or unusable.	Step_6_2_generate_bioacoustic_embeddings.py:116
`state={state.get('status', 'unknown')}`	Shard state is not the expected complete status.	Step_6_2_generate_bioacoustic_embeddings.py:121
`failed_ids={len(state_failed)}`	Shard state contains failed IDs.	Step_6_2_generate_bioacoustic_embeddings.py:125
`task_error`	Shard state contains a task-level error.	Step_6_2_generate_bioacoustic_embeddings.py:127
`shard_count_mismatch`	Stored and configured shard counts differ.	Step_6_2_generate_bioacoustic_embeddings.py:129
`missing_rows={shard_missing}`	Expected work keys are not completed in the shard.	Step_6_2_generate_bioacoustic_embeddings.py:146
`required_models_incomplete`	At least one required model is incomplete for the recording.	Step_6_6_bioacoustic_quality_control.py:91
`model_inference_failed`	At least one model reports an inference failure for the ID.	Step_6_6_bioacoustic_quality_control.py:93

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`started`	`checkpoint_error_is_repairable(exc)`	Step_6_0_bioacoustic_model_preflight.py:195
`repaired`	`checkpoint_error_is_repairable(exc)`	Step_6_0_bioacoustic_model_preflight.py:213
`failed`	`except Exception`	Step_6_0_bioacoustic_model_preflight.py:222
`failed`	`Emitted by main`	Step_6_0_bioacoustic_model_preflight.py:243
`has_warnings`	`Emitted by main`	Step_6_0_bioacoustic_model_preflight.py:243
`validated`	`Emitted by main`	Step_6_0_bioacoustic_model_preflight.py:243
`rejected`	`Emitted by build_worklist`	Step_6_1_prepare_bioacoustic_worklist.py:85
`rejected`	`not duplicate_audio.empty`	Step_6_1_prepare_bioacoustic_worklist.py:102
`complete`	`Emitted by main`	Step_6_1_prepare_bioacoustic_worklist.py:222
`complete`	`Emitted by verify_shards`	Step_6_2_generate_bioacoustic_embeddings.py:150
`incomplete`	`Emitted by verify_shards`	Step_6_2_generate_bioacoustic_embeddings.py:150
`failed`	`Emitted by verify_shards`	Step_6_2_generate_bioacoustic_embeddings.py:184
`raw_model_prediction`	`Emitted by normalise_model`	Step_6_3_normalise_species_predictions.py:68
`complete`	`Emitted by main`	Step_6_3_normalise_species_predictions.py:105
`not_evaluated`	`allowlist.empty`	Step_6_4_filter_germany_taxonomy.py:120
`accepted`	`Emitted by apply_filter`	Step_6_4_filter_germany_taxonomy.py:147
`flagged`	`Emitted by apply_filter`	Step_6_4_filter_germany_taxonomy.py:148
`not_evaluated`	`Emitted by apply_filter`	Step_6_4_filter_germany_taxonomy.py:153
`complete`	`Emitted by main`	Step_6_4_filter_germany_taxonomy.py:197
`complete`	`Emitted by main`	Step_6_5_aggregate_bioacoustic_results.py:121
`validated`	`Emitted by build_qc`	Step_6_6_bioacoustic_quality_control.py:94
`partial`	`Emitted by build_qc`	Step_6_6_bioacoustic_quality_control.py:94
`failed`	`Emitted by build_qc`	Step_6_6_bioacoustic_quality_control.py:94
`complete`	`Emitted by main`	Step_6_6_bioacoustic_quality_control.py:149

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`AttributeError: bacpipe.ensure_models_exist is unavailable`	`not callable(ensure_models)`	Step_6_0_bioacoustic_model_preflight.py:177
`FileNotFoundError: Audio inventory not found: {inventory_path}`	`not inventory_path.is_file()`	Step_6_1_prepare_bioacoustic_worklist.py:151
`FileNotFoundError: Bioacoustic model registry not found: {registry_path}`	`not registry_path.is_file()`	Step_6_1_prepare_bioacoustic_worklist.py:162
`RuntimeError: Bioacoustic model preflight registry has failed status.`	`registry.get('status') == 'failed'`	Step_6_1_prepare_bioacoustic_worklist.py:167
`ValueError: Unexpected empty embedding shape: {array.shape}`	`array.ndim != 2 or not array.size`	Step_6_2_generate_bioacoustic_embeddings.py:245
`ValueError: Classifier model returned no interpretable class-score output.`	`bool(work.get('classifier_available')) and (not native_rows)`	Step_6_2_generate_bioacoustic_embeddings.py:624
`KeyError: Taxonomy allowlist missing columns: {sorted(missing)}`	`missing`	Step_6_4_filter_germany_taxonomy.py:77
`KeyError: Missing required 'bioacoustics' configuration section.`	`not isinstance(section, dict)`	bioacoustics_common.py:63
`ValueError: Each bioacoustic model needs a non-empty name.`	`not name`	bioacoustics_common.py:75
`ValueError: Bioacoustic model names must be unique.`	`len(names) != len(set(names))`	bioacoustics_common.py:85
`KeyError: Missing bioacoustics.{key} in configuration.`	`not value`	bioacoustics_common.py:93
`ValueError: bioacoustics.model_checkpoint_dir must not be empty.`	`not raw`	bioacoustics_common.py:104
`ValueError: Bacpipe 1.3.1 requires model_checkpoint_dir to end in 'bacpipe/model_checkpoints'; got: {checkpoint_dir}`	`checkpoint_dir.name.lower() != 'model_checkpoints' or checkpoint_dir.parent.name.lower() != 'bacpipe'`	bioacoustics_common.py:118
`RuntimeError: Could not set bacpipe {attribute}={value}: {exc}`	`except Exception`	bioacoustics_common.py:180

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`1 if issues else 0`	`Emitted by main`	Step_6_0_bioacoustic_model_preflight.py:276
`0`	`Emitted by main`	Step_6_1_prepare_bioacoustic_worklist.py:237
`1`	`except Exception`	Step_6_1_prepare_bioacoustic_worklist.py:240
`2`	`not worklist_path.is_file()`	Step_6_2_generate_bioacoustic_embeddings.py:83
`2`	`blocking_models`	Step_6_2_generate_bioacoustic_embeddings.py:205
`0`	`Emitted by verify_shards`	Step_6_2_generate_bioacoustic_embeddings.py:207
`0`	`model_index >= len(models)`	Step_6_2_generate_bioacoustic_embeddings.py:433
`0`	`pending.empty`	Step_6_2_generate_bioacoustic_embeddings.py:501
`1`	`except Exception`	Step_6_2_generate_bioacoustic_embeddings.py:507
`2`	`except Exception`	Step_6_2_generate_bioacoustic_embeddings.py:528
`3`	`stop_requested`	Step_6_2_generate_bioacoustic_embeddings.py:661
`0 if not failed_by_id else 2`	`Emitted by main`	Step_6_2_generate_bioacoustic_embeddings.py:676
`0`	`Emitted by main`	Step_6_3_normalise_species_predictions.py:115
`1`	`except Exception`	Step_6_3_normalise_species_predictions.py:118
`0`	`Emitted by main`	Step_6_4_filter_germany_taxonomy.py:214
`1`	`except Exception`	Step_6_4_filter_germany_taxonomy.py:217
`0`	`Emitted by main`	Step_6_5_aggregate_bioacoustic_results.py:132
`1`	`except Exception`	Step_6_5_aggregate_bioacoustic_results.py:135
`0`	`Emitted by main`	Step_6_6_bioacoustic_quality_control.py:161
`1`	`except Exception`	Step_6_6_bioacoustic_quality_control.py:164

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

46 Step 7_0 - Finale Mastertabelle

Readmes/step_7_0_master_table/README_DE.md

Aktueller Betrieb (2026-10-12)

Der direkte Slurm-Start serialisiert Zwischenupdates. Der tmux-/Hybrid-Controller sammelt Abhaengigkeiten und schreibt einen finalen Master. Kern und lokaler Lauf verwenden denselben Writer mit Schema v7 (106 Spalten). `time_has_issues` ist nur bei Quelle `localtimes` False; `duration_seconds` kommt aus dem Audio-QC. Reine Dauerabweichungen und reine Wetter-`missing_value`-Codes bleiben Hinweise. Kein Clean-Filter entfernt Zeilen aus der Mastertabelle; 40 Sekunden sind eine spaetere Analysewahl. Inkrementelle und Bioakustik-Updates migrieren auch die Zeitflag aelterer Zeilen. Die vollstaendigen Regeln stehen in der Master-Referenz.

Zweck

Step 7_0 erzeugt eine kompakte Mastertabelle pro `dawn_chorus_id`. Sie fuehrt die wichtigsten Statusinformationen aus Metadaten, 100m-/10m-Formation, Audio, Fotos, Sentinel-2, HOSTRADA-Punktwetter und HOSTRADA-100m-Rasterstatus zusammen.

Input

```
outputs/step_1_metadata/dawnchorus_metadata_clean.csv
outputs/step_1_metadata/dawnchorus_metadata_log.csv
outputs/step_2_variants/<primary_suffix>/step_2_2/DawnChorus_LRT_Grid_Assignment_<primary_suffix>
.csv
outputs/step_2_variants/<primary_suffix>/step_2_4_susi_10m/Formation_Status_10m_Grid_withLRTCode_
<primary_suffix>.parquet
outputs/Bio_O_Ton_Formation_Variants.parquet
outputs/step_3_0_a_audio_inventory/audio_inventory_*.csv
outputs/step_3_0_b_photo_inventory/photo_inventory_*.csv
outputs/step_4_0_Sentinel2_inventory/sentinel2_inventory_*.csv
outputs/step_5_1_weather_inventory/weather_inventory_compact.csv
outputs/step_5_4_hostrada_raster_products/
outputs/step_5_5_hostrada_raster_quality_check/hostrada_raster_quality.csv
```

Output

```
Bio_O_Ton_Master.csv
Bio_O_Ton_Master.parquet
Bio_O_Ton_Master_summary.json
outputs/step_0_control/status_events.csv
Bio_O_Ton_Formation_Variants.csv
Bio_O_Ton_Formation_Variants.parquet
```

Die Output-Dateien liegen direkt unter `Data_automatisation_skripts/outputs`. Step 7_1 erzeugt die normalisierte Variantentabelle; Step 7_0 liest daraus kompakte Vollstaendigkeits- und Abdeckungszaehler.

Abhaengigkeiten

Der zentrale Slurm-Orchestrator reiht nach jedem relevanten Step-2-, Step-3-, Step-4-, Step-5- und Step-6-Ergebnis einen seriellen `bio_master_*`-Job ein. Bei ID-spezifischen Steps wird `--ids-file` verwendet: Nur diese Zeilen werden ersetzt oder bei fehlender ID in den Clean-Metadaten entfernt; alle anderen bestehenden Masterzeilen bleiben erhalten. Ein Vollupdate uebernimmt die gesamte aktuelle ID-Menge der Clean-Metadaten. Eine gueltige Datei nur mit Spaltenkopf ergibt eine leere Mastertabelle; eine fehlende Datei fuehrt zum Abbruch. Entfernte IDs werden auch bei Teillaeufen im Event-Log protokolliert. Nach globalen Grid- oder Rasterprodukten erfolgt ein Vollupdate. Der letzte Masterjob laeuft vor `bio_validate`.

Hinweise

Bei jedem Masterupdate (auch direkt nach dem Metadatenlauf) werden fehlende `grid_100m_id` aus dem vollstaendigen `point_lrt_assignment.grid_gpkg` ergaenzt, unabhaengig von vorhandenen Formationsprodukten. Die Zuordnung verwendet unveraendert die Projektion, `within`-Pruefung und Mehrfachtreffer-Auswahl aus Step 2.2. `grid_10m_id` wird weiterhin mit derselben bestehenden Funktion aus der 100-m-Elternzelle abgeleitet. Vorhandene 100-m-IDs bleiben erhalten. Ohne gueltige Koordinaten oder ausserhalb des Referenzrasters bleibt die ID leer. Ein konfiguriertes, nicht lesbares Raster fuehrt zum Fehler statt zu erfundenen IDs.

Eine bestehende CSV kann separat mit `tools/backfill_master_grid_ids.py` (`--input`, `--output`, `--grid-gpkg`) ergaenzt werden. Das Werkzeug prueft, dass vorhandene IDs und alle fachfremden Spaltenwerte unveraendert bleiben.

Die Mastertabelle ersetzt nicht die Detail-Logs. Sie verdichtet nur deren wichtigste ID-Level-Informationen. Die vollstaendige Spaltendokumentation steht in `MASTER_TABLE_README.md`. Kanonische Domaenenstatus, Quellfingerprint, Workflow-Run-ID und manuelle Freigabefelder werden in der Mastertabelle gespeichert; Statusaenderungen stehen im Event-Log.

`datetime_local` now stores the German clock without an offset (`YYYY-MM-DD HH:MM:SS`, schema v6). UTC is derived from the aware Step-1 product before removing that offset. A focused local refresh marks preserved weather/Sentinel results for rechecking when their relevant recording date changes. It does not regenerate those products.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Code / Meldung	Bedeutung / Ausloeser	Quelle
`time_source_not_localtimes`	Zeitquelle ist datetime, unbekannt oder fehlt; time_has_issues=True, kombinierte Clean-Flags=False.	Step_7_0_update_master_table.py:204
`recording_date_changed_recheck_required`	Lokales Aufnahmedatum geaendert; erhaltene Wetter-/Sentinel-Ergebnisse erneut pruefen.	Step_7_0_update_master_table.py:487
`inventory_not_run`	Kein verwendbares kompaktes Inventar vorhanden.	Step_7_0_update_master_table.py:544
`qc_not_run`	Raster vorhanden, aber QC-Ergebnis fehlt.	Step_7_0_update_master_table.py:559
`duration_unavailable`	Audiodauer konnte nicht bestimmt werden.	Step_7_0_update_master_table.py:611
`bioacoustic_qc_not_run`	Es liegt kein verwendbares kompaktes Bioakustik-QC vor.	Step_7_0_update_master_table.py:659
`bioacoustic_result_missing`	Fuer diese Master-ID fehlt ein QC-Ergebnis.	Step_7_0_update_master_table.py:695
`inventory_not_run`	Kein verwendbares kompaktes Inventar vorhanden.	Step_7_0_update_master_table.py:765
`missing_file`	Erwartete Datei fehlt.	Step_7_0_update_master_table.py:808
`quality_score_missing`	Fuer die Aufnahme liegt kein Score vor.	Step_7_0_update_master_table.py:813
`too_cloudy_sentinel_not_available`	Sentinel-Score ist genau 0: zu bewoelkt, Sentinel-Daten nicht nutzbar; eine Datei kann trotzdem vorhanden sein.	Step_7_0_update_master_table.py:815
`inventory_not_run`	Kein verwendbares kompaktes Inventar vorhanden.	Step_7_0_update_master_table.py:852
`qc_not_run`	Raster vorhanden, aber QC-Ergebnis fehlt.	Step_7_0_update_master_table.py:894
`missing_file`	Erwartete Datei fehlt.	Step_7_0_update_master_table.py:895
`missing_raster`	Keine erwarteten HOSTRADA-Raster vorhanden.	Step_7_0_update_master_table.py:932
`all_nodata`	Mindestens ein QC-Band besteht nur aus NoData.	Step_7_0_update_master_table.py:939
`unexpected_shape`	QC meldet ein nicht quadratisches Raster.	Step_7_0_update_master_table.py:941
`raster_structure_warning`	QC meldet konstante oder reine NoData-Zeilen/-Spalten.	Step_7_0_update_master_table.py:944
`raster_qc_read_error_{type(exc).__name__}`	Raster-QC-Datei konnte nicht gelesen werden.	Step_7_0_update_master_table.py:947
`qc_not_run`	Raster vorhanden, aber QC-Ergebnis fehlt.	Step_7_0_update_master_table.py:949
`timestamp_missing_or_invalid`	Mindestens ein benoetigter Zeitwert im Master fehlt oder ist ungueltig.	Step_7_0_update_master_table.py:1453
`time_source_not_localtimes`	Zeitquelle ist datetime, unbekannt oder fehlt; time_has_issues=True, kombinierte Clean-Flags=False.	Step_7_0_update_master_table.py:1455
`coordinates_missing_or_invalid`	Master-Koordinaten fehlen oder liegen ausserhalb gueltiger WGS84-Grenzen.	Step_7_0_update_master_table.py:1457
`{prefix}_missing`	Die benannte Datendomaene fehlt fuer diese Aufnahme.	Step_7_0_update_master_table.py:1462

Code / Meldung	Bedeutung / Ausloeser	Quelle
`{prefix}_issue`	Die benannte Datendomaene ist vorhanden, meldet aber Probleme.	Step_7_0_update_master_table.py:1464
`grid_100m_missing_majority_formation`	Keine 100-m-Mehrheitsformation; das bedeutet nicht zwingend fehlende Grid-ID.	Step_7_0_update_master_table.py:1466
`grid_10m_missing_majority_formation`	Keine 10-m-Mehrheitsformation; das bedeutet nicht zwingend fehlende Grid-ID.	Step_7_0_update_master_table.py:1468

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`partial`	`'record_status' in table`	Step_7_0_update_master_table.py:209
`not_started`	`'release_status' in table`	Step_7_0_update_master_table.py:211
`not_started`	`compact.empty`	Step_7_0_update_master_table.py:657
`majority_formation_status_10m`	`Ausgabe in add_10m_formation`	Step_7_0_update_master_table.py:1232
`validated`	`Ausgabe in add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1376
`has_issues`	`Ausgabe in add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1377
`missing`	`Ausgabe in add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1383
`partial`	`Ausgabe in add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1472
`not_started`	`Ausgabe in add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1488
`manual_review_required`	`Ausgabe in add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1489
`approved`	`Ausgabe in add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1492

Explizite Fehlermeldungen

Code / Meldung	Bedeutung / Ausloeser	Quelle
`FileNotFoundError: Step 1 clean metadata is missing or empty: {status_dir / 'dawnchorus_metadata_clean.csv'}`	`clean.empty`	Step_7_0_update_master_table.py:331
`ValueError: Run the core pipeline first: an existing master is required.`	`previous.empty`	Step_7_0_update_master_table.py:725
`ValueError: Expected HOSTRADA raster resolution_m=100 for the master table, got {resolution}.`	`resolution != 100`	Step_7_0_update_master_table.py:922
`ValueError: The INSPIRE grid has no CRS.`	`not info['crs']`	Step_7_0_update_master_table.py:1021
`ValueError: FORMATION_10M_UNREADABLE: {path}`	`except Exception`	Step_7_0_update_master_table.py:1200
`FileNotFoundError: Master update ID file not found: {args.ids_file}`	`not args.ids_file.is_file()`	Step_7_0_update_master_table.py:1669
`ValueError: FORMATION_10M_MISSING: {ten_path}; existing master retained.`	`ten_path and (not Path(ten_path).is_file())`	Step_7_0_update_master_table.py:1683
`ValueError: Proximity thresholds must be finite and positive`	`not all((math.isfinite(v) and v > 0 for v in (radius, seconds, cluster_radius, cluster_seconds)))`	spatiotemporal_duplicates.py:27
`ValueError: Cluster thresholds must include duplicate thresholds`	`radius > cluster_radius or seconds > cluster_seconds`	spatiotemporal_duplicates.py:29
`ValueError: Cluster radius cannot exceed half Earth's circumference`	`cluster_radius > math.pi * EARTH_RADIUS_M`	spatiotemporal_duplicates.py:31
`ValueError: Proximity input must contain unique recording IDs`	`len(set(ids)) != len(ids)`	spatiotemporal_duplicates.py:34
`ValueError: POINT_ASSIGNMENT_INCOMPLETE: {len(gaps)} recording IDs missing, duplicated or at different coordinates in {path}. Run Step 2.2 first; the existing master has not been replaced.`	`gaps`	input_consistency.py:54
`ValueError: SOURCE_POPULATION_REDUCED: source has {len(current)} IDs; {len(removed)} of {len(previous)} existing IDs would disappear. Restore the complete original source. For an intentional reduction set metadata_extraction.allow_large_source_reduction=true explicitly.`	`previous and len(removed) > max(10, len(previous) * 0.2) and (not settings.get('allow_large_source_reduction', False))`	input_consistency.py:73

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`execution_phase() == 'bioacoustics'`	Step_7_0_update_master_table.py:1664
`0`	`except ValueError`	Step_7_0_update_master_table.py:1688
`0`	`Ausgabe in main`	Step_7_0_update_master_table.py:1794
`1`	`except Exception`	Step_7_0_update_master_table.py:1797

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten.

`failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

Sentinel-Score genau `0`: `sentinel_has_issues=True`, Fehlercode `too_cloudy_sentinel_not_available` (zu bewoelkt; Sentinel-Daten nicht nutzbar). Zahlenwert und Dateiverfuegbarkeit bleiben erhalten. Positive niedrige Scores allein loesen diesen Code nicht aus.

47 Step 7_0 - Final Master Table

Readmes/step_7_0_master_table/README_EN.md

Current Operation (2026-10-12)

Direct Slurm submission serializes intermediate updates. The tmux/hybrid controller collects dependencies and writes one final master. Core and local runs share the v7 writer (106 columns). `time_has_issues` is false only for source `localtimes`; `duration_seconds` comes from audio QC. Pure duration deviations and weather `missing_value` codes remain informational. No clean-data filter removes master rows; 40 seconds is a later analysis choice. Incremental and bioacoustic updates also migrate time flags on older rows. Full rules are in the master reference.

Purpose

Step 7_0 creates a compact ID-level master table for each `dawn_chorus_id`. It combines the most important status fields from metadata, 100m/10m formation products, direct LRT intersections, audio, photos, Sentinel-2, HOSTRADA point weather/raster status, bioacoustic QC, readiness, and manual release state.

Input

```
outputs/step_1_metadata/dawnchorus_metadata_clean.csv
outputs/step_1_metadata/dawnchorus_metadata_log.csv
outputs/step_2_variants/<primary_suffix>/step_2_2/DawnChorus_LRT_Grid_Assignment_<primary_suffix>
.csv
outputs/step_2_variants/<primary_suffix>/step_2_4_susi_10m/Formation_Status_10m_Grid_withLRTCode_
<primary_suffix>.parquet
outputs/Bio_O_Ton_Formation_Variants.parquet
outputs/step_3_0_a_audio_inventory/audio_inventory_*.csv
outputs/step_3_0_b_photo_inventory/photo_inventory_*.csv
outputs/step_4_0_Sentinel2_inventory/sentinel2_inventory_*.csv
outputs/step_5_1_weather_inventory/weather_inventory_compact.csv
outputs/step_5_4_hostrada_raster_products/
outputs/step_5_5_hostrada_raster_quality_check/hostrada_raster_quality.csv
outputs/step_6_5_bioacoustic_recording_summary/recording_summary.csv
outputs/step_6_6_bioacoustic_quality_control/bioacoustic_qc_compact.csv
```

Output

```
Bio_O_Ton_Master.csv
Bio_O_Ton_Master.parquet
Bio_O_Ton_Master_summary.json
outputs/step_0_control/status_events.csv
Bio_O_Ton_Formation_Variants.csv
Bio_O_Ton_Formation_Variants.parquet
Bio_O_Ton_Variant_Summary.csv
Bio_O_Ton_Variant_Temporal_Summary.csv
```

The output files are written directly below `Data_automatisation_skripts/outputs`. Step 7_1 creates the normalized variant table; Step 7_0 reads it to add compact coverage and completeness counts.

Dependencies

The central Slurm orchestrator submits a serial `bio_master_*` job after each relevant Step 2, Step 3, Step 4, Step 5 and Step 6 result. ID-specific steps use `--ids-file`: only those rows are replaced or removed when absent from clean metadata; all other master rows are preserved. A full update adopts the complete current clean-metadata ID set. A valid header-only input produces an empty master; a missing input aborts the update. Deletions are logged for partial updates too. Global grid or raster products trigger a full update. The final master job runs before `bio_validate`.

Notes

The master table does not replace the detailed logs. It only condenses their most important ID-level information. All 104 columns and their exact readiness rules are defined in the master-table reference. Canonical domain statuses, the source fingerprint, workflow run ID, and preserved manual review fields are stored in the master table; status changes are written to the event log.

`datetime_local` now stores the German clock without an offset (`YYYY-MM-DD HH:MM:SS`, schema v6). UTC is derived from the aware Step-1 product before removing that offset. A focused local refresh marks preserved weather/Sentinel results for rechecking when their relevant recording date changes. It does not regenerate those products.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

Code / Message	Meaning / Trigger	Source
`time_source_not_localtimes`	Time source is datetime, unknown or absent; time_has_issues=True and combined clean readiness flags=False.	Step_7_0_update_master_table.py:204
`recording_date_changed_recheck_required`	Local recording date changed; recheck retained weather/Sentinel products.	Step_7_0_update_master_table.py:487
`inventory_not_run`	No usable compact inventory is available.	Step_7_0_update_master_table.py:544
`qc_not_run`	Raster exists but QC result is missing.	Step_7_0_update_master_table.py:559
`duration_unavailable`	Audio duration could not be determined.	Step_7_0_update_master_table.py:611
`bioacoustic_qc_not_run`	No usable compact bioacoustic QC is available.	Step_7_0_update_master_table.py:659
`bioacoustic_result_missing`	This master ID has no matching QC result.	Step_7_0_update_master_table.py:695
`inventory_not_run`	No usable compact inventory is available.	Step_7_0_update_master_table.py:765
`missing_file`	Expected file is missing.	Step_7_0_update_master_table.py:808
`quality_score_missing`	No score is available for the recording.	Step_7_0_update_master_table.py:813
`too_cloudy_sentinel_not_available`	Sentinel score is exactly 0: too cloudy, Sentinel data unavailable for use; a file may still exist.	Step_7_0_update_master_table.py:815
`inventory_not_run`	No usable compact inventory is available.	Step_7_0_update_master_table.py:852
`qc_not_run`	Raster exists but QC result is missing.	Step_7_0_update_master_table.py:894
`missing_file`	Expected file is missing.	Step_7_0_update_master_table.py:895
`missing_raster`	Expected HOSTRADA rasters are absent.	Step_7_0_update_master_table.py:932
`all_nodata`	At least one QC band contains only NoData.	Step_7_0_update_master_table.py:939
`unexpected_shape`	QC reports a nonsquare raster.	Step_7_0_update_master_table.py:941
`raster_structure_warning`	QC reports constant or all-NoData rows/columns.	Step_7_0_update_master_table.py:944
`raster_qc_read_error_{type(exc).__name__}`	Raster QC file could not be read.	Step_7_0_update_master_table.py:947
`qc_not_run`	Raster exists but QC result is missing.	Step_7_0_update_master_table.py:949
`timestamp_missing_or_invalid`	A required master timestamp is missing or invalid.	Step_7_0_update_master_table.py:1453
`time_source_not_localtimes`	Time source is datetime, unknown or absent; time_has_issues=True and combined clean readiness flags=False.	Step_7_0_update_master_table.py:1455
`coordinates_missing_or_invalid`	Master coordinates are missing or outside valid WGS84 bounds.	Step_7_0_update_master_table.py:1457

Code / Message	Meaning / Trigger	Source
`{prefix}_missing`	The named data domain is missing for the recording.	Step_7_0_update_master_table.py:1462
`{prefix}_issue`	The named data domain exists but reports issues.	Step_7_0_update_master_table.py:1464
`grid_100m_missing_majority_formation`	No 100 m majority formation; this does not necessarily mean no grid ID.	Step_7_0_update_master_table.py:1466
`grid_10m_missing_majority_formation`	No 10 m majority formation; this does not necessarily mean no grid ID.	Step_7_0_update_master_table.py:1468

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`partial`	`'record_status' in table`	Step_7_0_update_master_table.py:209
`not_started`	`'release_status' in table`	Step_7_0_update_master_table.py:211
`not_started`	`compact.empty`	Step_7_0_update_master_table.py:657
`majority_formation_status_10m`	`Emitted by add_10m_formation`	Step_7_0_update_master_table.py:1232
`validated`	`Emitted by add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1376
`has_issues`	`Emitted by add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1377
`missing`	`Emitted by add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1383
`partial`	`Emitted by add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1472
`not_started`	`Emitted by add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1488
`manual_review_required`	`Emitted by add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1489
`approved`	`Emitted by add_agreement_and_ready_flags`	Step_7_0_update_master_table.py:1492

Explicit Failure Messages

Code / Message	Meaning / Trigger	Source
`FileNotFoundError: Step 1 clean metadata is missing or empty: {status_dir / 'dawnchorus_metadata_clean.csv'}`	`clean.empty`	Step_7_0_update_master_table.py:331
`ValueError: Run the core pipeline first: an existing master is required.`	`previous.empty`	Step_7_0_update_master_table.py:725
`ValueError: Expected HOSTRADA raster resolution_m=100 for the master table, got {resolution}.`	`resolution != 100`	Step_7_0_update_master_table.py:922
`ValueError: The INSPIRE grid has no CRS.`	`not info['crs']`	Step_7_0_update_master_table.py:1021
`ValueError: FORMATION_10M_UNREADABLE: {path}`	`except Exception`	Step_7_0_update_master_table.py:1200
`FileNotFoundError: Master update ID file not found: {args.ids_file}`	`not args.ids_file.is_file()`	Step_7_0_update_master_table.py:1669
`ValueError: FORMATION_10M_MISSING: {ten_path}; existing master retained.`	`ten_path and (not Path(ten_path).is_file())`	Step_7_0_update_master_table.py:1683
`ValueError: Proximity thresholds must be finite and positive`	`not all((math.isfinite(v) and v > 0 for v in (radius, seconds, cluster_radius, cluster_seconds)))`	spatiotemporal_duplicates.py:27
`ValueError: Cluster thresholds must include duplicate thresholds`	`radius > cluster_radius or seconds > cluster_seconds`	spatiotemporal_duplicates.py:29
`ValueError: Cluster radius cannot exceed half Earth's circumference`	`cluster_radius > math.pi * EARTH_RADIUS_M`	spatiotemporal_duplicates.py:31
`ValueError: Proximity input must contain unique recording IDs`	`len(set(ids)) != len(ids)`	spatiotemporal_duplicates.py:34
`ValueError: POINT_ASSIGNMENT_INCOMPLETE: {len(gaps)} recording IDs missing, duplicated or at different coordinates in {path}. Run Step 2.2 first; the existing master has not been replaced.`	`gaps`	input_consistency.py:54
`ValueError: SOURCE_POPULATION_REDUCED: source has {len(current)} IDs; {len(removed)} of {len(previous)} existing IDs would disappear. Restore the complete original source. For an intentional reduction set metadata_extraction.allow_large_source_reduction=true explicitly.`	`previous and len(removed) > max(10, len(previous) * 0.2) and (not settings.get('allow_large_source_reduction', False))`	input_consistency.py:73

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`execution_phase() == 'bioacoustics'`	Step_7_0_update_master_table.py:1664
`0`	`except ValueError`	Step_7_0_update_master_table.py:1688
`0`	`Emitted by main`	Step_7_0_update_master_table.py:1794
`1`	`except Exception`	Step_7_0_update_master_table.py:1797

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a

pipeline lock only after inspecting its jobs.

Sentinel quality score exactly `0` sets `sentinel_has_issues=True` and the issue code `too_cloudy_sentinel_not_available` (too cloudy; Sentinel data unavailable for use). The numeric score and physical file-presence flag are retained. Positive low scores alone do not trigger this code.

48 Step 9 Visuelle Reports (DE)

Readmes/step_9_visual_reports/README_DE.md

Der Reportgenerator `tools/generate_pipeline_visual_reports.py` liest nur die kompakten Ergebnisdateien der abgeschlossenen bzw. teilweise abgeschlossenen Steps. Er erstellt keine neue Analyse und veraendert keine Quell- oder Fachprodukte.

Output:

- `outputs/step_9_visual_reports/index.html`: Gesamtuebersicht

- `outputs/step_9_visual_reports/01_step_1_metadata.html` bis

`07_step_7_mastertable.html`: einzelne Step-Seiten

- `outputs/step_9_visual_reports/report_manifest.json`: technische Liste der

erzeugten Seiten

Der Slurm-Workflow startet den Generator nach `final_validation`; die Ausfuehrung ist auch bei unvollstaendigen Steps robust. Zum lokalen Oeffnen kann die `index.html` direkt im Browser geoeffnet werden.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Statuswerte (nicht alle sind Fehler)

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Explizite Fehlermeldungen

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0`	`Ausgabe in main`	generate_pipeline_visual_reports.py:259

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

49 Step 9 Visual Reports (EN)

Readmes/step_9_visual_reports/README_EN.md

`tools/generate_pipeline_visual_reports.py` reads only compact outputs from completed or partial steps. It creates no new analysis and does not modify source or domain products.

Outputs:

- `outputs/step_9_visual_reports/index.html`: overview

- `outputs/step_9_visual_reports/01_step_1_metadata.html` through

`07_step_7_mastertable.html`: per-step pages

- `outputs/step_9_visual_reports/report_manifest.json`: technical page list

The Slurm workflow runs the generator after `final_validation`. It remains robust when some steps are incomplete; open `index.html` directly in a browser.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

No fixed values in this category in the associated source.

Status Values (Not All Are Errors)

No fixed values in this category in the associated source.

Explicit Failure Messages

No fixed values in this category in the associated source.

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0`	`Emitted by main`	generate_pipeline_visual_reports.py:259

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.

50 Zeitkonflikte und LRT-Zellen

Readmes/time_review/README.md

> Historischer Stand: Befunde und Zahlen gelten fuer den damaligen Audit/Lauf. Aktuelle Betriebsanweisungen stehen in der Haupt-README und Readmes/pipeline_phases.md.

Prüfung vom 16.09.2026, aktuelle Hauptvariante **no_K_post2017**.

Keine der 47 Aufnahmen mit widersprüchlichen Quellenzeiten liegt in einer 100-m-Zelle mit LRT-Flächen dieser Variante.

Prüfung	Ergebnis
Aufnahmen mit Zeitkonflikt	47
Zugeordnete, unterschiedliche INSPIRE-100-m-Zellen	29
Aufnahmen in einer Zelle mit LRT-Flächenüberlappung	0
Aufnahmen mit 100-m-Mehrheitsformation	0
Aufnahmepunkte innerhalb oder auf einer LRT-Polygongrenze	0

Die tatsächlichen 29 Zellgeometrien wurden aus dem INSPIRE-GeoPackage gelesen. Alle 47 Punkte liegen in ihrer zugeordneten Zelle. Für jede Zelle wurde der LRT-GeoPackage räumlich abgefragt und auf positive Flächenüberlappung sowie reine Grenzberührung geprüft. Auch reine Grenzberührungen kommen nicht vor. Zusätzlich wurden die Punktzuordnung, die 100-m-Mehrheitstabelle und die Master-Flags abgeglichen. Alle Prüfungen stimmen überein.

Ein vorhandener `grid_100m_id` bedeutet daher hier eine INSPIRE-Zuordnung, aber keine LRT-Zuordnung. Das Ergebnis gilt für `no_K_post2017`; andere LRT-Varianten wurden in dieser zusätzlichen Geometrieprüfung nicht untersucht. Die 47 Zeitkonflikte bleiben für andere Auswertungen relevant.

- Alle 47 Fälle mit Zeitwerten und LRT-Prüfung

- Alle 29 geprüften Zellen

- Prüfergebnis und verwendete Dateien

- Ausführlicher Zeitreport

- Zeitreport als PDF (Stand des lokalen Zeitlaufs vor Git-Übertragung)

- ci-tec-README

Reproduzierbare Geometrieprüfung mit den Pfaden einer passenden Konfiguration:

```
python tools/check_time_conflicts_lrt.py --config scripts_local_run/config.local.generated.json
--conflicts Readmes/time_review/recording_time_conflicts.csv --output-dir Readmes/time_review
```

Die großen Master-, Geometrie- und vollständigen Auditdateien bleiben im lokalen/LSDF-Datenbereich. Git enthält Code, Dokumentation, Prüfergebnisse und die kompakten Falllisten. Es wurden keine Horeka-Jobs gestartet.

51 Validation And Formation-Status Comparison (DE)

Readmes/validation_and_comparison/README_DE.md

Aktueller Betrieb (2026-09-23)

Der Abschlussbericht nennt `phase`. `core` fordert keine Step-6-Produkte; `bioacoustics` prueft Step 6 und seine Voraussetzungen. Ohne `BIOOTON_RUN_PLAN` prueft ein Direktaufruf den gesamten bisherigen Scope. `readiness_policy` und einzelne require_*-Schalter bleiben wirksam innerhalb der Phase.

Zweck

Vergleicht Formation-Status-Produkte und erzeugt finalen Validierungsreport.

Script

`tools/compare_formation_status_products.py, tools/final_validation_report.py`

Eingaben

- `outputs/step_2_*`
- `optional legacy/reference formation-status files`

Outputs

- `outputs/step_8_susi_compatibility/*`
- `outputs/step_9_validation/*`

Abhaengigkeiten und Invalidierung

Die logischen Schrittvertraege stehen in `pipeline_steps.json` unter `step_7_0_master_table`. Der zentrale Run-Planer gibt nur betroffene IDs weiter und plant globale Schritte nur bei geaenderten Inputs, ergebnisrelevanter Konfiguration oder fehlenden Outputs.

Konfiguration

Ergebnisrelevante Einstellungen stehen zentral in `config.horeka.json`: `master_table`, `final_validation`, `susi_sanity_check`. Datenpfade und fachliche Schwellen stammen aus der Konfiguration. Slurm-Ressourcen und BIOOTON_*-Overrides werden vom Launcher und optionalen Clusterprofil aufgeloest.

Ausfuehrung

`bash slurm_add_new_ids.sh` startet den inkrementellen Kernlauf; `bash slurm_from_scratch.sh` baut den Kern neu auf. Step 6 startet separat mit `bash slurm_bioacoustics.sh`. Ein isolierter technischer Direktlauf ist mit folgenden Befehlen moeglich:

- `python tools/compare_formation_status_products.py --config config.horeka.json`
- `python tools/final_validation_report.py --config config.horeka.json`

Batch- und Parallelisierungslogik

SLURM_CPUS_PER_TASK begrenzt die tatsaechliche Parallelitaet. Der Step verwendet nur die in der Konfiguration erlaubte Zahl von Prozessen/Workern. IDs oder Chunks besitzen eindeutige Status- bzw. Checkpoint-Schluessel; der globale Pipeline-Lock verhindert konkurrierende schreibende Gesamtlaeufe.

Checkpoint/Resume

Reports werden pro Lauf neu geschrieben; Inputs bleiben unveraendert.

Qualitaetskontrolle

Outputs gelten nicht allein wegen ihrer Existenz als gueltig. Kompakte und detaillierte Logs, Batch-Statusdateien und das Run-Manifest dokumentieren Validierung und Fehler. Der finale Gate wird mit `bash run_final_validation_report.sh` erzeugt; Formation-Produkte koennen zusaetzlich mit `bash slurm_compare_formation_status.sh` verglichen werden.

Status, Manifeste und Mastertabelle

Der Slurm-Orchestrator schreibt fuer diesen Step ein Manifest unter `outputs/step_0_manifests/<step>/<step_run_id>.json` mit `workflow_run_id`, Inputs, Parametern, Laufzeit, Logs und Outputs. Step 7 fasst die ID-bezogenen Ergebnisse in der Mastertabelle zusammen; technische Details bleiben in den Step-Logs. Kanonische Run-Statuswerte stehen in `scripts/common.py`; `schemas/status_model.json` definiert Status-Event-Felder.

Typische Fehler

Fehlende Inputs oder Konfigurationsabschnitte beenden den Step mit Exit-Code ungleich null. Datenprobleme einzelner IDs werden nach Moeglichkeit im Detail-/Retry-Log als `missing`, `has_issues` oder `failed` festgehalten. Nach einem Timeout wird derselbe Betriebsmodus erneut submitted; gueltige Checkpoints werden wiederverwendet.

Fehlercodes, Status und Exit-Codes

Abgleich mit dem lokalen Code: 2026-10-12. Die Tabellen trennen Daten-/QC-Codes, Statuswerte, explizite Exceptions und Prozess-Exit-Codes. `{...}` sind variable Werte, keine festen Codes. Bedingungen nennen den konkreten Ausloeser im Quellcode; mehrere Codes koennen gleichzeitig auftreten.

`0` bedeutet nur einen erfolgreichen Prozessabschluss, nicht automatisch fehlerfreie Daten. `argparse` kann bei ungueltigen Argumenten mit `2` abbrechen; ungefangene Python-Exceptions typischerweise mit `1`. Slurm-Zustaende wie `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` und `DependencyNeverSatisfied` sind Scheduler-Meldungen, keine fachlichen Fehlercodes. Dynamische Bibliotheks-/HTTP-/Dateisystemfehler stehen mit ihrer Originalmeldung im Log.

Daten- und QC-Codes

Code / Meldung	Bedeutung / Ausloeser	Quelle
`master_table_missing_or_empty`	Masterdatei fehlt oder ist leer.	final_validation_report.py:249
`master_table_read_error:{type(exc).__name__}`	Masterdatei konnte nicht gelesen werden.	final_validation_report.py:256
`master_missing_column:{column}`	Aktivierte Readiness-Pruefung benoetigt eine fehlende Master-Spalte.	final_validation_report.py:295
`{column}_not_ready:{not_ready}`	Unter strikter Readiness-Policy sind Zeilen fuer diese Anforderung nicht bereit.	final_validation_report.py:307

Statuswerte (nicht alle sind Fehler)

Code / Meldung	Bedeutung / Ausloeser	Quelle
`ok`	`Ausgabe in check_path`	final_validation_report.py:35
`missing`	`required and (not exists)`	final_validation_report.py:37
`empty`	`required and (not nonempty)`	final_validation_report.py:39
`optional_missing`	`not required and (not exists)`	final_validation_report.py:41
`missing_column`	`column not in table.columns`	final_validation_report.py:289
`ok`	`Ausgabe in master_readiness`	final_validation_report.py:301
`not_ready`	`Ausgabe in master_readiness`	final_validation_report.py:301
`validated`	`Ausgabe in main`	final_validation_report.py:435
`has_issues`	`critical or failed_manifest_steps or missing_planned_manifests or readiness['critical']`	final_validation_report.py:442
`ready`	`Ausgabe in main`	final_validation_report.py:446
`manual_review_required`	`Ausgabe in main`	final_validation_report.py:446
`approved`	`Ausgabe in main`	final_validation_report.py:449
`not_started`	`Ausgabe in main`	final_validation_report.py:449

Explizite Fehlermeldungen

Keine festen Werte dieser Kategorie im zugeordneten Quellcode.

Explizite Prozessrueckgaben

Code / Meldung	Bedeutung / Ausloeser	Quelle
`0 if technical_status == 'validated' else 2`	`Ausgabe in main`	final_validation_report.py:495
`0`	`Ausgabe in main`	compare_formation_status_products.py:509

Vorgehen: Zuerst das Step-Log und den Manifest-Eintrag des betreffenden Runs lesen, dann Detail-/Retry-/QC-Logs der betroffenen ID bzw. des Chunks. Konfiguration oder Quelldatei korrigieren und dieselbe Phase erneut starten. `failed`/`partial` nicht allein durch Existenz einer Datei als erledigt behandeln. Eine Pipeline-Sperre erst nach Pruefung der zugehoerigen Jobs freigeben.

52 Validation And Formation-Status Comparison (EN)

Readmes/validation_and_comparison/README_EN.md

Current Operation (2026-09-23)

The final report identifies its `phase`. Core does not require Step-6 products; bioacoustics checks Step 6 and its prerequisites. Without `BIOOTON_RUN_PLAN`, a direct command validates the legacy full scope. Readiness policy and individual require_* settings remain effective within the selected phase.

Purpose

Compares formation-status products and creates the final validation report.

Script

`tools/compare_formation_status_products.py, tools/final_validation_report.py`

Inputs

- `outputs/step_2_*`
- `optional legacy/reference formation-status files`

Outputs

- `outputs/step_8_susi_compatibility/*`
- `outputs/step_9_validation/*`

Dependencies And Invalidation

Logical step contracts are recorded in `pipeline_steps.json` under `step_7_0_master_table`. The central run planner passes only affected IDs and schedules global work only for changed inputs, result-relevant config or missing outputs.

Configuration

Result-relevant settings are centralised in `config.horeka.json`: `master_table`, `final_validation`, `susi_sanity_check`. Data paths and scientific thresholds come from the configuration. Slurm resource defaults and BIOOTON_* overrides are resolved by the launcher and optional cluster profile.

Execution

`bash slurm_add_new_ids.sh` starts the incremental core phase; `bash slurm_from_scratch.sh` rebuilds the core. Step 6 starts separately with `bash slurm_bioacoustics.sh`. An isolated technical direct run is available with:

- `python tools/compare_formation_status_products.py --config config.horeka.json`
- `python tools/final_validation_report.py --config config.horeka.json`

Batch And Parallel Execution

`SLURM_CPUS_PER_TASK` limits effective parallelism. The step uses no more processes/workers than configured. IDs or chunks have unique status/checkpoint keys, while the global pipeline lock prevents concurrent writing workflows.

Checkpoint/Resume

Reports are rebuilt for each run; their input products remain unchanged.

Quality Control

Output existence alone is not treated as validity. Compact and detailed logs, batch status files and the run manifest record validation and failures. `bash run_final_validation_report.sh` creates the final gate; formation products can additionally be compared with `bash slurm_compare_formation_status.sh`.

Status, Manifests And Master Table

The Slurm orchestrator writes a manifest under `outputs/step_0_manifests/<step>/<step_run_id>.json` containing the `workflow_run_id`, inputs, parameters, runtime, logs and outputs. Step 7 summarises ID-level results in the master table while technical detail remains in step logs. Canonical run statuses are defined in `scripts/common.py`; `schemas/status_model.json` defines status-event fields.

Typical Failures

Missing inputs or configuration sections terminate the step with a non-zero exit code. Per-ID data problems are recorded where possible in detail/retry logs as `missing`, `has_issues` or `failed`. After a timeout, resubmit the same mode; valid checkpoints are reused.

Error Codes, Status And Exit Codes

Checked against local code: 2026-10-12. Tables separate data/QC codes, statuses, explicit exceptions and process exit codes. `{...}` denotes runtime values rather than fixed codes. Conditions identify the exact source trigger; multiple codes may occur together.

`0` only means process success, not necessarily clean data. Invalid argparse arguments can exit with `2`; uncaught Python exceptions typically exit with `1`. Slurm states such as `TIMEOUT`, `OUT_OF_MEMORY`, `CANCELLED` and `DependencyNeverSatisfied` are scheduler messages, not data-quality codes. Dynamic library/HTTP/filesystem errors retain their original messages in logs.

Data And QC Codes

Code / Message	Meaning / Trigger	Source
`master_table_missing_or_empty`	Master file is missing or empty.	final_validation_report.py:249
`master_table_read_error:{type(exc).__name__}`	Master file could not be read.	final_validation_report.py:256
`master_missing_column:{column}`	An enabled readiness check requires a missing master column.	final_validation_report.py:295
`{column}_not_ready:{not_ready}`	Under strict readiness policy, rows do not meet this requirement.	final_validation_report.py:307

Status Values (Not All Are Errors)

Code / Message	Meaning / Trigger	Source
`ok`	`Emitted by check_path`	final_validation_report.py:35
`missing`	`required and (not exists)`	final_validation_report.py:37
`empty`	`required and (not nonempty)`	final_validation_report.py:39
`optional_missing`	`not required and (not exists)`	final_validation_report.py:41
`missing_column`	`column not in table.columns`	final_validation_report.py:289
`ok`	`Emitted by master_readiness`	final_validation_report.py:301
`not_ready`	`Emitted by master_readiness`	final_validation_report.py:301
`validated`	`Emitted by main`	final_validation_report.py:435
`has_issues`	`critical or failed_manifest_steps or missing_planned_manifests or readiness['critical']`	final_validation_report.py:442
`ready`	`Emitted by main`	final_validation_report.py:446
`manual_review_required`	`Emitted by main`	final_validation_report.py:446
`approved`	`Emitted by main`	final_validation_report.py:449
`not_started`	`Emitted by main`	final_validation_report.py:449

Explicit Failure Messages

No fixed values in this category in the associated source.

Explicit Process Returns

Code / Message	Meaning / Trigger	Source
`0 if technical_status == 'validated' else 2`	`Emitted by main`	final_validation_report.py:495
`0`	`Emitted by main`	compare_formation_status_products.py:509

Recovery: Read the step log and manifest for the affected run first, then the detail/retry/QC log for the ID or chunk. Correct the configuration or input and restart the same phase. File existence does not resolve `failed`/`partial` results. Release a pipeline lock only after inspecting its jobs.